University of California, Berkeley – Mechanical Engineering Department

ME249, Fall 2026 – Prof. V. P. Carey

Project 2

Genetic-algorithm modelling of nucleate boiling of a water/2-propanol mixture in variable gravity, and calibration of a heat-pipe heat-exchanger model

---

Adnan AHALI

Individual project (team of one)

Report due October 27, 2026

Summary

All tasks of Project 2 were completed, including those that individuals working alone may omit (Task 2.1(b), Task 2.3) and the extra tasks ET1 and ET2 assigned to teams of three. The genetic algorithm (GA) of the skeleton programs was kept, but one bug of the skeleton code and two data values had to be corrected. The main results are:

Task	Best-fit relation (GA)	Error (target)
2.2	$q'' = 1.72 \times 10^{-3}\,(T_w - T_{sat})^{2.9747}\,g^{0.0438}$	$[F_{err}]_{mean} = 0.02870$ (< 0.035)
2.3	$q'' = 1.20{\times}10^{-4}\,(T_w{-}T_{sat})^{3.0216}(g{+}1.3485\,g_{en}\gamma)^{0.4619}P^{0.5435}$	0.02097 (< 0.03); validation 0.02566
ET2	$(T_w - T_{sat}) = 20.27\,(q'')^{0.3074}(g + 1.308\,g_{en}\gamma)^{-0.1455}P^{-0.1485}$	0.00843
2.5	$(UA)_e = 5.705$ W/°C, $(UA)_c = 5.425$ W/°C, $\alpha = 1.130$	MAE 0.0166 / 0.0208 (< 0.045)

The three-constant power law of Task 2.2 fits the data only to ±16.6 % (rms), which is worse than the ±10 % data uncertainty, because it cannot describe the Marangoni effect. The five-constant model of Task 2.3 brings the overall deviation close to the noise level (12.3 %), although the $1\,g_{en}$ and $2\,g_{en}$ (5.5 kPa) groups keep a bias of 11–16 %. It shows that for $\gamma = 1.79$ the Marangoni flow acts like an extra gravity of about $2.4\,g_{en}$. For the heat pipe, the data determine only the overall conductance $UA_{eff} = 2.907$ W/°C, not $(UA)_e$ and $(UA)_c$ separately. About 300 GA runs show that the most important property of the initial guesses is that the output they predict has the right order of magnitude. Far from the solution, the GA either stagnates (steps that are too small along a narrow valley) or freezes completely (the relative selection threshold keeps no survivor). Larger mutations, followed by a refinement with small mutations, and a larger selection fraction solve these problems.

Contents

Summary **1**

1 Introduction and organisation of the work **3**
1.1 Division of the work . . . 3
1.2 Software and reproducibility . . . 3
1.3 The genetic algorithm of the skeleton program . . . 3

2 Part 1 – Nucleate boiling of a water/2-propanol mixture in variable gravity **4**
2.1 Task 2.1 – Data preparation and first look at the data . . . 4
2.2 Task 2.2 – Three-constant model $q'' = n_1(T_w - T_{sat})^{n_2} g^{n_3}$ . . . 5
2.3 Task 2.3 – Five-constant model $q'' = n_1(T_w - T_{sat})^{n_2}(g + n_4 g_{en}\gamma)^{n_3} P^{n_5}$ . . . 9
2.4 Extra Task ET1 – Fractional sensitivity of the heat flux to the inputs . . . 13
2.5 Extra Task ET2 – Inverse relation for the superheat . . . 15

3 Part 2 – Heat-pipe heat exchanger for an electronics cabinet **17**
3.1 Task 2.4 – Genetic algorithm program for the heat-pipe model . . . 17
3.2 Task 2.5 – Results . . . 17

4 Conclusions **21**
4.1 Effect of the initial guesses on the stability and speed of convergence . . . 21
4.2 Main results . . . 23

5 Summary of the program modifications (Task 2.6) 23

5.1 Changes to the basic structure of the example code (all GA programs) 23

5.2 Program-by-program modifications 24

A Programs 25

A.1 Program A – Task 2.1 (`P2_Task2.1.ipynb`) 25

A.2 Program B – Task 2.2 (`P2_Task2.2.ipynb`) 29

A.3 Program C – Task 2.3 (`P2_Task2.3.ipynb`) 36

A.4 Program D – Extra Task ET1 (`P2_ET1.ipynb`) 46

A.5 Program E – Extra Task ET2 (`P2_ET2.ipynb`) 47

A.6 Program F – Tasks 2.4 and 2.5 (`P2_Task2.4-2.5.ipynb`) 53

1 Introduction and organisation of the work

This report documents the work done for Project 2 of ME249. In Part 1, a genetic algorithm (GA) is used to extract, from nucleate boiling data obtained for a water/2-propanol mixture at gravity levels between $0.01\,g_{en}$ and $2\,g_{en}$, the dependence of the heat flux on wall superheat, gravity, Marangoni (surface-tension) effects and pressure. In Part 2, the same GA is adapted to calibrate a simple performance model of a heat-pipe heat exchanger that cools an electronics cabinet.

1.1 Division of the work

I worked alone on this project, so all tasks (programming, runs, analysis, figures and report) were done by me. The adjustment sheet for individuals working alone allows omitting part (b) of Task 2.1 and Task 2.3. The two extra tasks ET1 and ET2 are assigned to teams of three. **I still completed the whole project**: Tasks 2.1(a) and (b), 2.2, 2.3, ET1, ET2, 2.4, 2.5 and 2.6.

Table 1: Tasks, programs and where the results are reported. All programs are Jupyter notebooks in the folder `code/`. Their full listings are in Appendix A.

Task	Program (notebook)	Content	Section
2.1 (a), (b)	`P2_Task2.1.ipynb`	data check, log-log plot of q'' vs. superheat	2.1
2.2	`P2_Task2.2.ipynb`	three-constant model, Eq. (1.1), 45 points	2.2
2.3	`P2_Task2.3.ipynb`	five-constant model, Eq. (1.5), 77 + 15 points	2.3
ET1	`P2_ET1.ipynb`	fractional sensitivity of q'' to its inputs	2.4
ET2	`P2_ET2.ipynb`	inverse relation for the superheat, Eq. (ET2.1)	2.5
2.4, 2.5	`P2_Task2.4-2.5.ipynb`	heat-pipe model, $(UA)_e$, $(UA)_c$, α	3
2.6	this report	summary, code changes, conclusions	4, 5

1.2 Software and reproducibility

All programs were written in Python 3 (NumPy, Matplotlib) as Jupyter notebooks. They are built on the skeleton programs `CodeP2.1F26`, `CodeP2.2F26` and `CodeP2.3F26`. Every run is reproducible because both random number generators are seeded (see Section 5). Each notebook writes its figures to `report/figures/` and its numerical results to `code/results/*.json`. The tables of this report are generated from these files. The best-fit constants are always the ones found by the genetic algorithm. In addition, a deterministic minimisation (SciPy Nelder–Mead, multi-start) of *exactly the same* error function is used *only as an independent check*: it tells us how close the GA constants are to the true minimum of the error function.

1.3 The genetic algorithm of the skeleton program

The evolution loop of `CodeP2.2F26` was kept unchanged in all the programs. Each organism ("DNA strand") j is a vector $[n_0 = -1, n_1, \ldots, n_5]$ of model constants. The population has $N_S = N_D$ organisms, and at every generation k the loop does the following:

1. **Fitness.** Organism i is compared with data point i only. Its error $f_{err,i}$ (e.g. Eq. (1.3)) gives the absolute fractional error $\mathrm{aFerr}_i = |f_{err,i}|/|\ln q''_{data,i}|$. Because mating and selection constantly reorder the population, the pairing between organisms and data points is random.
2. **Selection.** The cut-off is $c_{lim} = \mathrm{MFRAC} \times \mathrm{median(aFerr)}$, with $\mathrm{MFRAC} = 0.5$. The `nkeep` organisms with $\mathrm{aFerr} < c_{lim}$ survive unchanged.
3. **Mating and mutation.** $N_S - $ `nkeep` offspring are created. For each offspring, two parents are drawn at random among the first `nkeep` + 1 organisms of the previous population. Each

constant is copied from one of the two parents (probability 1/2) and multiplied by $1 + \text{MUT} \cdot 2(0.5 - U)$, with U uniform in $[0, 1]$. The mutation is therefore uniform in $\pm$MUT (MUT $= 0.085$ in the skeleton code, $\pm 9\,\%$ in the statement).

4. **Population average.** The averages $\bar{n}_j(k)$ of the constants over the population are computed. The error of these average constants over *all* data points, $[F_{err}]_{mean}(k) = F_{err}/N_D$ (`aFerrmeanavgn`), measures the quality of the fit. Its minimum over all generations, `aFerrmeanavgnMin`, and the corresponding constants (`n1min`, ...) are the best fit. Time averages of $\bar{n}_j$ for $k > 800$ are also computed.

Two properties of this algorithm explain most of the behaviour reported below. First, the selection threshold is *relative*: it compares errors to their median. Second, the mutations are *multiplicative*: a constant moves by at most a factor $(1 \pm \text{MUT})$ per generation, and it can never change sign.

2 Part 1 – Nucleate boiling of a water/2-propanol mixture in variable gravity

2.1 Task 2.1 – Data preparation and first look at the data

(a) Installation and check of the data array. `CodeP2.1F26` was run in the first cell of the notebook `P2_Task2.1.ipynb`. It prints the array `ydata` (45 rows $[q'',\ T_w - T_{sat},\ g,\ \gamma,\ P]$, those printed in black in the Appendix) and a single element. Instead of comparing the printed array with the Appendix by eye, I wrote a small check program (cell 3). It reads all the rows of the original skeleton file (active and commented-out lines, 77 rows) and compares them, row by row, with the Appendix listing typed from the project statement. **Two rows of the skeleton code do not match the Appendix** (Table 2). Both rows are in the active $g = 0.098$ m/s^2 group. I corrected them to the Appendix values in all my programs. After the correction, the 45 active rows agree exactly with the Appendix, and the single element `ydata[3][0]` $= 59.2$ is the 4th heat flux of the listing, as expected (indices start at 0).

Table 2: Differences between the data of the skeleton program `CodeP2.1F26` and the Appendix of the statement.

Row (index)	Skeleton code	Appendix	Correction applied
0	`[44.2, 32.5, 0.098, 1.79, 5.5]`	`[44.1, 32.5, 0.098, 1.79, 5.5]`	q'': $44.2 \to 44.1$ W/cm^2
6	`[76.3, 37.8, 0.098, 1.79, 5.5]`	`[76.3, 37.5, 0.098, 1.79, 5.5]`	$T_w - T_{sat}$: $37.8 \to 37.5\,^\circ$C

(b) Log-log plot of heat flux versus superheat. Figure 1 shows the first two groups of the Appendix ($g = 0.098$ and 9.8 m/s^2, $P = 5.5$ kPa, $\gamma = 1.79$). To quantify the trends, I fitted a power law $q'' = C(T_w - T_{sat})^m$ to each group (least squares on the logarithms):

$$g = 0.098 \text{ m/s}^2 :\ q'' = 1.73 \times 10^{-3}\,(T_w - T_{sat})^{2.94}, \quad g = 9.8 \text{ m/s}^2 :\ q'' = 2.95 \times 10^{-3}\,(T_w - T_{sat})^{2.80}.$$

Two observations guide the rest of Part 1:

- The heat flux varies **strongly with superheat**. The exponent is close to 3, the classical value for nucleate boiling of pure fluids, so $n_2 \approx 3$ is a physically motivated initial guess.
- The heat flux varies **very weakly with gravity**. The two curves almost coincide: $q''(9.8)/q''(0.098) = 1.05$, 1.01 and 0.98 at 32, 40 and 50 °C, while g changes by a factor of 100. For a pure fluid, $q'' \sim g^n$ with $n \approx 0.01$–0.25 would predict a decrease by a factor of about 1.5–3 between $1\,g_{en}$ and $0.01\,g_{en}$. This is the signature of the Marangoni effect described in the statement: at low gravity, the surface-tension-driven flow still brings liquid to the heated surface, so the heat flux does not drop. A simple power law in g (Eq. (1.1)) is therefore not expected to describe the gravity effect well.

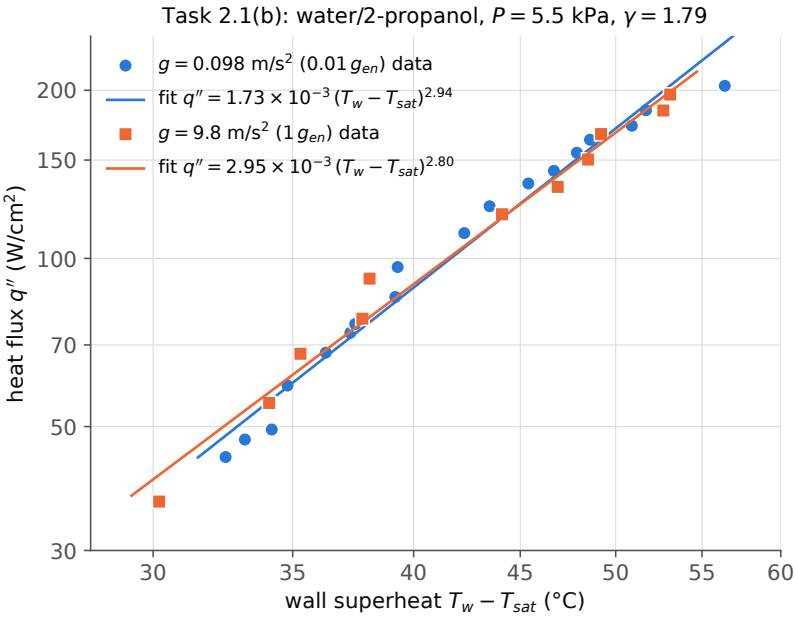


Figure 1: Task 2.1(b): heat flux versus wall superheat for $g = 0.098$ and 9.8 m/s^2 (log-log axes), with a power-law fit for each gravity level.

2.2 Task 2.2 – Three-constant model $q'' = n_1(T_w - T_{sat})^{n_2} g^{n_3}$

Assembly of the program. `CodeP2.1F26` (data, corrected as above) and `CodeP2.2F26` were placed in two consecutive cells of `P2_Task2.2.ipynb`. **The program provided does not run as it is**: the line `import random` at the top of `CodeP2.2F26` gives the name `random` to the *module* and hides the *function* `random()` imported by `CodeP2.1F26`. The initialisation `n1i+0.001*random()` then stops with `TypeError: 'module' object is not callable`. I replaced it with `from random import random, seed`. The other changes (Section 5) do not modify the algorithm. The evolution loop is placed in a function `run_GA(n1i, n2i, n3i, NGEN, MUT, MFRAC, rseed)`, so that it can be re-run for many initial guesses. The mutation amplitude (0.085 in the code) becomes the argument `MUT`, and both random generators are seeded.

First run (skeleton initial guesses). With the initial guesses of the skeleton ($n_1 = 0.00027$, $n_2 = 4.0$, $n_3 = 0.063$), MUT = 0.085 and NGEN = 6000, $[F_{err}]_{mean}$ drops below the target 0.035 after only 12 generations. The minimum, $[F_{err}]_{mean} = 0.03084$, is reached at generation 5202 (first lines of Table 4). The target is met. However, $n_2 = 3.34$ is far from the value $n_2 \approx 3$ of Task 2.1(b), and a second look shows that this solution is not the best one.

Effect of the initial guesses, of the mutation amplitude and of NGEN. Four sets of initial guesses were tested (Table 3). Set 1 is the skeleton guess and Set 2 a physics-based guess ($n_2 = 3$, $0.01 < n_3 < 0.1$). Set 3 is far from the solution ($n_2 = 2$, $n_3 = 0.25$) and Set 4 very far (n_1 about 1500 times too small, $n_2 = 5$). Each set was run with 5 different random seeds, for three maximum mutation amplitudes (MUT = ±8.5, ±20 and ±40 %), with a longer run (NGEN = 20000), and with a two-stage strategy: exploration with MUT = 0.40, then a restart from the best constants with MUT = 0.03. In total 88 runs were made. Figure 2 shows the convergence histories and Figure 3 the trajectories of the population-average constants on the error landscape.

Table 3: Task 2.2: effect of the initial guesses, of the maximum mutation amplitude MUT and of NGEN (5 seeds per line, 2 for NGEN = 20000). "best/median/worst" = minimum $[F_{err}]_{mean}$ reached in a run; "< 0.035" = number of runs that reach the target; $k_{0.035}$ = median generation at which the target is first reached; last column = spread of the best-fit n_2 between seeds. Deterministic reference minimum: $[F_{err}]_{mean} = 0.02869$ at $n_2 = 3.0103$.

Set	Initial guesses (n_1, n_2, n_3)	MUT	NGEN	best $[F_{err}]_{mean}$	median	worst	< 0.035 runs	$k_{0.035}$	n_2 (min–max)
				min. over the run					
Set 1	$(2.70 \times 10^{-4}, 4, 0.063)$	0.085	6000	0.02931	0.02964	0.03084	5/5	12	3.08–3.34
		0.2	6000	0.02872	0.02878	0.02901	5/5	6	2.98–3.07
		0.4	6000	0.02881	0.02903	0.02921	5/5	6	2.96–3.09
		0.085	20000	0.02884	0.02958	0.03032	2/2	56	3.05–3.28
		0.40/0.03	6000+6000	0.02870	0.02883	0.02892	5/5	6	2.97–3.08
Set 2	$(1.00 \times 10^{-3}, 3, 0.05)$	0.085	6000	0.02877	0.02880	0.02898	5/5	2	3.02–3.09
		0.2	6000	0.02871	0.02875	0.02884	5/5	4	3.00–3.05
		0.4	6000	0.02877	0.02883	0.02925	5/5	6	2.94–3.09
		0.085	20000	0.02879	0.02880	0.02881	2/2	2	3.04–3.04
		0.40/0.03	6000+6000	0.02870	0.02880	0.02891	5/5	6	2.94–3.07
Set 3	(0.01, 2, 0.25)	0.085	6000	0.03051	0.09523	0.11062	1/5	662	2.36–2.67
		0.2	6000	0.02890	0.02972	0.03044	5/5	229	2.65–2.91
		0.4	6000	0.02874	0.02909	0.02939	5/5	126	2.80–3.00
		0.085	20000	0.08451	0.08939	0.09427	0/2	never	2.47–2.50
		0.40/0.03	6000+6000	0.02872	0.02897	0.02925	5/5	126	2.84–2.98
Set 4	$(1.00 \times 10^{-6}, 5, 0.01)$	0.085	6000	0.05942	0.06395	0.06575	0/5	never	4.37–4.61
		0.2	6000	0.03304	0.04140	0.05609	1/5	5333	2.93–4.23
		0.4	6000	0.03161	0.03249	0.03378	5/5	1635	2.76–3.37
		0.085	20000	0.06024	0.06215	0.06407	0/2	never	4.43–4.58
		0.40/0.03	6000+6000	0.03099	0.03214	0.03337	5/5	1635	2.72–3.36

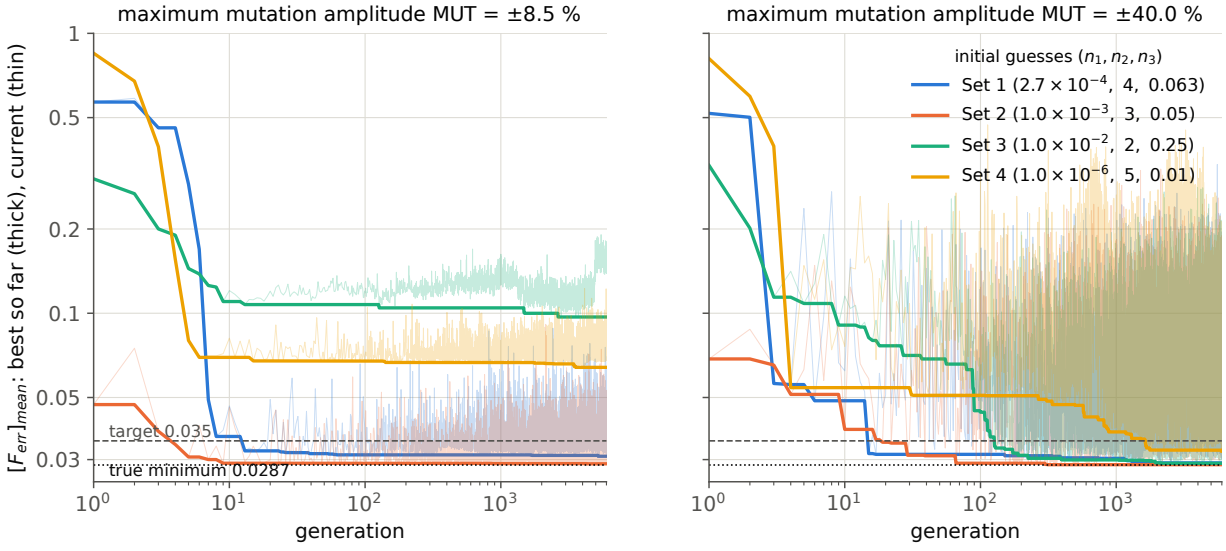


Figure 2: Task 2.2: convergence of $[F_{err}]_{mean}$ for the four sets of initial guesses (seed 1). Thick lines: best value so far; thin lines: value at each generation. Left: skeleton mutation amplitude (±8.5 %); right: ±40 %.

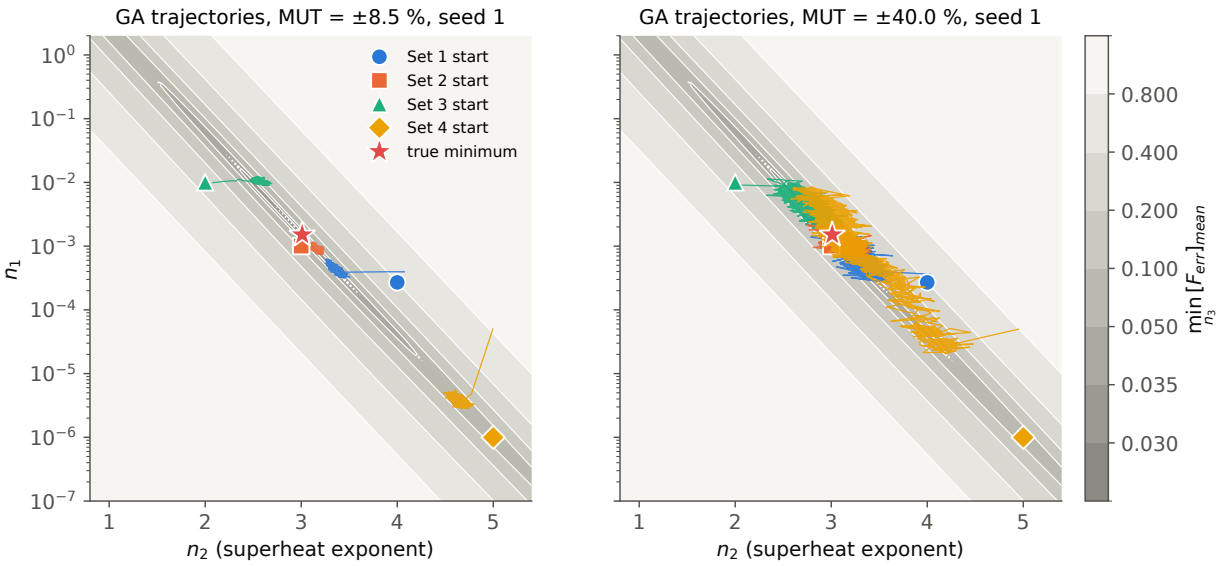


Figure 3: Task 2.2: error landscape (minimum over n_3 of $[F_{err}]_{mean}$ in the (n_2, n_1) plane) and trajectories of the population-average constants of the four sets (seed 1). The error has a long, narrow valley along $\ln n_1 + 3.7\, n_2 \approx$ const.

The main observations are:

- **Guesses close to the solution (Sets 1 and 2) converge for every mutation amplitude and every seed**, in a few generations. The target 0.035 is reached after 2–12 generations, because the GA only has to adjust n_1 to the level of the data. Even so, with the skeleton amplitude (±8.5 %), Set 1 stops at $n_2 = 3.08$–3.34 depending on the seed. The error is below the target, but these constants are not the optimum.
- **The error surface is a narrow valley** (Fig. 3). Since $\ln q'' = \ln n_1 + n_2 \ln(T_w - T_{sat}) + n_3 \ln g$ and $\ln(T_w - T_{sat}) \approx 3.7$ in the middle of the data, $\ln n_1$ and n_2 compensate each other: along the valley floor, $[F_{err}]_{mean}$ stays below 0.035 for $2.2 \lesssim n_2 \lesssim 3.6$. A mutation of ±8.5 % on n_2 (±0.26) changes $\ln q''$ by about ±1, which is much more than the scatter of the data (about 0.15). As a result, almost only the offspring whose mutations of n_1 and n_2 compensate each other survive, and the population moves along the valley very slowly, like a random walk.
- **Far guesses (Sets 3 and 4) do not converge with the skeleton amplitude**, and giving them more generations does not help. With MUT = 0.085, Set 3 reaches the target in only 1 run out of 5 and Set 4 never does. NGEN = 20000 gives no improvement (0/2): the population remains trapped at $n_2 \approx 2.5$ with $n_3 \approx 0.2$ (Set 3), or at $n_2 \approx 4.4$ (Set 4), with $[F_{err}]_{mean} \approx 0.06$–$0.10$. As suggested in the statement, **increasing the mutation amplitude solves the problem**. With ±40 %, all runs of Sets 3 and 4 reach the target, after about 130 and 1600 generations. The cost is a much noisier population (thin lines in Fig. 2), so the minimum is reached only occasionally.
- **The two-stage strategy (large mutations to explore, then small mutations to refine) combines robustness and accuracy.** It keeps the 100 % success rate of the ±40 % runs, even from Set 4. It lowers the median error of every set compared with the single ±40 % run, and it produced the overall best fit. For guesses that are already close to the solution, a moderate amplitude (±20 %) in a single run works just as well.

Best fit. The best of the 88 runs (Set 2, two-stage, seed 2) gives the constants of Table 4:

$$\boxed{q'' = 1.72 \times 10^{-3} \, (T_w - T_{sat})^{2.9747} \, g^{0.0438}} \qquad [F_{err}]_{mean} = 0.02870 \; (< 0.035).$$

This error is within 0.05 % of the true minimum of the error function (0.02869) found by the deterministic check. The constants themselves differ slightly from the reference (n_2: −1.2 %, n_3: −6 %, n_1: +14 %). This difference is the flat valley again: these sets of constants describe the data equally well. The robust result is $n_2 \approx 3.0 \pm 0.05$ and a weak exponent $n_3 \approx 0.045$.

Table 4: Task 2.2: constants of Eq. (1.1) printed by the program (MINIMUM = population-average constants at the minimum of $[F_{err}]_{mean}$, ENDING = last generation, TIME AVG = average over generations 801–NGEN). †the TIME AVG error printed by the skeleton is the average of $[F_{err}]_{mean}$ over all generations.

		n_1	n_2	n_3	$[F_{err}]_{mean}$
Skeleton guesses (Set 1, MUT = 0.085):	MINIMUM	4.56×10^{-4}	3.3359	0.0591	0.03084
	ENDING	5.08×10^{-4}	3.3249	0.0737	0.03398
	TIME AVG	4.27×10^{-4}	3.3633	0.0724	0.03685†
Best GA fit (Set 2, two-stage, seed 2):	MINIMUM	$\mathbf{1.72 \times 10^{-3}}$	**2.9747**	**0.0438**	**0.02870**
Deterministic reference minimum (check)		1.51×10^{-3}	3.0103	0.0464	0.02869

Predicted versus measured heat flux and rms deviation. Figure 4(a) shows each pair (q''_{pred}, q''_{exp}) as a discrete point on log-log axes, with the line of slope 1 and the ±10 % lines. Figure 4(b) shows the fractional deviation for each gravity level. The deviation statistics are:

- rms deviation of q''_{exp} from q''_{pred}: 23.1 W/cm^2 (mean heat flux 113 W/cm^2);
- rms fractional deviation $\sqrt{\overline{[(q''_{exp} - q''_{pred})/q''_{pred}]^2}} = 16.6\,\%$, mean absolute fractional deviation 13.2 %, maximum 33 %;
- only 47 % of the points are within ±10 % of the prediction.

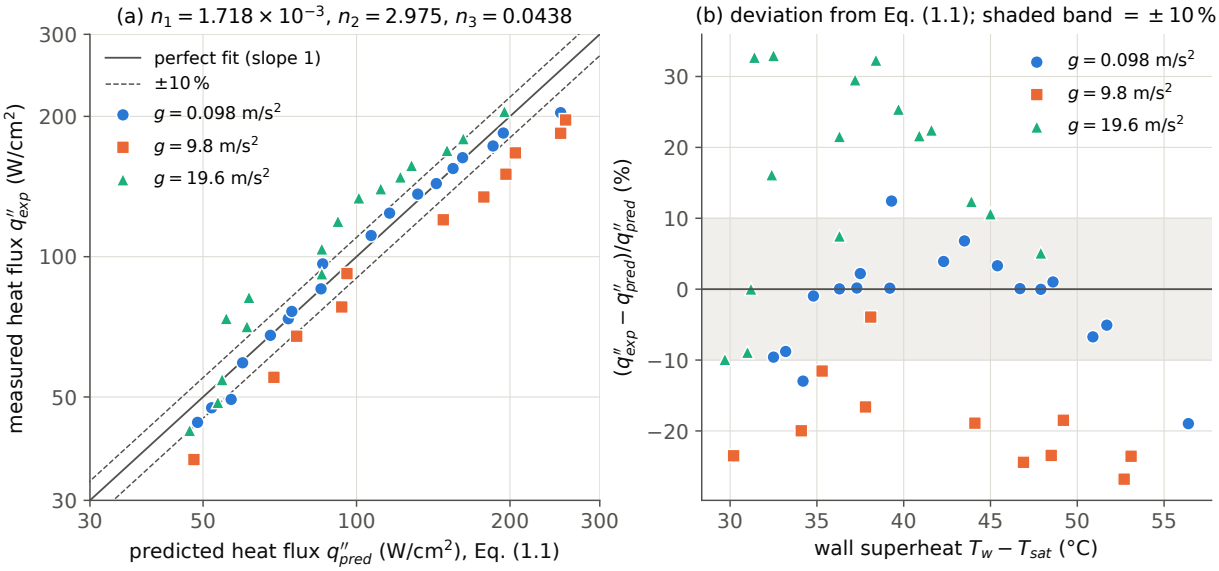


Figure 4: Task 2.2: (a) measured versus predicted heat flux for the best-fit constants (log-log); (b) fractional deviation versus superheat. Each gravity level has its own symbol.

Assessment of the fit with respect to the ±10 % uncertainty. The fit is not tighter than the uncertainty of the data: the rms deviation (16.6 %) and the mean absolute deviation (13.2 %) are larger than the estimated ±10 % measurement uncertainty. The superheat is also uncertain: ±0.5 °C at 40 °C gives ±4 % on q'' through $n_2 \approx 3$, so the combined noise is about

10.7 %. That is still clearly below the rms deviation. **The deviation of the fit is therefore above the noise level of the data.** The groups in Fig. 4(b) show where it comes from. The deviations are *systematic*, not random:

Group	mean deviation	rms deviation
$g = 0.098$ m/s^2 (18 points)	-1.8 %	7.5 %
$g = 9.8$ m/s^2 (11 points)	-19.2 %	20.2 %
$g = 19.6$ m/s^2 (16 points)	$+15.7$ %	20.8 %

The $0.01\,g_{en}$ group is fitted to within the noise (7.5 % rms). The $1\,g_{en}$ group is overpredicted by about 19 % and the $2\,g_{en}$ group underpredicted by about 16 %. With a single exponent, g^{n_3} cannot represent at the same time an almost zero change between 0.01 and $1\,g_{en}$ (Task 2.1(b)) and a 15–30 % increase between 1 and $2\,g_{en}$, so n_3 ends up as a compromise. This is a **model-form error**: the Marangoni contribution, which keeps the heat flux finite when $g \to 0$, is missing from Eq. (1.1).

Consequences for the use of Eq. (1.1):

- *Accuracy of the predictions*: a heat flux predicted by Eq. (1.1) is uncertain by about ± 17 % (one standard deviation), and up to ± 30 % for some conditions. This is larger than the measurement uncertainty and depends on the gravity level (biased predictions at $1\,g_{en}$ and $2\,g_{en}$).
- *Accuracy of the predicted trends*: the superheat trend ($n_2 \approx 3$) is robust. It agrees with Task 2.1(b) and changes little between the runs. The gravity trend is not reliable: the model predicts that q'' increases by $100^{n_3} - 1 \approx 22$ % between $0.01\,g_{en}$ and $1\,g_{en}$, while the data show almost no change. Extrapolating to $g \to 0$ would even predict $q'' \to 0$, which is the opposite of the observed Marangoni behaviour.
- To bring the deviation down to the noise level, the model must contain the missing physics. The five-constant model of Task 2.3 adds it, and removes most (not all) of this bias.

2.3 Task 2.3 – Five-constant model $q'' = n_1(T_w - T_{sat})^{n_2}(g + n_4 g_{en}\gamma)^{n_3} P^{n_5}$

Program. `P2_Task2.3.ipynb` is a copy of the Task 2.2 program with modifications (i)–(vii) of the statement (red data uncommented, $N_D = N_S = 77$, error function Eq. (1.7) in the two loops, mating of n_4 and n_5, five-constant print-out and plot, initial guesses (vii)). They are listed in Section 5. I also added the 15 validation points of the Appendix (`vdata`). They are never used by the GA; they only test the fitted relation on data it has not seen.

Run with the initial guesses (vii). The initial guesses of the statement already give $[F_{err}]_{mean} = 0.03212$. The GA (MUT = ± 9 %, NGEN = 6000) goes below the target 0.03 at generation 193 and reaches a minimum of 0.02808 at generation 2309 (Table 6). The target is met. However, the deterministic check shows that the error function can go down to 0.02085, about 25 % lower, so a broader range of initial guesses was explored.

Range of initial guesses. Five sets of initial guesses were run with two mutation amplitudes and five seeds each (Table 5): (A) the guesses (vii) of the statement; (B) no pressure effect ($n_5 = 0.01$) and the pure-fluid gravity exponent $n_3 = 0.25$; (C) stronger gravity/Marangoni and pressure effects ($n_3 = n_5 = 0.5$, $n_4 = 2$); (D) a far guess; (E) all constants equal to 1. The best single run of each seed was then refined in two more stages, restarted from its best constants with MUT = 0.03 and then 0.01 (last column of Table 5).

Table 5: Task 2.3: effect of the initial guesses (NGEN = 6000, 5 seeds per line, 3 for MFRAC = 1). "frac. nkeep= 0" = fraction of generations without any survivor. "refined" = best and worst $[F_{err}]_{mean}$ after the two refinement stages (MUT = 0.03 then 0.01) for 3 seeds. Deterministic reference minimum: 0.02085.

Set	Initial guesses $(n_1, \ldots, n_5)$	MUT	MFRAC	best	median	worst	< 0.03	$k_{0.03}$	frac.	refined
				single run, $[F_{err}]_{mean}$			runs		nkeep= 0	best–worst
A	$(4.76 \times 10^{-4}, 3.028, 0.2249, 1.054, 0.217)$	0.09	0.5	0.02548	0.02739	0.02899	5/5	336	0.00	0.02211–0.02341
		0.20	0.5	0.02463	0.02592	0.02972	5/5	358	0.00	
B	$(1.00 \times 10^{-3}, 3, 0.25, 1, 0.01)$	0.09	0.5	0.03037	0.03070	0.03156	0/5	never	0.00	0.02508–0.02773
		0.20	0.5	0.02781	0.02959	0.03239	3/5	2004	0.00	
C	$(1.00 \times 10^{-4}, 3, 0.5, 2, 0.5)$	0.09	0.5	0.02167	0.02203	0.02216	5/5	3	0.00	0.02097–0.02155
		0.20	0.5	0.02170	0.02191	0.02224	5/5	2	0.00	
D	(0.01, 2.5, 0.1, 0.5, 1)	0.09	0.5	0.03607	0.03856	0.03969	0/5	never	0.00	0.03348–0.03466
		0.20	0.5	0.03260	0.03538	0.03660	0/5	never	0.00	
E	(1, 1, 1, 1, 1)	0.09	0.5	0.07376	0.94966	0.96786	0/5	never	0.90	0.03302–0.04089
		0.20	0.5	0.05004	0.05871	0.06364	0/5	never	0.00	
		0.09	1.0	0.04968	0.05125	0.05337	0/3	never	0.00	

The behaviour is richer than for three constants:

- **Set C converges almost immediately** (target reached in 2–3 generations, $[F_{err}]_{mean} \approx 0.022$), and its refinement gives the best fit, within 0.6 % of the true minimum.
- **Set A** (statement) always meets the target, but it stops at 0.025–0.030, and at 0.022–0.023 after refinement. It starts with $n_3 \approx n_5 \approx 0.22$, on the wrong side of a long valley in the (n_3, n_5) directions, and the population moves along this valley only slowly.
- **Set B shows a limitation of multiplicative mutations.** A constant guessed close to zero can hardly move: a ±9 % mutation of $n_5 = 0.01$ changes it by at most ±0.0009. The pressure exponent therefore stays close to 0, the other constants have to compensate, and only 3/10 runs reach the target. *A constant whose sign or order of magnitude is unknown should not be initialised close to zero.*
- **Sets D and E fail**, even after refinement ($[F_{err}]_{mean} \geq 0.033$). With Set E and the skeleton settings, the population is even *frozen*: in 90 % of the generations no organism has an error below 0.5× median, so `nkeep` = 0. All offspring are then mutations of the same organism `n[0]`, which is never replaced (the median error stays at 0.95). Larger mutations (MUT = 0.2) or a less selective threshold (MFRAC = 1) unfreeze the population, but 6000 generations are not enough to reach the valley. The same mechanism is analysed in detail for Part 2 (Section 3, Fig. 11).

Best fit. The best-fit constants are given in Table 6 (set C, seed 1, refined):

$$\boxed{q'' = 1.20 \times 10^{-4} \, (T_w - T_{sat})^{\,3.0216} \, \left(g + 1.3485 \, g_{en}\gamma\right)^{0.4619} \, P^{\,0.5435}}$$

with $[F_{err}]_{mean} = 0.02097$ (< 0.03) (q'' in W/cm^2, T in °C, g in m/s^2, P in kPa). Figure 5 shows the evolution of the constants and of the error. The deterministic check gives $[F_{err}]_{mean} = 0.02085$, with slightly different constants ($n_2 = 2.9771$, $n_3 = 0.4808$, $n_4 = 1.2586$, $n_5 = 0.4708$). The profiles of Fig. 6 explain this difference. When one constant is fixed and the four others are re-optimised, the minimum error increases by only 3 % (to about 0.0215) over the ranges $n_2 \approx 2.8$–3.2 (±7 %), $n_3 \approx 0.41$–0.54 (±14 %), $n_4 \approx 1.1$–1.65 (±20 %) and $n_5 \approx 0.28$–0.65 (±40 %). The superheat exponent is therefore well determined by the data, n_3 reasonably well, and n_4 and n_5 only weakly. Constants with this kind of uncertainty can differ between equally good fits. This is expected for n_5: pressures other than 5.5 kPa appear only in two groups, and the 7.0 kPa group is also the only one with $\gamma = 0$.

Table 6: Task 2.3: constants of Eq. (1.5). †average of $[F_{err}]_{mean}$ over all generations.

	n_1	n_2	n_3	n_4	n_5	$[F_{err}]_{mean}$ (77 training)	$[F_{err}]_{mean}$ (15 validation)
Initial guesses (vii)	4.76×10^{-4}	3.028	0.2249	1.054	0.217	0.03212	–
Run with guesses (vii): MINIMUM	5.73×10^{-4}	2.8994	0.2960	1.2831	0.2304	0.02808	–
ENDING	5.98×10^{-4}	2.9737	0.2383	1.2640	0.2206	0.03781	–
TIME AVG	5.85×10^{-4}	2.9436	0.2294	1.2952	0.2337	0.03378†	–
Best GA fit (set C, refined)	$\mathbf{1.20 \times 10^{-4}}$	**3.0216**	**0.4619**	**1.3485**	**0.5435**	**0.02097**	**0.02566**
Deterministic reference minimum	1.56×10^{-4}	2.9771	0.4808	1.2586	0.4708	0.02085	–

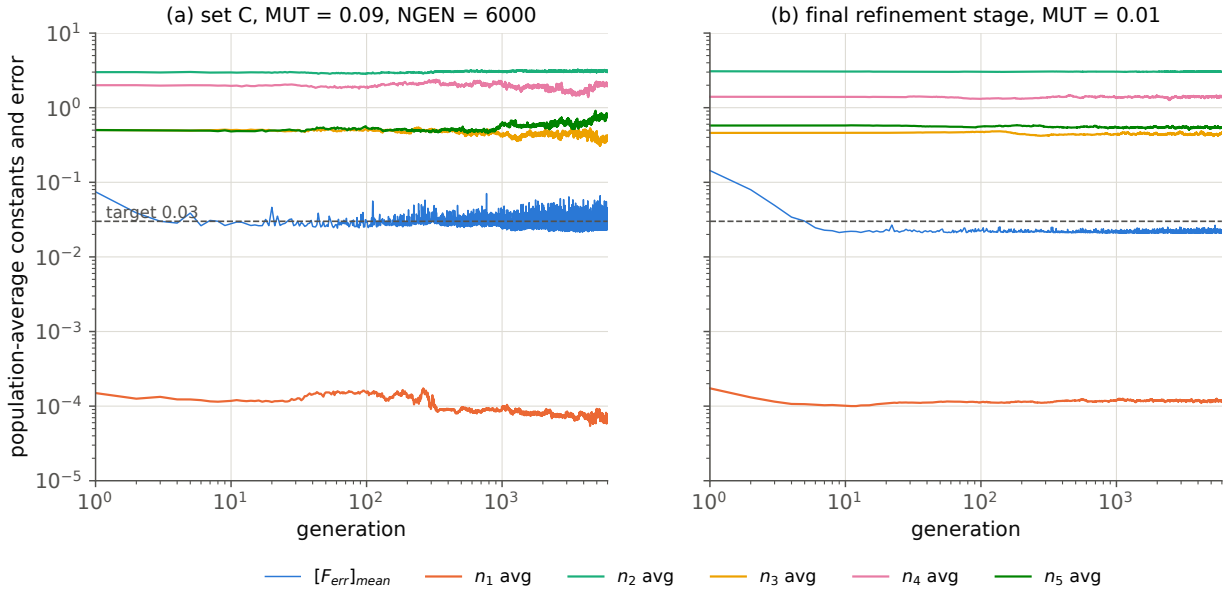


Figure 5: Task 2.3: population-average constants and error of the best-fit sequence. (a) Main GA run from set C; (b) last refinement stage (MUT = 0.01).

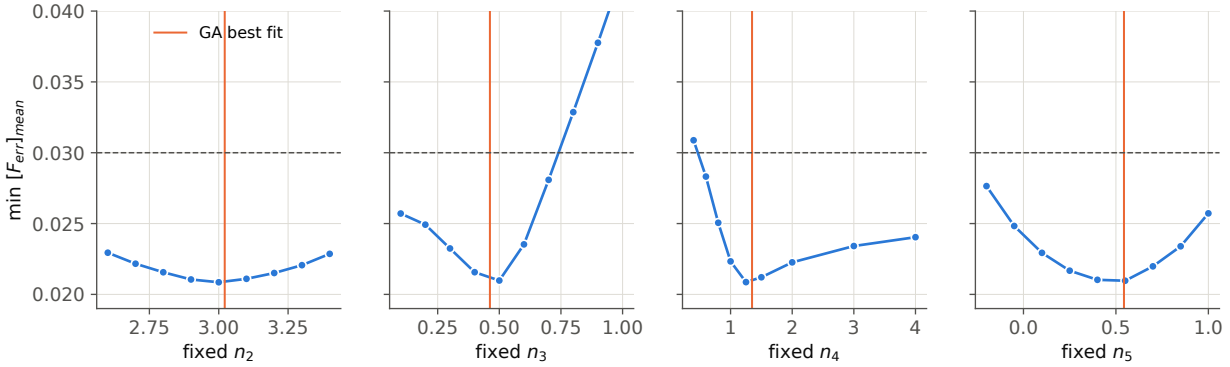


Figure 6: Task 2.3: minimum achievable $[F_{err}]_{mean}$ when one constant is fixed and the four others are re-optimised. A flat profile means that the data determine this constant only weakly.

Quality of the fit and validation. Figure 7 compares the predictions with the 77 training points and the 15 validation points.

- Training data: rms fractional deviation 12.3 %, mean absolute deviation 9.4 %, 58 % of the points within ±10 %. For the 45 points of Task 2.2, the three-constant model gave 16.6 %, 13.2 % and 47 %.

- Validation data: $[F_{err}]_{mean} = 0.02566$, rms fractional deviation 14.1 %, mean absolute deviation 10.9 %.

Overall, the deviation is now close to the estimated noise of the data (about 10–11 %). The validation error is only slightly larger than the training error, so the five-constant model does not over-fit (77 points for 5 constants). The largest validation deviation is the point $(q'', T_w - T_{sat}) = (50.9,\ 37.2)$ at $0.01\,g_{en}$: its superheat is 3 °C higher than that of the training point with almost the same heat flux (49.4 W/cm^2 at 34.2 °C), which points to measurement scatter.

The remaining deviation is not entirely random, however. Table 7 gives the mean deviation of each test condition. Four conditions are fitted within the noise (mean deviation between −4 % and +6 %). Two are still biased: the $1\,g_{en}$ group is overpredicted by about 16 % (mean deviation −16 %) and the $2\,g_{en}$ group at 5.5 kPa is underpredicted by about 11 % (mean deviation +11 %). The equivalent-gravity term reduces the gravity bias of Eq. (1.1) (−19 % and +16 % for the same two groups) but does not remove it. With a single exponent n_3, the form $(g + n_4 g_{en}\gamma)^{n_3}$ cannot give at the same time almost no change between 0.01 and $1\,g_{en}$ and a 40–50 % increase between 1 and $2\,g_{en}$ at 5.5 kPa (Fig. 7). Part of the gravity effect is therefore still missing from Eq. (1.5), and predictions at 1 and $2\,g_{en}$ carry this systematic error on top of the scatter.

Table 7: Task 2.3: mean and rms deviation $(q''_{exp} - q''_{pred})/q''_{pred}$ for each test condition (training data). Mean deviations larger than 10 % are in bold. Last column: the three-constant fit of Task 2.2 for the groups it was fitted on.

Test condition	points	Eq. (1.5), five constants		Eq. (1.1), three constants
		mean deviation	rms deviation	mean deviation
$g = 0.098$ m/s^2, $\gamma = 1.79$, $P = 5.5$ kPa	18	−2.1 %	7.6 %	−1.8 %
$g = 9.8$ m/s^2, $\gamma = 1.79$, $P = 5.5$ kPa	11	**−15.8 %**	17.3 %	−19.2 %
$g = 19.6$ m/s^2, $\gamma = 1.79$, $P = 5.5$ kPa	16	**+11.0 %**	17.1 %	+15.7 %
$g = 19.6$ m/s^2, $\gamma = 1.79$, $P = 9.5$ kPa	17	−4.1 %	11.6 %	–
$g = 9.8$ m/s^2, $\gamma = 0$, $P = 7$ kPa	7	+0.9 %	2.6 %	–
$g = 0.098$ m/s^2, $\gamma = 1.71$, $P = 5.5$ kPa	8	+6.4 %	7.0 %	–

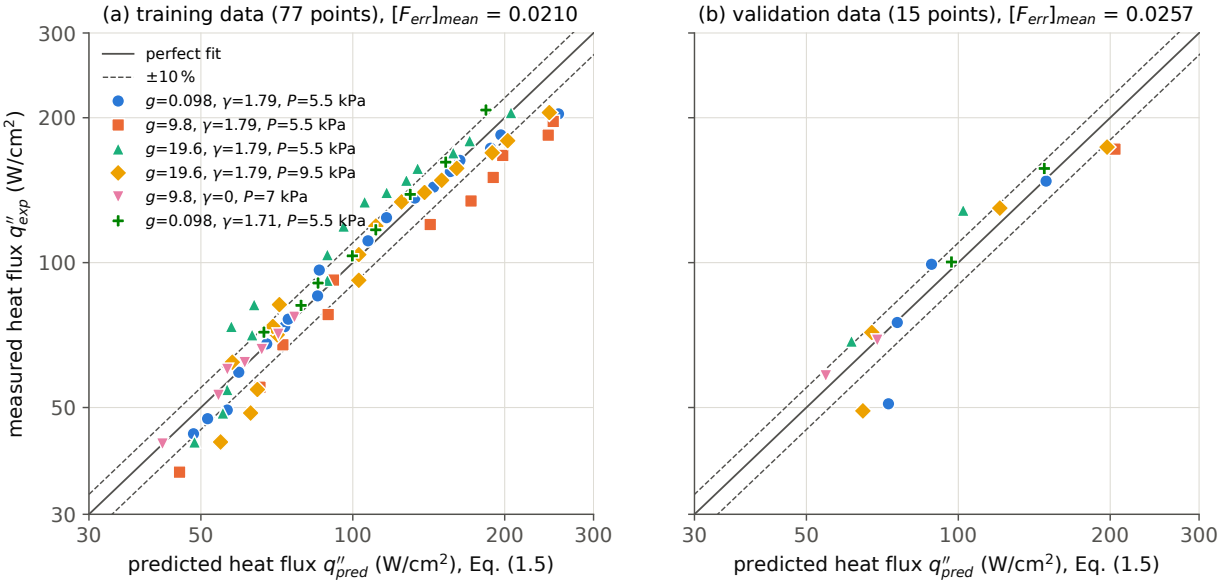


Figure 7: Task 2.3: measured versus predicted heat flux, Eq. (1.5) with the best-fit constants. (a) 77 training points; (b) 15 validation points. One symbol per test condition.

Surface plot and physical interpretation. Figure 8 shows $q''/(T_w - T_{sat})^{n_2} = n_1(g + n_4 g_{en}\gamma)^{n_3} P^{n_5}$ for $1 < g < 20$ m/s^2 and $0 < \gamma < 2$ at $P = 10$ kPa. Three conclusions follow:

- **The Marangoni effect acts like an additional gravity.** The term $n_4 g_{en}\gamma$ is an "equivalent gravity" produced by the surface tension gradients. For $\gamma = 1.79$ it equals 23.7 m/s^2 ($\approx 2.4\, g_{en}$), which is larger than earth gravity. This is why q'' depends so little on gravity at low g: between $0.01\, g_{en}$ and $1\, g_{en}$ the effective gravity only goes from about 24 to 33 m/s^2. The model still predicts +17 % over this range, where the data show almost no change (Task 2.1(b)); this is the residual bias of Table 7.
- **The gravity dependence depends on γ.** Without Marangoni effect ($\gamma = 0$), $q''/(T_w - T_{sat})^{n_2}$ increases by a factor of 4 between $g = 1$ and 20 m/s^2. For $\gamma = 2$ it increases by only 27 %. At $g = 1$ m/s^2, going from $\gamma = 0$ to $\gamma = 2$ multiplies the heat flux by 4.6. This is why a mixture with a positive γ is attractive for low-gravity applications.
- The superheat exponent $n_2 \approx 3.0$ agrees with Tasks 2.1(b) and 2.2. The positive pressure exponent ($n_5 \approx 0.5$) agrees with the known increase of nucleate boiling heat transfer with pressure (through the fluid properties).

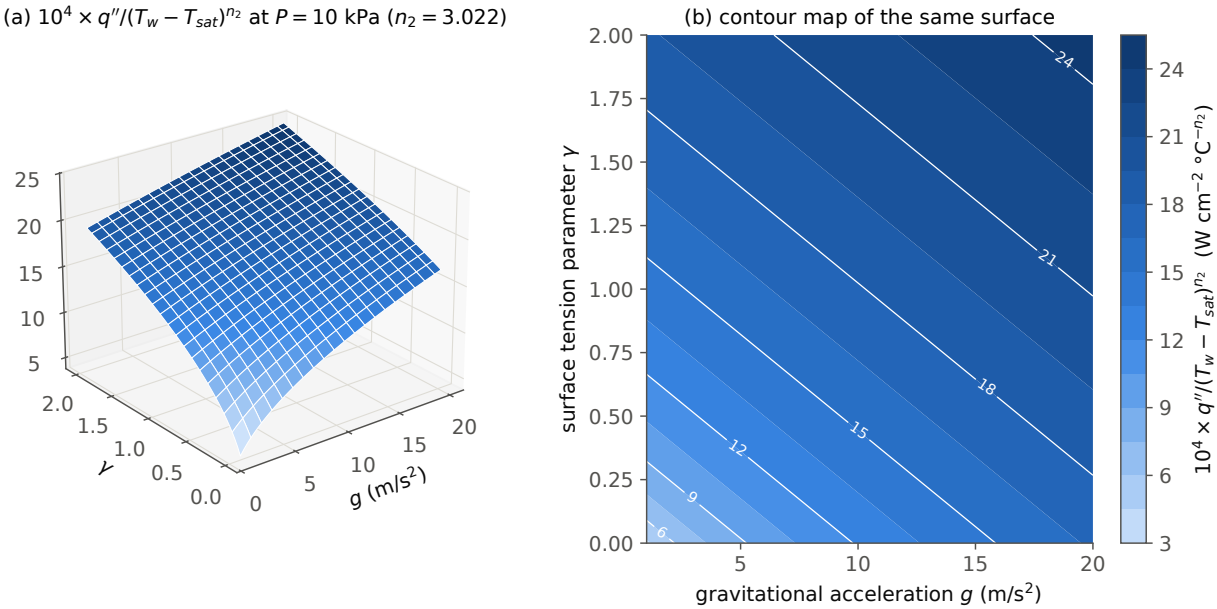


Figure 8: Task 2.3: $q''/(T_w - T_{sat})^{n_2}$ versus g and γ at $P = 10$ kPa for the best-fit constants: (a) surface plot; (b) contour map of the same surface.

2.4 Extra Task ET1 – Fractional sensitivity of the heat flux to the inputs

Method. Program `P2_ET1.ipynb` uses the best-fit constants of Task 2.3 and follows steps 1–3 of the statement for three typical sets of inputs: S1 (earth gravity, $(T_w - T_{sat})_1 = 40$ °C, $g_1 = 9.8$ m/s^2, $\gamma_1 = 1.79$, $P_1 = 5.5$ kPa), S2 (the same at $0.01\, g_{en}$) and S3 ($2\, g_{en}$, 9.5 kPa, 35 °C).

1) q''_1 is computed with Eq. (1.5) for the typical inputs;
2) one input at a time is changed: the superheat by $\Delta(T_w - T_{sat}) = 2$ °C, as in the statement, and g, γ, P by 5 % (the same relative step as 2 °C at 40 °C), giving q''_2;
3) the fractional change ratio is $S_x = \dfrac{\Delta q''/q''_1}{\Delta x/x_1} = \dfrac{(q''_2 - q''_1)\, x_1}{\Delta x\, q''_1}$.

For comparison, the limit $\Delta x \to 0$ gives analytical values: $S_T = n_2$, $S_g = n_3\, g/(g + n_4 g_{en}\gamma)$, $S_\gamma = n_3\, n_4 g_{en}\gamma/(g + n_4 g_{en}\gamma)$ and $S_P = n_5$.

Table 8: ET1: fractional change ratios (finite steps). The analytical limit $\Delta x \to 0$ is given in parentheses.

State	$(T_w - T_{sat})_1$	g_1	γ_1	P_1	q''_1	fractional change ratio $(\Delta q''/q''_1)/(\Delta x/x_1)$			
	°C	m/s^2		kPa	W/cm^2	$x = T_w - T_{sat}$	$x = g$	$x = \gamma$	$x = P$
S1 (earth gravity)	40	9.8	1.79	5.5	106.0	3.177 (3.022)	0.135 (0.135)	0.324 (0.327)	0.537 (0.544)
S2 (microgravity)	40	0.098	1.79	5.5	90.5	3.177 (3.022)	0.002 (0.002)	0.454 (0.460)	0.537 (0.544)
S3 (2 g_{en}, 9.5 kPa)	35	19.6	1.79	9.5	107.3	3.199 (3.022)	0.208 (0.209)	0.251 (0.253)	0.537 (0.544)

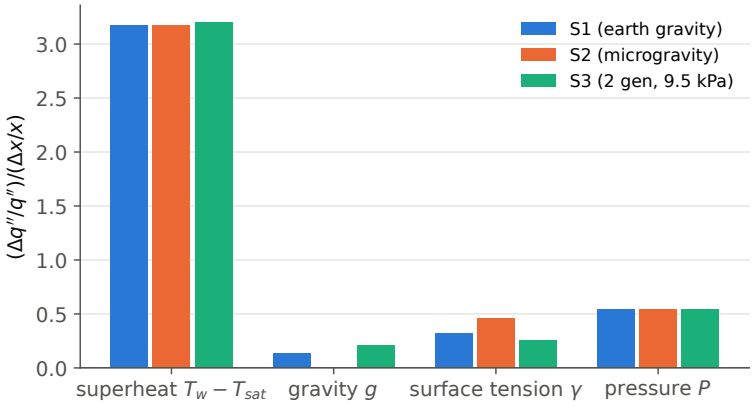


Figure 9: ET1: fractional sensitivity of the predicted heat flux to each input for the three typical states.

Results (Table 8, Fig. 9).

- **The heat flux is by far most sensitive to the superheat**: $S_T = 3.18$. A 1 % error on $T_w - T_{sat}$ gives a 3 % error on q''. The finite-step value is a little higher than $n_2 = 3.02$ because $(1.05)^{3.02} - 1 = 0.159$ is more than 3.02×0.05 (non-linearity of the power law).
- The sensitivity to pressure is the same in all states ($S_P \approx n_5 \approx 0.54$).
- The sensitivities to g and γ depend on the state, because the two effects share the effective gravity $g + n_4 g_{en} \gamma$. At earth gravity, $S_g = 0.135$ and $S_\gamma = 0.324$. In microgravity, $S_g = 0.0019$: the gravity level no longer matters, and S_γ increases to 0.454 because Marangoni flow provides almost all the liquid supply.

Limiting fractional inaccuracy of the inputs for $|\Delta q''/q''| < 0.15$. To first order, $\Delta q''/q'' = S_T\,\varepsilon_T + S_g\,\varepsilon_g + S_\gamma\,\varepsilon_\gamma + S_P\,\varepsilon_P$, where $\varepsilon_x = \Delta x/x$. Three estimates are given in Table 9: (i) only one input is inaccurate, $\varepsilon_x < 0.15/|S_x|$; (ii) all inputs have the same inaccuracy ε and the errors add up in the worst direction, $\varepsilon < 0.15/\sum |S_x|$; (iii) the same ε, but independent random errors combined as a root-sum-square, $\varepsilon < 0.15/\sqrt{\sum S_x^2}$.

Table 9: ET1: limiting fractional inaccuracy of the inputs for a fractional inaccuracy of the predicted heat flux below 0.15. Last column: corresponding superheat uncertainty for case (ii).

State	(i) one input at a time: $0.15/\lvert S_x\rvert$				(ii) worst case	(iii) RSS	superheat
	$T_w - T_{sat}$	g	γ	P	$0.15/\sum \lvert S_x\rvert$	$0.15/\sqrt{\sum S_x^2}$	limit (ii)
S1 (earth gravity)	0.047	1.113	0.464	0.279	0.0359	0.0463	±1.44 °C
S2 (microgravity)	0.047	> 10	0.330	0.279	0.0360	0.0461	±1.44 °C
S3 (2 g_{en}, 9.5 kPa)	0.047	0.721	0.598	0.279	0.0358	0.0460	±1.25 °C

Conclusion. The superheat must be known to about 4.7 % if it is the only inaccurate input (±1.9 °C at 40 °C). If all inputs have the same fractional inaccuracy and the errors add up, they must all be known to about 3.6 % (±1.4 °C on the superheat). With random, independent errors the limit is about 4.6 %. These estimates were checked with the full non-linear model: changing all inputs by 3.6 % in the unfavourable direction gives $\Delta q''/q'' = 15.3\,\%$. Since the superheat dominates, a practical error budget is, for example, superheat ±3 % (±1.2 °C), pressure ±3 %, γ ±5 % and g ±10 %. Even in the worst case, this gives $3.18(0.03) + 0.54(0.03) + 0.32(0.05) + 0.13(0.10) = 0.14 < 0.15$. **The accuracy of the predicted heat flux is therefore limited mainly by the accuracy of the wall superheat measurement.** An uncertainty of ±2 °C, which is not unusual for a wall temperature measurement in a flight experiment, would by itself use up the whole 15 % budget.

2.5 Extra Task ET2 – Inverse relation for the superheat

Method. Program `P2_ET2.ipynb` reorganises the 77 data points of Task 2.3 as $[T_w - T_{sat},\ q'',\ g,\ \gamma,\ P]$, so that q'', g, γ, P are the inputs and the superheat is the output. It then fits Eq. (ET2.1), $(T_w - T_{sat}) = \hat{n}_1 (q'')^{\hat{n}_2} (g + \hat{n}_4 g_{en}\gamma)^{\hat{n}_3} P^{\hat{n}_5}$, with a copy of the Task 2.3 GA (error normalised by $|\ln(T_w - T_{sat})|$). Three sets of initial guesses were used: (I) the values implied by Task 2.3, (II) (10, 0.5, −0.1, 1, −0.1) and (III) (40, 0.25, −0.25, 2, −0.3), each with 3 seeds and the same refinement as in Task 2.3. The best GA fit is $(T_w - T_{sat}) = 20.27\,(q'')^{0.3074}(g + 1.308\,g_{en}\gamma)^{-0.1455} P^{-0.1485}$ with $[F_{err}]_{mean} = 0.00843$. The superheat is predicted with an rms deviation of 1.8 °C (mean absolute deviation 3 %, about one third of the deviation on q'', consistent with $n_2 \approx 3$).

Table 10: ET2: best-fit $\hat{n}_i$ from the GA fit of (ET2.1), compared with the values computed from the best-fit n_i of Task 2.3 with the relations of the statement. Right: the same comparison between the exact minima of the two error functions (deterministic check). Difference = (fit − implied)/|implied|; for the negative exponents $\hat{n}_3$ and $\hat{n}_5$, a positive difference means a smaller magnitude.

	genetic algorithm			exact minima (check)		
	GA fit of (ET2.1)	from Task 2.3 GA n_i	difference	min. of (ET2.1)	from min. of (1.5)	difference
$\hat{n}_1 = (n_1)^{-1/n_2}$	20.2740	19.8579	+2.1 %	20.3192	19.0156	+6.9 %
$\hat{n}_2 = 1/n_2$	0.3074	0.3310	−7.1 %	0.3077	0.3359	−8.4 %
$\hat{n}_3 = -n_3/n_2$	−0.1455	−0.1529	+4.8 %	−0.1445	−0.1615	+10.5 %
$\hat{n}_4 = n_4$	1.3081	1.3485	−3.0 %	1.3305	1.2586	+5.7 %
$\hat{n}_5 = -n_5/n_2$	−0.1485	−0.1799	+17.4 %	−0.1514	−0.1581	+4.3 %
$[F_{err}]_{mean}$ of (ET2.1)	0.00843	0.00865		0.00842		

Do the two sets of $\hat{n}_i$ agree? They are *close but not identical* (Table 10). The differences are typically 3–10 %: $\hat{n}_1$ +2.1 %, $\hat{n}_2$ −7.1 %, $\hat{n}_3$ +4.8 %, $\hat{n}_4$ −3.0 %, and up to +17.4 % for the weakly determined pressure exponent $\hat{n}_5$. The only systematic difference is on $\hat{n}_2$: the direct fit of the superheat gives an exponent on q'' about 7–8 % smaller than $1/n_2$. The same differences appear between the exact minima of the two error functions (right part of the table), so they are not caused by the GA. The implied values also give a larger error for (ET2.1) (0.00865 vs. 0.00843), which shows that the two fits really are different optima.

Is the noise in the data responsible? Yes. Fitting y versus x and x versus y gives two slopes whose product is $r^2 < 1$ when the data scatter, so the inverse exponent $\hat{n}_2$ must come out smaller than $1/n_2$. To check this quantitatively, I made synthetic data with Eq. (1.5) and the Task 2.3 constants, at the 77 measured conditions (Table 11):

(a) **Noise-free data**: the GA fit of (ET2.1) ($[F_{err}]_{mean} = 2.7\times10^{-6}$) recovers the implied values to within 0.03 %. The method and the relations of the statement are therefore exact when

there is no noise.

(b) **Scatter added to q''** (log-normal, standard deviation 0.126, equal to the rms residual of the Task 2.3 fit; 30 realisations): the inverse fit gives $\hat{n}_2$ $7 \pm 2\,\%$ below $1/n_2$, exactly the difference found with the measured data. The other exponents scatter by ±5–10 % from one realisation to the next, which is the size of the differences in Table 10.

(c) **The same scatter put on the superheat instead** gives the same result. Which variable carries the noise only changes which of the two fits is unbiased, not the fact that they disagree.

Table 11: ET2: percentage difference between the inverse fit and the values implied by the forward fit, for the measured data and for synthetic data generated with Eq. (1.5) (mean ± standard deviation over 30 noise realisations).

	measured data (exact minima)	(a) noise-free synthetic, GA	(b) scatter on q'' synthetic, 30 realisations	(c) scatter on $T_w - T_{sat}$ synthetic, 30 realisations
$\hat{n}_1$	+6.9 %	−0.01 %	+8.2 ± 5.1 %	+8.6 ± 5.2 %
$\hat{n}_2$	−8.4 %	+0.00 %	−7.1 ± 2.3 %	−6.5 ± 2.6 %
$\hat{n}_3$	+10.5 %	+0.00 %	+1.2 ± 5.0 %	+3.2 ± 4.9 %
$\hat{n}_4$	+5.7 %	−0.03 %	−8.0 ± 8.5 %	−3.8 ± 9.4 %
$\hat{n}_5$	+4.3 %	+0.01 %	+4.0 ± 10.2 %	+0.1 ± 9.8 %

Conclusion. The differences between the directly fitted $\hat{n}_i$ and those computed from the Task 2.3 constants (≈ 2–17 %, typically 5–10 %) come from the scatter in the data: it produces a systematic ≈ 7 % reduction of $\hat{n}_2$ (regression dilution) and random differences of the other constants of the same size as their run-to-run variability. The two relations are equally valid descriptions of the data. Each one should be used in the direction in which it was fitted: the heat flux from the superheat with Eq. (1.5), and the superheat from the heat flux with Eq. (ET2.1).

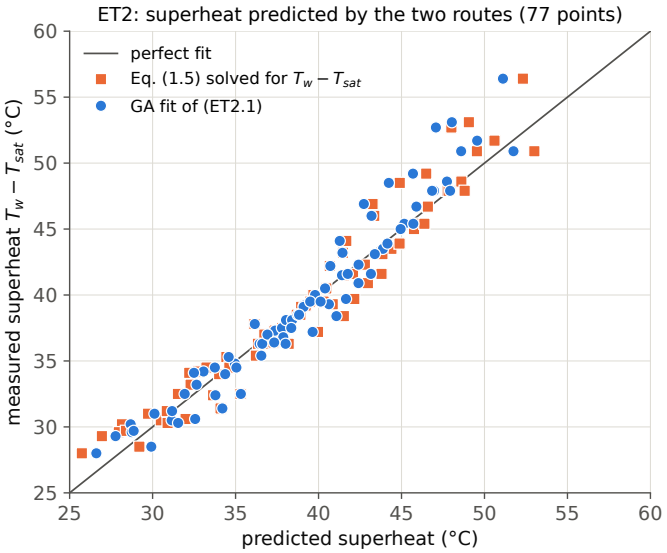


Figure 10: ET2: measured superheat versus the superheat predicted by the GA fit of (ET2.1) and by Eq. (1.5) solved for $T_w - T_{sat}$.

3 Part 2 – Heat-pipe heat exchanger for an electronics cabinet

3.1 Task 2.4 – Genetic algorithm program for the heat-pipe model

Model. Equation (2.3) can be written as two thermal conductances in series:

$$\dot{q} = \frac{T_{a,in} - T_{a,out}}{\dfrac{1}{(UA)_e} + \dfrac{1}{(UA)_c\,(1 + \alpha\, h_r/h_{c,ref})}}, \qquad h_r = \sigma\,(T_{a,in} + T_{a,out})(T_{a,in}^2 + T_{a,out}^2), \tag{1}$$

with absolute temperatures in h_r, $\sigma = 5.67 \times 10^{-8}$ W m^{-2}K^{-4} and $h_{c,ref} = 80$ W m^{-2}K^{-1}. Over the range of the data, h_r varies only between 5.8 and 7.2 W m^{-2}K^{-1}, i.e. $h_r/h_{c,ref} = 0.073$–0.090. The radiation correction is therefore a small, slowly varying term. As in the statement, $n_1 = (UA)_e$, $n_2 = (UA)_c$ and $n_3 = \alpha$.

Program and data. `P2_Task2.4-2.5.ipynb` is a copy of the Task 2.2 program (modifications listed in Section 5). The 122 data points of `CodeP2.3F26` were split at random, with a fixed seed, into a **training set of 98 points** (80.3 %) and a **validation set of 24 points** (19.7 %). The GA uses only the training set ($N_S = N_D = 98$). The error function is the fractional error of the heat transfer rate, Eqs. (2.4)–(2.5). It is coded once in `qdot_model()` (Eq. (2.6)) and used in the two locations of the code (fitness loop and population-average loop) and in the plots. The mean absolute error (MAE) of the validation set is computed at each generation with the population-average constants, for monitoring only.

3.2 Task 2.5 – Results

Initial guesses of order one. Following the statement, four sets of positive initial guesses of order one were used: Set 1 $(1, 1, 1)$, Set 2 $(2, 2, 0.5)$, Set 3 $(5, 5, 1)$ and Set 4 $(10, 3, 2)$. Each set was run with the skeleton selection (MFRAC = 0.5) and two mutation amplitudes (±9 %, ±30 %), and with MFRAC = 1.0, with 3 seeds each and NGEN = 6000 (Table 12).

Table 12: Task 2.5: effect of the initial guesses, of the mutation amplitude and of the selection fraction MFRAC (NGEN = 6000, 3 seeds per line). MAE = mean absolute fractional error, Eq. (2.5)/N_D. "valid. MAE" is the MAE of the validation set for the best run. "frac. nkeep= 0" = fraction of generations without any survivor.

Set	Initial guesses	MUT	MFRAC	training MAE (3 seeds)			valid.	< 0.045	$k_{0.045}$	frac.	best run	
	(n_1, n_2, n_3)			best	median	worst	MAE	runs		nkeep= 0	n_1, n_2	n_3
Set 1	(1, 1, 1)	0.09	0.5	0.8108	0.8113	0.8138	0.8132	0/3	never	1.00	1.02, 1.10	1.00
		0.30	0.5	0.8051	0.8056	0.8079	0.8076	0/3	never	1.00	1.05, 1.13	1.02
		0.09	1.0	0.0166	0.0166	0.0166	0.0208	3/3	84	0.00	5.42, 5.71	1.21
Set 2	(2, 2, 0.5)	0.09	0.5	0.6362	0.6363	0.6390	0.6408	0/3	never	1.00	2.03, 2.11	0.50
		0.30	0.5	0.6245	0.6262	0.6277	0.6293	0/3	never	1.00	2.10, 2.17	0.51
		0.09	1.0	0.0166	0.0166	0.0166	0.0207	3/3	55	0.00	5.75, 5.51	0.83
Set 3	(5, 5, 1)	0.09	0.5	0.0166	0.0166	0.0166	0.0208	3/3	4	0.00	5.56, 5.57	1.15
		0.30	0.5	0.0166	0.0166	0.0167	0.0205	3/3	2	0.00	5.50, 5.70	1.01
		0.09	1.0	0.0166	0.0166	0.0166	0.0208	3/3	6	0.00	5.85, 5.30	1.10
Set 4	(10, 3, 2)	0.09	0.5	0.0166	0.0167	0.0167	0.0208	3/3	3	0.00	6.10, 4.94	1.49
		0.30	0.5	0.0166	0.0166	0.0166	0.0207	3/3	2	0.00	5.28, 5.79	1.43
		0.09	1.0	0.0167	0.0167	0.0167	0.0207	3/3	6	0.00	9.98, 3.66	1.44

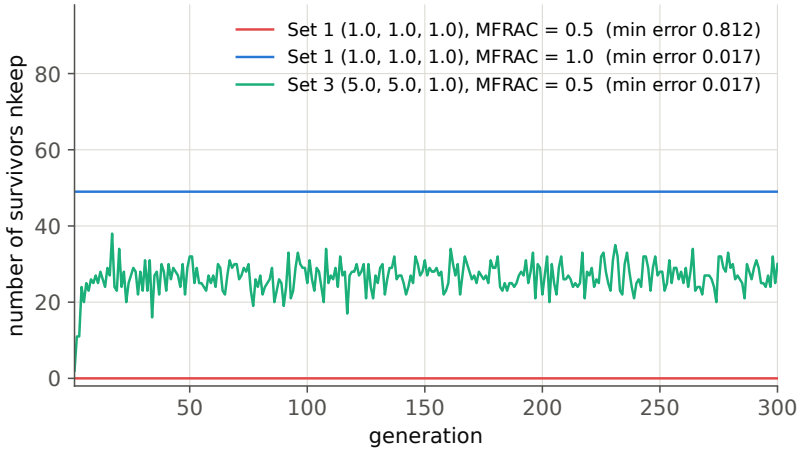


Figure 11: Task 2.5: number of survivors per generation. With $(1, 1, 1)$ and the skeleton threshold (MFRAC $= 0.5$) no organism ever survives and the population is frozen. With MFRAC $= 1$, half of the population survives at every generation.

- **Sets 3 and 4 converge immediately.** Their initial effective conductance is already close to the data ($\dot{q}/(T_{a,in} - T_{a,out}) \approx 2.9$ W/°C). The target 0.045 is reached in 2–6 generations, and all runs end at a training MAE of 0.0166–0.0167.
- **Sets 1 and 2 do not converge at all with the skeleton algorithm, whatever the mutation amplitude.** The error stays at its initial value (0.81 and 0.64). Figure 11 and the column "nkeep= 0" show why. With $(1, 1, 1)$, every organism predicts a heat transfer rate about 5 times too small, so all the fractional errors are close to 0.81 with a very small spread. No error is below 0.5× median, so `nkeep` = 0 at *every* generation. The offspring are then all mutations of organism `n[0]`, and this organism is never replaced (offspring are stored from index `nkeep+1`). Each generation is a new random cloud around the same frozen parent, nothing accumulates, and a larger mutation amplitude does not help.
- **With MFRAC** $= 1$ (the organisms with an error below the median survive), half of the population survives at every generation. Sets 1 and 2 then converge (target reached after 84 and 55 generations) to the same error as Sets 3 and 4. Changing a single selection parameter turns a frozen algorithm into a robust one.

Effect of the number of generations. Table 13 shows that, once the population is in the right region, NGEN has almost no effect. From $(5, 5, 1)$, 100 generations already give the final error. From $(1, 1, 1)$ with MFRAC $= 1$, about 500–2000 generations are needed. More generations do not lower the validation error either (no over-fitting with only 3 constants): once converged, it stays between 0.0204 and 0.0208.

Table 13: Task 2.5: effect of NGEN (range over 3 seeds; MUT = ±9 %).

NGEN	Set 3 $(5, 5, 1)$, MFRAC $= 0.5$		Set 1 $(1, 1, 1)$, MFRAC $= 1.0$	
	training MAE	validation MAE	training MAE	validation MAE
100	0.0166–0.0166	0.0207–0.0208	0.0168–0.0170	0.0199–0.0224
500	0.0166–0.0166	0.0207–0.0207	0.0166–0.0167	0.0204–0.0208
2000	0.0166–0.0166	0.0207–0.0208	0.0166–0.0166	0.0208–0.0208
6000	0.0166–0.0166	0.0207–0.0208	0.0166–0.0166	0.0207–0.0208
20000	0.0166–0.0166	0.0207–0.0208	0.0166–0.0166	0.0207–0.0208

Best fit. The best run (lowest *training* error; the validation set was not used to select it) is Set 3 $(5, 5, 1)$ with NGEN = 20000 (seed 1):

$n_1 = (UA)_e$	5.705 W/°C
$n_2 = (UA)_c$	5.425 W/°C
$n_3 = \alpha$	1.130
MAE, training set (98 points)	**0.0166**
MAE, validation set (24 points)	**0.0208**
MAE, all 122 points	0.0174
overall conductance $UA_{eff} = [1/n_1 + 1/(n_2(1 + n_3\bar{h}_r/h_{c,ref}))]^{-1}$	2.907 W/°C

Both errors are **far below the 0.045 target**: training 0.0166, validation 0.0208. The deterministic check gives the same training minimum (0.01659). The validation MAE is slightly higher than the training MAE, as expected for points that were not used in the fit. Figure 12 shows the evolution of the run, Figure 13 the predicted versus data heat transfer rates (training points as circles, validation points as squares), and Figure 14 the surface $\dot{q}(T_{a,in}, T_{a,out})$ over the range of the data. The largest deviations are at the smallest heat transfer rates (e.g. 8.6 W for a temperature difference of only 3.4 °C). There, a temperature uncertainty of ±0.1 °C already gives ±3 % on the driving temperature difference.

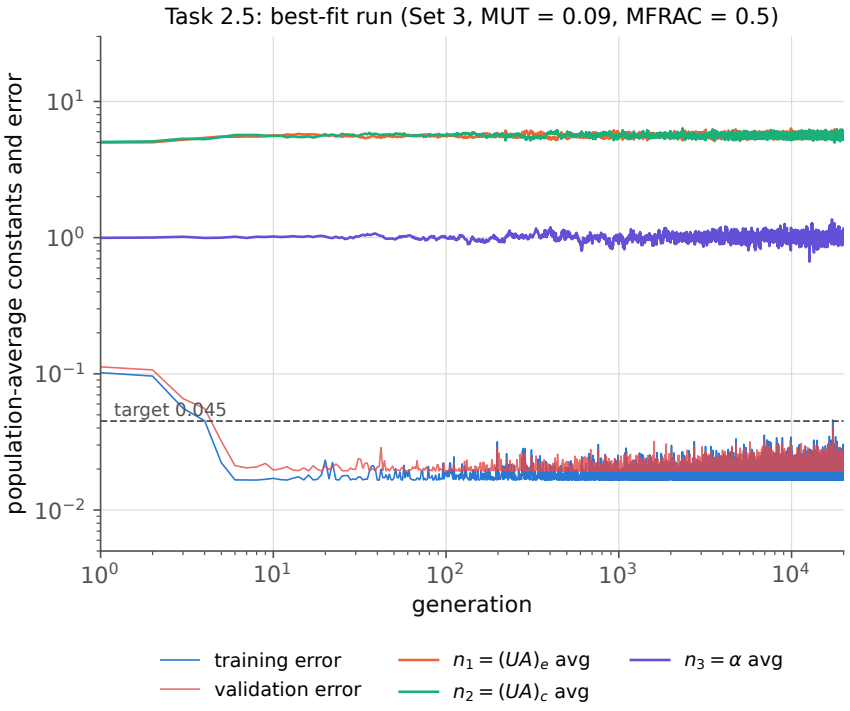


Figure 12: Task 2.5: evolution of the training and validation errors and of the population-average constants for the best-fit run.

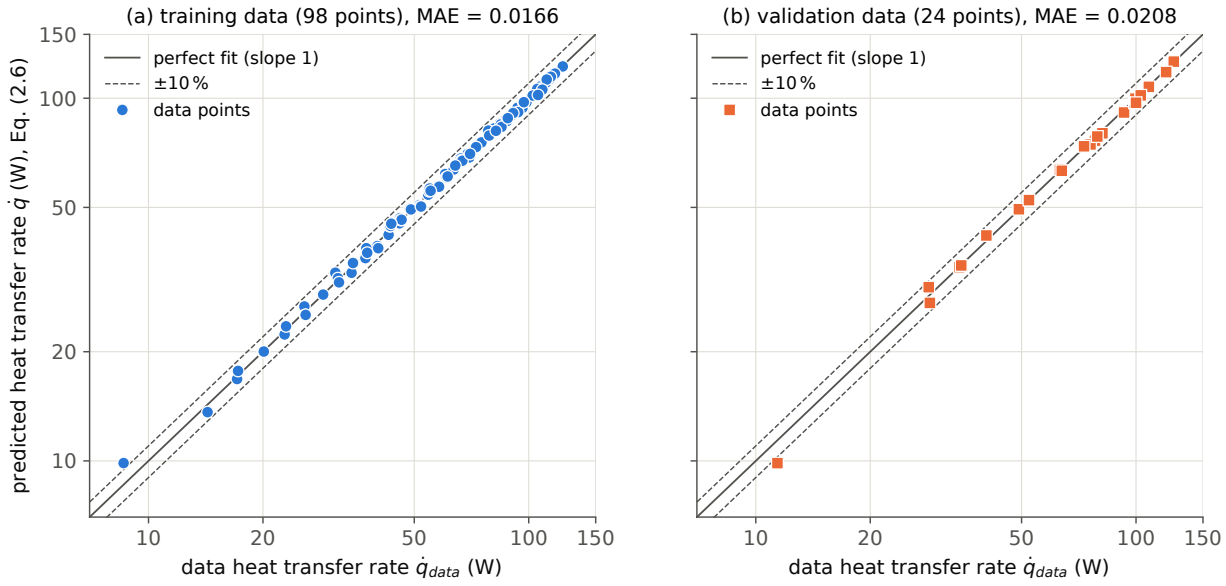


Figure 13: Task 2.5: predicted versus data heat transfer rate (log-log) for (a) the training data (circles) and (b) the validation data (squares).

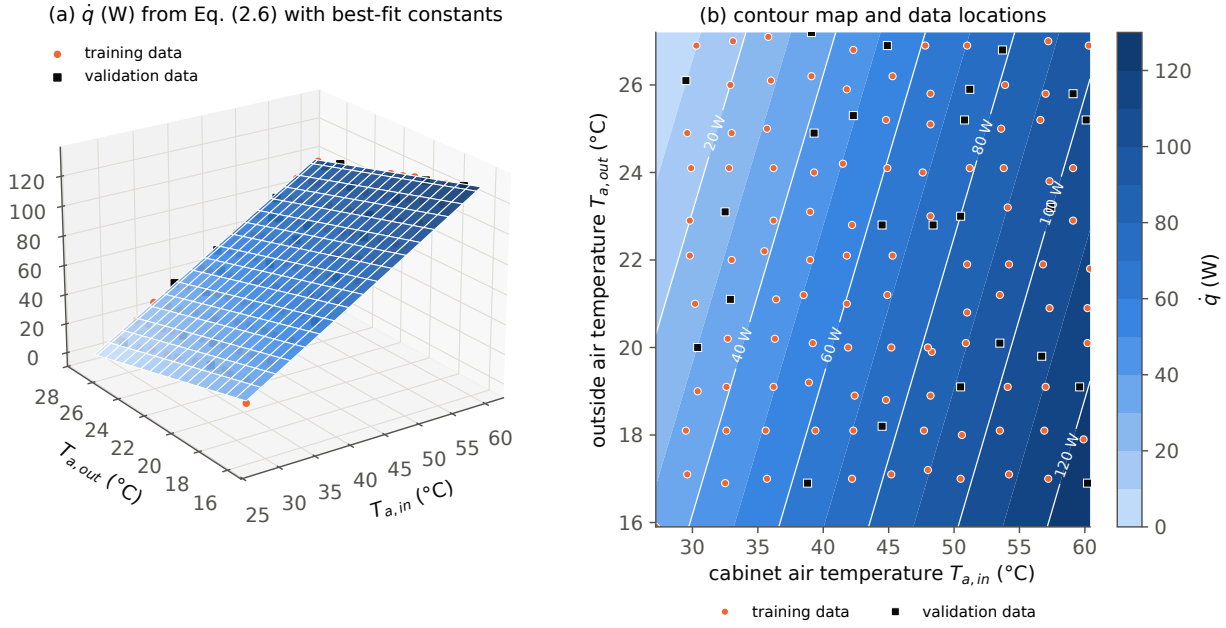


Figure 14: Task 2.5: heat transfer rate predicted by Eq. (2.6) with the best-fit constants as a function of $T_{a,in}$ and $T_{a,out}$ over the range of the data: (a) surface with the data points; (b) contour map.

How unique are $(UA)_e$, $(UA)_c$ and α? Table 12 shows that runs with the same error give quite different constants. Figure 15 explains this. When $(UA)_e$ is fixed anywhere between 3.2 and 1000 W/°C and the two other constants are re-optimised, the training MAE changes only from 0.016588 to 0.016591. The 54 converged GA runs give $(UA)_e$ = 4.96–11.28, $(UA)_c$ = 3.44–6.11 and α = 0.67–2.84, but the series conductance UA_{eff} only varies between 2.893 and 2.911 W/°C (±0.3 %). The data contain only the overall conductance UA_{eff} (two resistances in series) and its weak temperature dependence through h_r. **They cannot separate the evaporator and condenser conductances.** The final constants are therefore set by the initial guesses: the GA stops at the point of the valley closest to where it started. Starting from equal guesses $(UA)_e$ =

$(UA)_c$, as for Set 3, it returns nearly equal values (5.7 and 5.4 W/°C). This is physically sensible, because the evaporator and condenser fin sections of Fig. 2.1 have the same length (0.25 m). It also gives $\alpha \approx 1.1$, so radiation increases the condenser conductance by about 9 %. To determine $(UA)_e$ and $(UA)_c$ separately, an additional measurement would be needed, e.g. the heat-pipe wall (vapour) temperature.

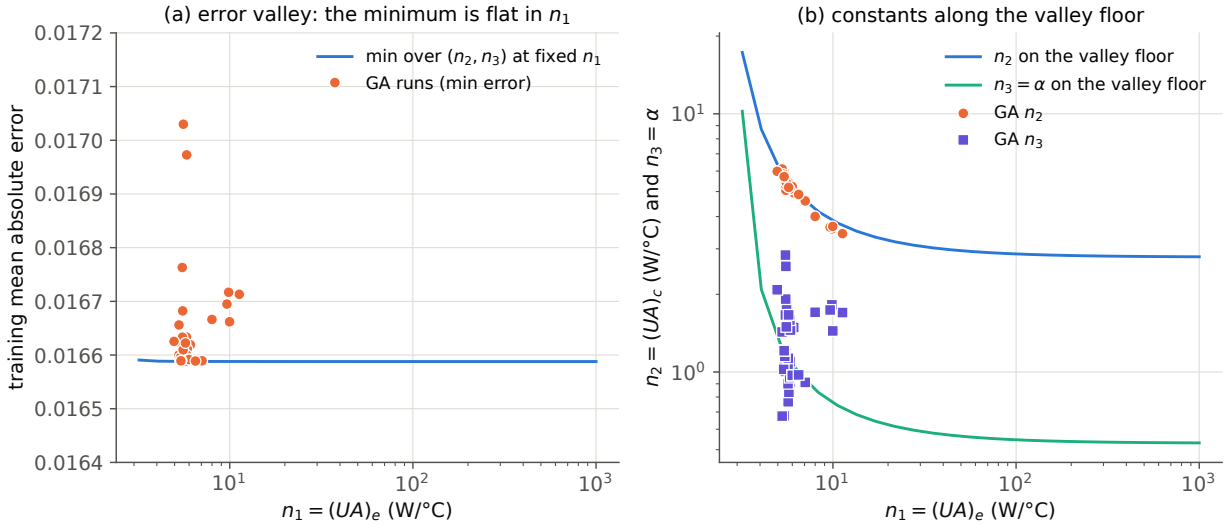


Figure 15: Task 2.5: (a) minimum training error when $(UA)_e$ is fixed and $(UA)_c$, α are re-optimised, with the converged GA runs; (b) values of $(UA)_c$ and α along the bottom of the valley.

4 Conclusions

4.1 Effect of the initial guesses on the stability and speed of convergence

About 300 GA runs were made in this project (Tables 3, 5, 12 and 13). Table 14 summarises how the initial guesses of the constants affect the algorithm.

Table 14: Summary: how the initial guesses affect the stability and the speed of convergence of the genetic algorithm (MFRAC = 0.5 and MUT $\approx \pm 9\,\%$ unless stated).

Initial guesses	Observed behaviour	Cause	Remedy
Close to the solution (predicted output of the right order of magnitude): 2.2 Sets 1–2, 2.3 Set C, 2.5 Sets 3–4	Stable. Target reached in **2–12 generations** in every run; final error within a few percent of the true minimum.	Only the level of the prediction (n_1) has to be adjusted.	Continue with small mutations to refine.
Right order of magnitude but wrong position along a valley: 2.2 Set 1, 2.3 Set A	Target reached quickly, but the constants depend on the guess and on the seed (e.g. $n_2 = 3.08$–3.34) and drift only slowly.	Correlated constants ($\ln n_1$ and n_2; n_3, n_4 and n_5): long, flat valley of the error.	Several seeds and guesses; larger MUT, then refinement; check the constants, not only the error.
Far from the solution: 2.2 Sets 3–4, 2.3 Set D	**Stagnation** at 2–4 times the minimum error. NGEN = 20000 does not help. With MUT $= \pm 40\,\%$: convergence after $\sim$130–1600 generations.	Steps of at most $\pm$MUT per generation; most mutations away from the valley floor are rejected.	Two stages: MUT = 0.2–0.4 to explore, then MUT = 0.03–0.01 to refine.
All organisms with nearly the same large error: 2.3 Set E, 2.5 Sets 1–2 $(1, 1, 1)$, $(2, 2, 0.5)$	**Population frozen**: nkeep = 0 at every generation, error constant (0.6–0.95). Unaffected by MUT and NGEN.	Relative threshold 0.5×median selects nobody; all offspring come from `n[0]`, which is never replaced.	MFRAC = 1 (converges in 55–84 generations), or better guesses.
A constant guessed close to zero: 2.3 Set B ($n_5 = 0.01$)	The constant stays near zero; only 3/10 runs reach the target.	Multiplicative mutation: the step is proportional to the value.	Guess a non-zero value of the expected order of magnitude.
Constants not identifiable from the data: 2.5 ($(UA)_e$, $(UA)_c$, α)	Same error for $(UA)_e$ from 5 to 11 W/°C; the final constants follow the guess.	The data only constrain the series conductance UA_{eff}.	Report UA_{eff}; physically motivated guess $(UA)_e \approx (UA)_c$; more measurements.

In short:

1. **The most important property of the initial guesses is that the model output they predict has the right order of magnitude.** When it does, the GA is stable and fast, whatever the exact values. When all organisms are far from the data, the relative selection threshold (MFRAC × median) stops working and the skeleton algorithm can freeze completely.
2. **Speed of convergence**: with good guesses, the target error is reached in a few to a few tens of generations, and the minimum is found within a few thousand generations. Beyond that, increasing NGEN does not help a trapped population. What helps is a larger mutation amplitude (exploration) followed by a small one (refinement), or a larger MFRAC.
3. **Reaching the target error is not proof of an accurate fit.** With correlated constants, many sets of constants meet the target. To be confident, several seeds and initial guesses must give the same constants. The deterministic check showed that my best GA fits are within 0.03 % (Task 2.2), 0.6 % (Task 2.3) and 0.01 % (Task 2.5) of the true minimum error.
4. **Constants that the data do not determine** (Task 2.5) stay at values that depend on the initial guess. Only identifiable combinations should be interpreted.

4.2 Main results

- **Task 2.2**: $q'' = 1.72 \times 10^{-3}\,(T_w - T_{sat})^{2.9747} g^{0.0438}$, $[F_{err}]_{mean} = 0.02870$. The rms deviation (16.6 %) is larger than the ±10 % uncertainty of the data, and the deviations are systematic between gravity levels. A power law in g cannot represent the Marangoni effect.
- **Task 2.3**: $q'' = 1.20 \times 10^{-4}\,(T_w - T_{sat})^{3.0216} (g + 1.3485\, g_{en}\gamma)^{0.4619} P^{0.5435}$, $[F_{err}]_{mean} = 0.02097$ (validation 0.02566). The overall deviation is close to the noise level, but the $1\,g_{en}$ and $2\,g_{en}$ (5.5 kPa) groups keep a bias of −16 % and +11 %. The Marangoni effect acts like an additional gravity of about $2.4\,g_{en}$ for $\gamma = 1.79$, which explains why the heat flux hardly depends on gravity for this mixture.
- **ET1**: the heat flux is about 6 times more sensitive to the superheat than to pressure, the next most influential input. Keeping $|\Delta q''/q''| < 0.15$ requires about 3.6 % accuracy on all inputs, or about ±1.4–1.9 °C on the superheat.
- **ET2**: the directly fitted inverse relation differs from the inverted forward relation by ≈ 5–10 %. This difference comes from the scatter of the data (regression dilution), as shown with synthetic data.
- **Tasks 2.4–2.5**: $(UA)_e = 5.705$ W/°C, $(UA)_c = 5.425$ W/°C, $\alpha = 1.130$. The MAE is 0.0166 for the training set and 0.0208 for the validation set, well below 0.045. Only the overall conductance $UA_{eff} = 2.907$ W/°C is determined uniquely by the data.

5 Summary of the program modifications (Task 2.6)

This section lists, program by program, the modifications made to the previous version of the code, and all the changes made to the basic structure of the example code. The complete programs are in Appendix A. Every modified line is also marked in the code by a comment.

5.1 Changes to the basic structure of the example code (all GA programs)

S1. Bug fix – `import random`. In `CodeP2.2F26`, `import random` replaces the function `random()` imported in `CodeP2.1F26` by the module, so the combined program stops at the initialisation with a `TypeError`. It was replaced by `from random import random, seed`.

S2. Data corrections. Two values of `CodeP2.1F26` were corrected to match the Appendix (Table 2).

S3. Evolution loop placed in a function. The skeleton code from "INITIALIZING PARAMETERS" to the final print was placed in `run_GA(n1i,...,n5i, NGEN, MUT, MFRAC, rseed, target, verbose)`. This allows many runs (initial guesses, mutation amplitudes, seeds, NGEN) in the same notebook. Inside the function, the statements of the loop (fitness, selection, mating, mutation, averaging, minimum search, time averages) are those of the skeleton, line for line.

S4. Parameters instead of literals. The mutation amplitude (`0.085` in the code) is the argument `MUT`, and the selection fraction `MFRAC` (= 0.5 in the skeleton) is an argument.

S5. Reproducibility. The skeleton seeds Python's `random` once but never seeds `numpy.random`, which is used for selection, mating and mutation. Every run therefore gave different results. `run_GA` now calls `seed(rseed)` and `numpy.random.seed(rseed)`, so any run of this report can be reproduced exactly from its seed.

S6. Diagnostics returned. The function returns the histories (`aFerrmeanavgn`, `n1avg`, ...), the generation of the minimum (`kmin`), the first generation below the target (`kconv`) and the number of survivors of each generation (`nkeephist`). None of these quantities affects the algorithm.

S7. Figures. The skeleton plots are kept (`plt.loglog()`, same legends). The `yticks` were extended down to 10^{-4} so that $n_1 \approx 10^{-3}$ is visible. Additional report-quality figures are saved as PDF files.

5.2 Program-by-program modifications

`P2_Task2.1` (from `CodeP2.1F26`). S2; a print of the number of rows and of one element; a cell that checks the data against the Appendix; a cell for the log-log plot and power-law fits of Task 2.1(b).

`P2_Task2.2` (from `CodeP2.1F26` + `CodeP2.2F26`). S1–S7. The error function of the skeleton (Eq. (1.3)) is unchanged. Added cells: first run with the skeleton guesses, deterministic reference, initial-guess study, best-fit plots and statistics, landscape.

`P2_Task2.3` (copy of `P2_Task2.2`). Modifications (i)–(vii) of the statement:

(i) the red-text data lines (32 rows with $P = 9.5$ and 7.0 kPa, $\gamma = 0$ and 1.71) are uncommented, giving 77 rows;

(ii) `ND = 77`, `NS = 77`;

(iii) the error function becomes Eq. (1.7) in both locations (fitness loop and population-average loop), and the prediction used in the plots becomes Eq. (1.5):

```
gen_en = 9.8   # earth-normal gravitational acceleration, m/s2
Ferr[i] = n[i][0]*lydata[i][0] + math.log(n[i][1]) + n[i][2]*lydata[i][1]
Ferr[i] = Ferr[i] + n[i][3]*math.log( ydata[i][2] + n[i][4]*gen_en*ydata[i][3] ) + n[i][5]*lydata[i][4]
...
Ferravgn[i] = -1.*lydata[i][0] + math.log(n1avg[k]) + n2avg[k]*lydata[i][1]
Ferravgn[i] = Ferravgn[i] + n3avg[k]*math.log( ydata[i][2] + n4avg[k]*gen_en*ydata[i][3] ) + n5avg[k]*
    lydata[i][4]
...
qpred = n1*row[1]**n2 * (row[2] + n4*gen_en*row[3])**n3 * row[4]**n5
```

(iv) the mating/mutation lines of n_4 and n_5 are uncommented (with amplitude `MUT`);

(v) the final print-out lists n_1–n_5 (`ENDING`, `MINUMUM`, `TIME AVG`);

(vi) the legend of the first plot is replaced by the five-constant legend and n_4, n_5 are plotted;

(vii) the default initial guesses are 0.000476, 3.028, 0.2249, 1.054, 0.217.

Added: the 15 validation points of the Appendix (`vdata`, used only to test the fit), the study over initial guesses and the multi-stage refinement, the surface plot, and the profiles of Fig. 6.

`P2_ET1` (new). Reads the best-fit constants of Task 2.3 and evaluates the fractional change ratios with finite steps, following steps 1–3 of the statement. It also computes the analytical logarithmic derivatives and the limiting input inaccuracies.

`P2_ET2` (copy of `P2_Task2.3`). The data rows are reorganised as $[T_w - T_{sat},\ q'',\ g,\ \gamma,\ P]$ and `lydata` is recomputed. Because column 0 is now $\ln(T_w - T_{sat})$ and column 1 is $\ln q''$, the error statements of Task 2.3 compute exactly the error of Eq. (ET2.1). Only the initial guesses (negative $\hat{n}_3$, $\hat{n}_5$: the multiplicative mutation keeps the sign), the labels and the analysis cells were changed.

`P2_Task2.4-2.5` (copy of `P2_Task2.2`).

- The data are those of `CodeP2.3F26` (122 points $[T_{a,in}, T_{a,out}, \dot{q}]$). A seeded random permutation splits them into a training set (`ydata`, $N_D = 98$, 80.3 %) and a validation set (`vdata`, $N_V = 24$, 19.7 %). The GA sees only the training set, and $N_S = N_D = 98$.

- The error function was changed in the two locations, Eqs. (2.4)–(2.5), with $n_1 = (UA)_e$, $n_2 = (UA)_c$, $n_3 = \alpha$. The model Eq. (2.6) is coded once and used in both locations and in the plots:

```
sigma = 5.67e-8      # Stefan-Boltzmann constant, W m-2 K-4
hcref = 80.0         # W m-2 K-1
def hrad(Tain, Taout):
    Ti = Tain + 273.15
    To = Taout + 273.15
    return sigma*(Ti + To)*(Ti**2 + To**2)
def qdot_model(n1, n2, n3, Tain, Taout):
    UAcr = n2*(1. + n3*hrad(Tain, Taout)/hcref)
    return UAcr*( n1*(Tain - Taout)/(n1 + UAcr) )
...
Ferr[i] = ydata[i][2] - qdot_model(n[i][1], n[i][2], n[i][3], ydata[i][0], ydata[i][1])
aFerr[i] = abs(Ferr[i])/abs(ydata[i][2])  #- absolute fractional error
...
Ferravgn[i] = ydata[i][2] - qdot_model(n1avg[k], n2avg[k], n3avg[k], ydata[i][0], ydata[i][1])
aFerravgn[i] = abs(Ferravgn[i])/abs(ydata[i][2])
```

- The mean absolute error of the validation set is computed at each generation with the population-average constants (`aFerrval`). It is used for monitoring only, not for selection.
- The plotting section was rewritten for the log-log plots of predicted versus data $\dot{q}$ (training and validation) and for the surface plot of $\dot{q}(T_{a,in}, T_{a,out})$.

A Programs

This appendix lists all the programs (code cells of the Jupyter notebooks in `code/`, executed in order). To avoid repeating the same code, a cell that is identical to a cell already listed (the data cell `CodeP2.1F26` and the figure-style cell) is replaced by a reference to the first listing. Modified or added lines are marked by comments. The executed notebooks, with all their outputs, are provided with the report.

A.1 Program A – Task 2.1 (`P2_Task2.1.ipynb`)

```
# ============================================================================
# Program A : P2_Task2.1.ipynb
# ============================================================================

# ------------------------------ Cell 1 ------------------------------
'''>>>>> start CodeP2.1F26
    V.P. Carey ME249, Fall 2026
    MODIFIED for Task 2.1 / 2.2:
      - two data values corrected to match the Appendix listing of the project
        statement: row 0  q" 44.2 -> 44.1 W/cm2,  row 6  superheat 37.8 -> 37.5 C
      - everything else unchanged (red data with P other than 5.5 kPa still
        commented out, ND = NS = 45)'''

# version 3 print function
from __future__ import print_function
# seed the pseudorandom number generator
from random import random
from random import seed
# seed random number generator
seed(1)

#import math and numpy packages
import math
import numpy

%matplotlib inline
# importing the required module
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [10, 8] # for square canvas

```

```

#import copy
from copy import copy, deepcopy


#create arrays
ydata = []
lydata = []

#Parameters for Evolution Loop
#set data parameters
ND = 45        #number of data vectors in array
DI = 5         #number of data items in vector
NS = 45        #total number of DNA strands

# j is column, i is row downward for ydata[i][j] - both start at zero
# so it is:  ydata[row][column]
# this is an array that is essentially a list of lists

#assembling data array
#store array where rows are data vectors
#[heat flux, superheat, gravity, surface tension parameter, pressure]

ydata =     [[44.1, 32.5, 0.098, 1.79, 5.5]]    # corrected (was 44.2)
ydata.append([47.4, 33.2, 0.098, 1.79, 5.5])
ydata.append([49.4, 34.2, 0.098, 1.79, 5.5])

ydata.append([59.2, 34.8, 0.098, 1.79, 5.5])
ydata.append([67.8, 36.3, 0.098, 1.79, 5.5])
ydata.append([73.6, 37.3, 0.098, 1.79, 5.5])
ydata.append([76.3, 37.5, 0.098, 1.79, 5.5])     # corrected (was 37.8)
ydata.append([85.3, 39.2, 0.098, 1.79, 5.5])
ydata.append([96.5, 39.3, 0.098, 1.79, 5.5])
ydata.append([111., 42.3, 0.098, 1.79, 5.5])
ydata.append([124., 43.5, 0.098, 1.79, 5.5])
ydata.append([136.2, 45.4, 0.098, 1.79, 5.5])

ydata.append([143.5, 46.7, 0.098, 1.79, 5.5])
ydata.append([154.6, 47.9, 0.098, 1.79, 5.5])
ydata.append([163.1, 48.6, 0.098, 1.79, 5.5])
ydata.append([172.8, 50.9, 0.098, 1.79, 5.5])
ydata.append([184.2, 51.7, 0.098, 1.79, 5.5])
ydata.append([203.7, 56.4, 0.098, 1.79, 5.5])

ydata.append([36.7, 30.2, 9.8, 1.79, 5.5])
ydata.append([55.1, 34.1, 9.8, 1.79, 5.5])
ydata.append([67.5, 35.3, 9.8, 1.79, 5.5])
ydata.append([78.0, 37.8, 9.8, 1.79, 5.5])
ydata.append([92.0, 38.1, 9.8, 1.79, 5.5])
ydata.append([120., 44.1, 9.8, 1.79, 5.5])
ydata.append([134.3, 46.9, 9.8, 1.79, 5.5])
ydata.append([150.3, 48.5, 9.8, 1.79, 5.5])
ydata.append([167., 49.2, 9.8, 1.79, 5.5])
ydata.append([184., 52.7, 9.8, 1.79, 5.5])
ydata.append([196.5, 53.1, 9.8, 1.79, 5.5])
'''
ydata.append([42.4, 28.0, 19.6, 1.79, 9.5])
ydata.append([48.7, 29.3, 19.6, 1.79, 9.5])
ydata.append([54.5, 29.6, 19.6, 1.79, 9.5])

ydata.append([62.1, 28.5, 19.6, 1.79, 9.5])
ydata.append([70.8, 30.5, 19.6, 1.79, 9.5])
ydata.append([73.7, 30.3, 19.6, 1.79, 9.5])
ydata.append([81.8, 30.6, 19.6, 1.79, 9.5])
ydata.append([91.9, 34.5, 19.6, 1.79, 9.5])
ydata.append([103.9, 34.5, 19.6, 1.79, 9.5])
ydata.append([119.1, 35.4, 19.6, 1.79, 9.5])
ydata.append([133.7, 36.8, 19.6, 1.79, 9.5])
ydata.append([139.9, 38.1, 19.6, 1.79, 9.5])
ydata.append([148.3, 39.1, 19.6, 1.79, 9.5])
ydata.append([157.0, 40.0, 19.6, 1.79, 9.5])
ydata.append([169.1, 42.2, 19.6, 1.79, 9.5])
ydata.append([179.2, 43.2, 19.6, 1.79, 9.5])
ydata.append([205.0, 46.0, 19.6, 1.79, 9.5])
'''
```

```
ydata.append([42.4, 29.7, 19.6, 1.79, 5.5])
ydata.append([48.7, 31.0, 19.6, 1.79, 5.5])
ydata.append([54.5, 31.2, 19.6, 1.79, 5.5])
ydata.append([70.8, 32.4, 19.6, 1.79, 5.5])
ydata.append([73.7, 31.4, 19.6, 1.79, 5.5])
ydata.append([81.8, 32.5, 19.6, 1.79, 5.5])
ydata.append([91.9, 36.3, 19.6, 1.79, 5.5])
ydata.append([103.9, 36.3, 19.6, 1.79, 5.5])
ydata.append([119.1, 37.2, 19.6, 1.79, 5.5])
ydata.append([133.7, 38.4, 19.6, 1.79, 5.5])
ydata.append([139.9, 39.7, 19.6, 1.79, 5.5])
ydata.append([148.3, 40.9, 19.6, 1.79, 5.5])
ydata.append([157.0, 41.6, 19.6, 1.79, 5.5])
ydata.append([169.1, 43.9, 19.6, 1.79, 5.5])
ydata.append([179.2, 45.0, 19.6, 1.79, 5.5])
ydata.append([205.0, 47.9, 19.6, 1.79, 5.5])
'''
ydata.append([77.0, 41.5, 9.8, 0.00, 7.0])
ydata.append([71.0, 40.5, 9.8, 0.00, 7.0])
ydata.append([66.0, 39.5, 9.8, 0.00, 7.0])
ydata.append([62.0, 38.5, 9.8, 0.00, 7.0])
ydata.append([42.0, 34.0, 9.8, 0.00, 7.0])
ydata.append([60.0, 37.5, 9.8, 0.00, 7.0])
ydata.append([53.0, 37.0, 9.8, 0.00, 7.0])

ydata.append([71.7, 36.4, 0.098, 1.71, 5.5])
ydata.append([81.5, 38.5, 0.098, 1.71, 5.5])
ydata.append([90.7, 39.5, 0.098, 1.71, 5.5])
ydata.append([103.3, 41.6, 0.098, 1.71, 5.5])
ydata.append([117.0, 43.1, 0.098, 1.71, 5.5])
ydata.append([138.6, 45.4, 0.098, 1.71, 5.5])
ydata.append([161.7, 47.9, 0.098, 1.71, 5.5])
ydata.append([207.5, 50.9, 0.098, 1.71, 5.5])
'''
# print the data array
print ('ydata =', ydata)
# Task 2.1(a): check of the array size and of a single array element
print ('number of rows =', len(ydata), ' ;  ydata[1][0] =', ydata[1][0], ' ;  ydata[6][1] =', ydata[6][1])


''' need deepcopy to create an array of the same size as ydata,
#   since this array is a list(rows) of lists (column entries)  '''
lydata = deepcopy(ydata)  # create array to store ln of data values


# j is column, i is row downward for ydata[i][j] - both start at zero
# so it is:  ydata[row][column]
#now store log values for data
for j in range(DI):
    for i in range(ND):
        lydata[i][j]=math.log(ydata[i][j]+0.000000000010)

#OK now have stored array of log values for data

#end CodeP2.1F26

# ------------------------------ Cell 2 ------------------------------
'''(added) common figure style for the report figures + output folders'''
import os, json, logging
logging.getLogger('fontTools').setLevel(logging.ERROR)   # silence font-embedding warnings
import matplotlib as mpl
from matplotlib.colors import LinearSegmentedColormap

# categorical colours (fixed order) and single-hue sequential ramp for surfaces
C = ['#2a78d6', '#eb6834', '#1baf7a', '#eda100', '#e87ba4', '#008300', '#6250d6', '#e34948']
MK = ['o', 's', '^', 'D', 'v', 'P', 'X', '*']          # marker shapes = secondary encoding
INK, INK2, GRID = '#0b0b0b', '#52514e', '#dedcd5'
BLUES = LinearSegmentedColormap.from_list('blues', ['#cde2fb', '#86b6ef', '#3987e5', '#256abf', '#184f95', '
    #0d366b'])

mpl.rcParams.update({
    'font.size': 11, 'axes.titlesize': 11, 'axes.labelsize': 11, 'legend.fontsize': 9.5,
    'axes.edgecolor': INK2, 'axes.labelcolor': INK, 'xtick.color': INK2, 'ytick.color': INK2,
    'axes.grid': True, 'grid.color': GRID, 'grid.linewidth': 0.6, 'axes.axisbelow': True,
    'axes.spines.top': False, 'axes.spines.right': False,
```

```
    'lines.linewidth': 1.6, 'lines.markersize': 6, 'legend.frameon': False,
    'figure.figsize': [6.4, 4.6], 'savefig.dpi': 200, 'savefig.bbox': 'tight', 'pdf.fonttype': 42})

FIGDIR = '../report/figures/'      # figures used in the LaTeX report
RESDIR = 'results/'                # numerical results (json) used in the report tables
os.makedirs(FIGDIR, exist_ok=True)
os.makedirs(RESDIR, exist_ok=True)

def savefig(name):
    plt.savefig(FIGDIR + name + '.pdf')
    plt.show()

def first_below(hist, thr):
    '''first generation at which a history drops below thr (-1 if never)'''
    for k, v in enumerate(hist):
        if v < thr:
            return k
    return -1

def sci(x, d=2):
    '''LaTeX scientific notation for figure labels, e.g. 1.73 x 10^-3'''
    e = int(math.floor(math.log10(abs(x))))
    return r'%.*f\times10^{%d}' % (d, x / 10**e, e)

def plain_log_ticks(ax, xt=None, yt=None):
    '''plain (non-scientific) tick labels on log axes'''
    from matplotlib.ticker import NullFormatter, NullLocator
    if xt is not None:
        ax.set_xticks(xt); ax.set_xticklabels(['%g' % v for v in xt])
        ax.xaxis.set_minor_locator(NullLocator()); ax.xaxis.set_minor_formatter(NullFormatter())
    if yt is not None:
        ax.set_yticks(yt); ax.set_yticklabels(['%g' % v for v in yt])
        ax.yaxis.set_minor_locator(NullLocator()); ax.yaxis.set_minor_formatter(NullFormatter())

def best_so_far(h):
    return numpy.minimum.accumulate(numpy.array(h))

# ------------------------------ Cell 3 ------------------------------
'''(added) Task 2.1(a): check of the data array against the Appendix of the project statement'''
import re

# Appendix - data for Part 1 : [q", Tw-Tsat, g, gamma, P]  (77 rows, black + red text)
appendix = [[44.1, 32.5, 0.098, 1.79, 5.5], [47.4, 33.2, 0.098, 1.79, 5.5], [49.4, 34.2, 0.098, 1.79, 5.5],
     [59.2, 34.8, 0.098, 1.79, 5.5],
            [67.8, 36.3, 0.098, 1.79, 5.5], [73.6, 37.3, 0.098, 1.79, 5.5], [76.3, 37.5, 0.098, 1.79, 5.5],
     [85.3, 39.2, 0.098, 1.79, 5.5],
            [96.5, 39.3, 0.098, 1.79, 5.5], [111.0, 42.3, 0.098, 1.79, 5.5], [124.0, 43.5, 0.098, 1.79, 5.5],
      [136.2, 45.4, 0.098, 1.79, 5.5],
            [143.5, 46.7, 0.098, 1.79, 5.5], [154.6, 47.9, 0.098, 1.79, 5.5], [163.1, 48.6, 0.098, 1.79,
     5.5], [172.8, 50.9, 0.098, 1.79, 5.5],
            [184.2, 51.7, 0.098, 1.79, 5.5], [203.7, 56.4, 0.098, 1.79, 5.5], [36.7, 30.2, 9.8, 1.79, 5.5],
     [55.1, 34.1, 9.8, 1.79, 5.5],
            [67.5, 35.3, 9.8, 1.79, 5.5], [78.0, 37.8, 9.8, 1.79, 5.5], [92.0, 38.1, 9.8, 1.79, 5.5], [120.0,
      44.1, 9.8, 1.79, 5.5],
            [134.3, 46.9, 9.8, 1.79, 5.5], [150.3, 48.5, 9.8, 1.79, 5.5], [167.0, 49.2, 9.8, 1.79, 5.5],
     [184.0, 52.7, 9.8, 1.79, 5.5],
            [196.5, 53.1, 9.8, 1.79, 5.5], [42.4, 28.0, 19.6, 1.79, 9.5], [48.7, 29.3, 19.6, 1.79, 9.5],
     [54.5, 29.6, 19.6, 1.79, 9.5],
            [62.1, 28.5, 19.6, 1.79, 9.5], [70.8, 30.5, 19.6, 1.79, 9.5], [73.7, 30.3, 19.6, 1.79, 9.5],
     [81.8, 30.6, 19.6, 1.79, 9.5],
            [91.9, 34.5, 19.6, 1.79, 9.5], [103.9, 34.5, 19.6, 1.79, 9.5], [119.1, 35.4, 19.6, 1.79, 9.5],
     [133.7, 36.8, 19.6, 1.79, 9.5],
            [139.9, 38.1, 19.6, 1.79, 9.5], [148.3, 39.1, 19.6, 1.79, 9.5], [157.0, 40.0, 19.6, 1.79, 9.5],
     [169.1, 42.2, 19.6, 1.79, 9.5],
            [179.2, 43.2, 19.6, 1.79, 9.5], [205.0, 46.0, 19.6, 1.79, 9.5], [42.4, 29.7, 19.6, 1.79, 5.5],
     [48.7, 31.0, 19.6, 1.79, 5.5],
            [54.5, 31.2, 19.6, 1.79, 5.5], [70.8, 32.4, 19.6, 1.79, 5.5], [73.7, 31.4, 19.6, 1.79, 5.5],
     [81.8, 32.5, 19.6, 1.79, 5.5],
            [91.9, 36.3, 19.6, 1.79, 5.5], [103.9, 36.3, 19.6, 1.79, 5.5], [119.1, 37.2, 19.6, 1.79, 5.5],
     [133.7, 38.4, 19.6, 1.79, 5.5],
            [139.9, 39.7, 19.6, 1.79, 5.5], [148.3, 40.9, 19.6, 1.79, 5.5], [157.0, 41.6, 19.6, 1.79, 5.5],
     [169.1, 43.9, 19.6, 1.79, 5.5],
            [179.2, 45.0, 19.6, 1.79, 5.5], [205.0, 47.9, 19.6, 1.79, 5.5], [77.0, 41.5, 9.8, 0.0, 7.0],
     [71.0, 40.5, 9.8, 0.0, 7.0],
```

```
            [66.0, 39.5, 9.8, 0.0, 7.0], [62.0, 38.5, 9.8, 0.0, 7.0], [42.0, 34.0, 9.8, 0.0, 7.0], [60.0,
    37.5, 9.8, 0.0, 7.0],
            [53.0, 37.0, 9.8, 0.0, 7.0], [71.7, 36.4, 0.098, 1.71, 5.5], [81.5, 38.5, 0.098, 1.71, 5.5],
    [90.7, 39.5, 0.098, 1.71, 5.5],
            [103.3, 41.6, 0.098, 1.71, 5.5], [117.0, 43.1, 0.098, 1.71, 5.5], [138.6, 45.4, 0.098, 1.71,
    5.5], [161.7, 47.9, 0.098, 1.71, 5.5],
            [207.5, 50.9, 0.098, 1.71, 5.5]]
black = list(range(0, 29)) + list(range(46, 62))      # rows printed in black in the Appendix (P = 5.5 kPa,
    gamma = 1.79)

# (1) rows of the ORIGINAL skeleton file CodeP2.1F26 (active and commented-out lines)
src = ''.join(json.load(open('../CodeP2.1F26.ipynb'))['cells'][0]['source'])
block = src.split('ydata =     [[')[1].split('# print the data array')[0]
skel = [[float(x) for x in r.split(',')] for r in re.findall(r'\[([-\d., ]+)\]', '[' + block)]
print('original skeleton: %d rows   Appendix: %d rows' % (len(skel), len(appendix)))
for i, (a, b) in enumerate(zip(skel, appendix)):
    if any(abs(x - y) > 1e-9 for x, y in zip(a, b)):
        print('   row %2d differs:  skeleton %s   Appendix %s' % (i, a, b))

# (2) corrected array used in this program (45 black rows)
ndiff = sum(any(abs(x - y) > 1e-9 for x, y in zip(ydata[i], appendix[j])) for i, j in enumerate(black))
print('corrected ydata: %d rows, %d differences with the black rows of the Appendix' % (len(ydata), ndiff))
print('single element check: ydata[3][0] =', ydata[3][0], ' (Appendix row 4, heat flux 59.2 W/cm2)')

# ------------------------------ Cell 4 ------------------------------
'''(added) Task 2.1(b): log-log plot of heat flux versus wall superheat for g = 0.098 and 9.8 m/s2
   (first two groups of the Appendix, P = 5.5 kPa, gamma = 1.79) + power-law fit q" = C (Tw-Tsat)^m
   of each group (least squares on ln q" vs ln(Tw-Tsat)) to quantify the trends'''
yarr = numpy.array(ydata)
fits21 = {}
fig, ax = plt.subplots(figsize=(6.4, 4.8))
for c, (gval, lab) in enumerate([(0.098, r'$g = 0.098$ m/s$^2$ ($0.01\,g_{en}$)'), (9.8, r'$g = 9.8$ m/s$^2$
    ($1\,g_{en}$)')]):
    sel = yarr[:, 2] == gval
    dT, qpp = yarr[sel, 1], yarr[sel, 0]
    m, lnC = numpy.polyfit(numpy.log(dT), numpy.log(qpp), 1)
    fits21[str(gval)] = dict(m=m, C=math.exp(lnC), npts=int(sel.sum()))
    ax.loglog(dT, qpp, MK[c], color=C[c], ms=7, mec='white', mew=0.8, label=lab + ' data')
    xx = numpy.linspace(dT.min() * 0.97, dT.max() * 1.03, 50)
    ax.loglog(xx, math.exp(lnC) * xx**m, '-', color=C[c], lw=1.4,
              label=r'fit $q^{\prime\prime} = %s\,(T_w-T_{sat})^{%.2f}$' % (sci(math.exp(lnC)), m))
    print('g = %6.3f m/s2 : %2d points, q" = %.4e (Tw-Tsat)^%.3f' % (gval, sel.sum(), math.exp(lnC), m))
ax.set_xlabel(r'wall superheat $T_w - T_{sat}$ (°C)')
ax.set_ylabel(r'heat flux $q^{\prime\prime}$ (W/cm$^2$)')
ax.set_xlim(28, 60); ax.set_ylim(30, 250)
ax.set_xticks([30, 35, 40, 45, 50, 55, 60]); ax.set_xticklabels(['30', '35', '40', '45', '50', '55', '60'])
ax.set_yticks([30, 50, 70, 100, 150, 200]); ax.set_yticklabels(['30', '50', '70', '100', '150', '200'])
ax.minorticks_off()
ax.legend(loc='upper left')
ax.set_title(r'Task 2.1(b): water/2-propanol, $P = 5.5$ kPa, $\gamma = 1.79$')
savefig('t21_loglog')

# ratio of the two fitted curves at a few superheats (gravity effect at fixed superheat)
for dTv in [32., 40., 50.]:
    q1 = fits21['0.098']['C'] * dTv**fits21['0.098']['m']
    q2 = fits21['9.8']['C'] * dTv**fits21['9.8']['m']
    print('Tw-Tsat = %4.1f C : q"(0.098) = %6.1f  q"(9.8) = %6.1f  ratio q"(9.8)/q"(0.098) = %.3f' % (dTv, q1
    , q2, q2 / q1))
json.dump(fits21, open(RESDIR + 'task21.json', 'w'), indent=1)
```

A.2 Program B – Task 2.2 (`P2_Task2.2.ipynb`)

```
# ============================================================================
# Program B : P2_Task2.2.ipynb
# ============================================================================

# ------------------------------ Cell 1 ------------------------------
# identical to Program A, Cell 1 (listed above)

# ------------------------------ Cell 2 ------------------------------
'''>>>>> start CodeP2.2F26
    V.P. Carey ME249, Fall 2026
```

```
    MODIFIED for Task 2.2 (three-constant model, Eq. 1.1):
      - the evolution loop of the skeleton code is placed, unchanged, inside a
        function run_GA(...) so that it can be re-run for several initial guesses,
        mutation amplitudes, numbers of generations and random seeds
      - the maximum mutation amplitude (0.085 in the skeleton) is now the
        argument MUT (relative amplitude, i.e. +/- MUT*100 %)
      - both random generators (python random and numpy.random) are seeded with
        the argument rseed so that every run is reproducible
      - the history of the population-average constants and of aFerrmeanavgn is
        returned, together with the generation at which aFerrmeanavgn first drops
        below the target value (convergence-speed indicator) and the number of
        survivors nkeep of each generation (selection diagnostic)
      - BUG FIX: the skeleton line "import random" re-binds the name random to the
        module and hides the function random() imported in CodeP2.1F26, so the
        initialisation n1i+0.001*random() raises "TypeError: 'module' object is
        not callable".  The line is replaced by an explicit import of the function.'''
from random import random, seed   # (was: import random)

def run_GA(n1i, n2i, n3i, n4i=1.215, n5i=0.145, NGEN=6000, MUT=0.085, MFRAC=0.5,
           rseed=1, target=0.035, verbose=True):

    '''INITIALIZING PARAMETERS'''
    seed(rseed)                 # python random  (initial population)
    numpy.random.seed(rseed)    # numpy random   (mating / mutation)
    n = []
    ntemp = []
    gen=[0]
    n1avg = [0.0]
    n2avg = [0.0]
    n3avg = [0.0]
    n4avg = [0.0]
    n5avg = [0.0]
    meanAFerr=[0.0]
    aFerrmeanavgn=[0.0]

    # here the number of data vectors equals the number of DNA strands (or organisms)
    for k in range(NGEN-1):
        gen.append(k+1)   # generation array stores the
        meanAFerr.append(0.0)
        aFerrmeanavgn.append(0.0)
        n1avg.append(0.0)
        n2avg.append(0.0)
        n3avg.append(0.0)
        n4avg.append(0.0)
        n5avg.append(0.0)

    '''guesses for initial solution population: n0i = -1 (fixed), n1i ... n5i are arguments'''
    n0i =  -1.0

    #i initialize array where rows are dna vectors [n0i,n1i,...n5i] with random pertubations
    n =  [[-1., n1i+0.001*random(), n2i+0.1*random(), n3i+0.0001*random(), n4i+0.0001*random(), n5i+0.0001*
     random()]]
    for i in range(ND):
        n.append([-1., n1i+0.0001*random(), n2i+0.001*random(), n3i+0.0001*random(), n4i+0.0001*random(), n5i
     +0.0001*random()])

    # store also in wtemp
    ntemp = deepcopy(n)

    #define arrays of Ferr (error) functions
    Ferr =  [[0.0]]
    Ferravgn =  [[0.0]]
    aFerr =  [[0.0]]
    aFerravgn =  [[0.0]]
    for i in range(ND-1):
        Ferr.append([0.0])
        aFerr.append([0.0])
        Ferravgn.append([0.0])
        aFerravgn.append([0.0])

    aFerrmeanavgnMin=1000000000.0
    # these store the  n values for minimum population average error durng NGEN generations
    n1min = 0.0
    n2min = 0.0
    n3min = 0.0
```

```
    n4min = 0.0
    n5min = 0.0
    kmin = 0
    aFerrta = 0.0
    # these store the time averaged n values durng from generation 800 to NGEN generations
    n1ta = 0.0
    n2ta = 0.0
    n3ta = 0.0
    n4ta = 0.0
    n5ta = 0.0
    kconv = -1      # first generation with aFerrmeanavgn < target (-1 = never)
    nkeephist = []  # (added) number of survivors nkeep at each generation

    '''START OF EVOLUTION LOOP'''
    for k in range(NGEN):

        '''CALCULATING ERROR (FITNESS) - Eq. (1.3): organism i is paired with data point i'''
        for i in range(ND):

            Ferr[i] = n[i][0]*lydata[i][0] + math.log(n[i][1]) + n[i][2]*lydata[i][1]
            Ferr[i] = Ferr[i] + n[i][3]*math.log( ydata[i][2] )

            aFerr[i] = abs(Ferr[i])/abs(lydata[i][0])  #- absolute fractional error
        #-------------
        aFerrmean = numpy.mean(aFerr) #mean error for population for this generation
        meanAFerr[k]=aFerrmean  #store aFerrmean for this generation gen[k]=k
        aFerrmedian = numpy.median(aFerr) #median error for population for this generation

        '''SELECTION'''
        clim = MFRAC*aFerrmedian  #cut off limit is a fraction/multiplier MFRAC of median value for population
        nkeep = 0
        for j in range(NS):  # NS Ferr values, one for each solution in population
            if (aFerr[j] < clim):
                nkeep = nkeep + 1
                #ntemp[nkeep][0] = n[j][0] = -1 so it is unchanged;
                ntemp[nkeep-1][1] = n[j][1];
                ntemp[nkeep-1][2] = n[j][2];
                ntemp[nkeep-1][3] = n[j][3];
                ntemp[nkeep-1][4] = n[j][4];
                ntemp[nkeep-1][5] = n[j][5];
        #compute number to be added by mating
        nnew = NS - nkeep
        nkeephist.append(nkeep)   # (added) number of survivors, diagnostic of selection pressure

        '''MATING'''
        for j in range(nnew):
            # pick two survivors randomly
            nmate1 = numpy.random.randint(low=0, high=nkeep+1)
            nmate2 = numpy.random.randint(low=0, high=nkeep+1)

            '''here, do not change property ntemp[nkeep+j+1][0], it's always fixed at the value -1'''
            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][1] = n[nmate1][1]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 1,
    mutation added
            else:
                ntemp[nkeep+j+1][1] = n[nmate2][1]*(1.+MUT*2.*(0.5-numpy.random.rand()))

            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][2] = n[nmate1][2]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 2,
    mutation added
            else:
                ntemp[nkeep+j+1][2] = n[nmate2][2]*(1.+MUT*2.*(0.5-numpy.random.rand()))

            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][3] = n[nmate1][3]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 3,
    mutation added
            else:
                ntemp[nkeep+j+1][3] = n[nmate2][3]*(1.+MUT*2.*(0.5-numpy.random.rand()))
            # properties 4 and 5 are not used by the three-constant model (left commented as in skeleton)
        #============================================
        n = deepcopy(ntemp)   # save ntemp as n for use in next generation (next iteration)

        '''AVERAGING OVER POPULATION AND OVER TIME, FINDING MINIMUM ERROR SET OF n VALUES'''
        n1avg[k] = 0.0;
        n2avg[k] = 0.0;
```

```
        n3avg[k] = 0.0;
        n4avg[k] = 0.0;
        n5avg[k] = 0.0;
        for j in range(NS):
            n1avg[k] = n1avg[k] + n[j][1]/NS;
            n2avg[k] = n2avg[k] + n[j][2]/NS;
            n3avg[k] = n3avg[k] + n[j][3]/NS;
            n4avg[k] = n4avg[k] + n[j][4]/NS;
            n5avg[k] = n5avg[k] + n[j][5]/NS;

        ''' CALCULATING MEAN ERROR FOR POPULATION (population-average constants vs all data)'''
        for i in range(ND):
            Ferravgn[i] = -1.*lydata[i][0] + math.log(n1avg[k]) + n2avg[k]*lydata[i][1]
            Ferravgn[i] = Ferravgn[i] + n3avg[k]*math.log( ydata[i][2] )

            aFerravgn[i] = abs(Ferravgn[i])/abs(lydata[i][0])
        #-------------
        aFerrmeanavgn[k] = numpy.mean(aFerravgn)
        if (kconv < 0 and aFerrmeanavgn[k] < target):
            kconv = k

        # time average of n values in population for generations k > 800
        aFerrta = aFerrta + aFerrmeanavgn[k]/NGEN
        if (k > 800):
            n1ta = n1ta + n1avg[k]/(NGEN-800)
            n2ta = n2ta + n2avg[k]/(NGEN-800)
            n3ta = n3ta + n3avg[k]/(NGEN-800)
            n4ta = n4ta + n4avg[k]/(NGEN-800)
            n5ta = n5ta + n5avg[k]/(NGEN-800)

        # save minimum population-average error and corresponding n(i) values
        if (aFerrmeanavgn[k] < aFerrmeanavgnMin):
            aFerrmeanavgnMin = aFerrmeanavgn[k]
            n1min = n1avg[k]
            n2min = n2avg[k]
            n3min = n3avg[k]
            n4min = n4avg[k]
            n5min = n5avg[k]
            kmin = k
        '''end of evolution loop'''

    if verbose:
        print('ENDING: pop. avg n1-n3,aFerrmean:', n1avg[k], n2avg[k], n3avg[k], aFerrmeanavgn[k])
        print('MINUMUM:  avg n1-n3,aFerrmeanMin:', n1min, n2min, n3min, aFerrmeanavgnMin, ' at generation',
     kmin)
        print('TIME AVG:    avg n1-n3,aFerrmean:', n1ta, n2ta, n3ta, aFerrta)

    return dict(nkeephist=nkeephist, gen=gen, n1avg=n1avg, n2avg=n2avg, n3avg=n3avg, aFerrmeanavgn=
     aFerrmeanavgn,
                n1min=n1min, n2min=n2min, n3min=n3min, aFerrmeanavgnMin=aFerrmeanavgnMin,
                kmin=kmin, kconv=kconv, n1end=n1avg[k], n2end=n2avg[k], n3end=n3avg[k],
                aFerrend=aFerrmeanavgn[k], n1ta=n1ta, n2ta=n2ta, n3ta=n3ta, aFerrta=aFerrta)
# end CodeP2.2F26 (function definition)

# ------------------------------ Cell 3 ------------------------------
# identical to Program A, Cell 2 (listed above)

# ------------------------------ Cell 4 ------------------------------
'''Task 2.2 - first run with the initial guesses of the skeleton code (n1i = 0.00027, n2i = 4.0,
   n3i = 0.063), the skeleton mutation amplitude (+/-8.5 %) and NGEN = 6000, followed by the
   skeleton plot of the evolution of the population-average constants and error'''
r0 = run_GA(0.00027, 4.0, 0.063, NGEN=6000, MUT=0.085, rseed=1)

plt.rcParams.update({'font.size': 18})
plt.figure(figsize=[10, 8])
plt.plot(r0['gen'], r0['aFerrmeanavgn'])
plt.plot(r0['gen'], r0['n1avg'])
plt.plot(r0['gen'], r0['n2avg'])
plt.plot(r0['gen'], r0['n3avg'])
plt.legend(['aFerrmeanavgn', 'n1 avg', 'n2 avg', 'n3 avg'], loc='lower left')
plt.xlabel('generation')
plt.ylabel('constants and error')
plt.loglog()
plt.yticks([0.0001, 0.001, 0.01, 0.1, 1.0, 10])
plt.xticks([1, 10, 100, 1000, 10000])
```

```
plt.show()
plt.rcParams.update({'font.size': 11})

# ------------------------------ Cell 5 ------------------------------
'''(added) independent check: deterministic minimisation (scipy Nelder-Mead, multi-start) of the SAME
   objective [Err]mean = (1/ND) sum |ln n1 + n2 ln(Tw-Tsat) + n3 ln g - ln q"| / |ln q"|  (Eq. 1.4).
   It is only used to judge how close the genetic-algorithm constants are to the true minimum.'''
from scipy.optimize import minimize
Y = numpy.array(ydata)
lq, ldT, lg = numpy.log(Y[:, 0]), numpy.log(Y[:, 1]), numpy.log(Y[:, 2])

def Fmean3(n1, n2, n3):
    return numpy.mean(numpy.abs(-lq + numpy.log(n1) + n2 * ldT + n3 * lg) / numpy.abs(lq))

best = None
for x0 in [(-6., 3., 0.05), (-8., 3.5, 0.1), (-4., 2.5, 0.0), (-3., 2., 0.2)]:
    rr = minimize(lambda x: Fmean3(math.exp(x[0]), x[1], x[2]), x0, method='Nelder-Mead',
                  options=dict(xatol=1e-10, fatol=1e-13, maxiter=20000))
    if best is None or rr.fun < best.fun:
        best = rr
ref3 = dict(n1=math.exp(best.x[0]), n2=best.x[1], n3=best.x[2], F=best.fun)
print('reference minimum: n1 = %.5e  n2 = %.4f  n3 = %.4f   [Err]mean = %.5f' % (ref3['n1'], ref3['n2'],
     ref3['n3'], ref3['F']))

# ------------------------------ Cell 6 ------------------------------
'''Task 2.2 - effect of the initial guesses, of the maximum mutation amplitude MUT and of NGEN.
   4 sets of initial guesses x 3 mutation amplitudes x 5 random seeds (NGEN = 6000), plus
   longer runs (NGEN = 20000) and a two-stage strategy (MUT = 0.40 then MUT = 0.03 restarted
   from the stage-1 best constants).'''
import time
SETS22 = {'Set 1': (0.00027, 4.0, 0.063),     # skeleton code values
          'Set 2': (0.001, 3.0, 0.05),        # physics: q" ~ (Tw-Tsat)^3, q" ~ g^n with 0.01 < n < 0.1
          'Set 3': (0.01, 2.0, 0.25),         # far: weaker superheat exponent, n3 = 0.25 (pure-fluid, g/gen >
     0.1)
          'Set 4': (1.0e-6, 5.0, 0.01)}       # very far: n1 1500 times too small, n2 = 5
MUTS22 = [0.085, 0.20, 0.40]
SEEDS = [1, 2, 3, 4, 5]
FNEAR = 1.01 * ref3['F']          # 'near-optimal' threshold (1 % above the true minimum)

def summary(r, **kw):
    d = dict(F=r['aFerrmeanavgnMin'], n1=r['n1min'], n2=r['n2min'], n3=r['n3min'], kmin=r['kmin'],
             k035=first_below(r['aFerrmeanavgn'], 0.035), knear=first_below(r['aFerrmeanavgn'], FNEAR),
             Fend=r['aFerrend'], nkeep_mean=float(numpy.mean(r['nkeephist'])),
             nkeep0=float(numpy.mean(numpy.array(r['nkeephist']) == 0)))
    d.update(kw)
    return d

t0 = time.time()
study22 = []
for name, init in SETS22.items():
    for MUT in MUTS22:
        for s in SEEDS:
            r = run_GA(*init, NGEN=6000, MUT=MUT, rseed=s, verbose=False)
            study22.append(summary(r, set=name, init=init, MUT=MUT, NGEN=6000, seed=s, mode='single'))
    # longer runs with the skeleton mutation amplitude
    for s in SEEDS[:2]:
        r = run_GA(*init, NGEN=20000, MUT=0.085, rseed=s, verbose=False)
        study22.append(summary(r, set=name, init=init, MUT=0.085, NGEN=20000, seed=s, mode='single'))
    # two-stage strategy: exploration with large mutations, then refinement with small mutations
    for s in SEEDS:
        r1 = run_GA(*init, NGEN=6000, MUT=0.40, rseed=s, verbose=False)
        r2 = run_GA(r1['n1min'], r1['n2min'], r1['n3min'], NGEN=6000, MUT=0.03, rseed=100 + s, verbose=False)
        d = summary(r2, set=name, init=init, MUT='0.40/0.03', NGEN='6000+6000', seed=s, mode='two-stage')
        if r1['aFerrmeanavgnMin'] < d['F']:           # keep stage 1 result if stage 2 did not improve it
            d.update(F=r1['aFerrmeanavgnMin'], n1=r1['n1min'], n2=r1['n2min'], n3=r1['n3min'])
        d['k035'] = first_below(r1['aFerrmeanavgn'], 0.035)
        study22.append(d)
print('study: %d runs in %.0f s' % (len(study22), time.time() - t0))

# summary table: one line per (set, MUT, NGEN)
print('\n%-6s %-10s %-9s | %-8s %-8s %-8s | %-5s | %-6s | %-9s %-7s %-7s' % ('set', 'MUT', 'NGEN', 'Fbest', '
     Fmedian', 'Fworst', '<.035', 'k035', 'n1', 'n2', 'n3'))
table22 = []
for name in SETS22:
    for key in [(m, 6000) for m in MUTS22] + [(0.085, 20000), ('0.40/0.03', '6000+6000')]:
```

```
        runs = [d for d in study22 if d['set'] == name and d['MUT'] == key[0] and d['NGEN'] == key[1]]
        Fs = numpy.array([d['F'] for d in runs])
        b = runs[int(numpy.argmin(Fs))]
        ks = [d['k035'] for d in runs if d['k035'] >= 0]
        row = dict(set=name, init=SETS22[name], MUT=key[0], NGEN=key[1], nruns=len(runs), Fbest=float(Fs.min()),
                   Fmed=float(numpy.median(Fs)), Fworst=float(Fs.max()), nsucc=int((Fs < 0.035).sum()),
                   k035med=float(numpy.median(ks)) if ks else -1, n1=b['n1'], n2=b['n2'], n3=b['n3'],
                   n2min=float(min(d['n2'] for d in runs)), n2max=float(max(d['n2'] for d in runs)),
                   nkeep0=float(numpy.mean([d['nkeep0'] for d in runs])), seed=b['seed'])
        table22.append(row)
        print('%-6s %-10s %-9s | %.5f  %.5f  %.5f | %d/%d   | %6s | %.3e %.4f  %.4f' % (name, key[0], key[1], row['Fbest'], row['Fmed'], row['Fworst'],
              row['nsucc'], row['nruns'], '%d' % row['k035med'] if ks else 'never', row['n1'], row['n2'], row['n3']))

# ------------------------------ Cell 7 ------------------------------
'''Task 2.2 - best fit: the run of the study with the lowest minimum error is replayed (same seed,
   so it is reproduced exactly) to obtain its histories, then the constants n1min, n2min, n3min are
   used for the plots and the deviation statistics.'''
bestrun = min(study22, key=lambda d: d['F'])
print('best run of the study:', bestrun['set'], bestrun['init'], 'MUT =', bestrun['MUT'], 'NGEN =', bestrun['NGEN'], 'seed =', bestrun['seed'])
if bestrun['mode'] == 'single':
    rb = run_GA(*bestrun['init'], NGEN=bestrun['NGEN'], MUT=bestrun['MUT'], rseed=bestrun['seed'])
    hist = rb
else:
    r1 = run_GA(*bestrun['init'], NGEN=6000, MUT=0.40, rseed=bestrun['seed'], verbose=False)
    r2 = run_GA(r1['n1min'], r1['n2min'], r1['n3min'], NGEN=6000, MUT=0.03, rseed=100 + bestrun['seed'])
    rb = r1 if r1['aFerrmeanavgnMin'] < r2['aFerrmeanavgnMin'] else r2
n1min, n2min, n3min, Fmin = rb['n1min'], rb['n2min'], rb['n3min'], rb['aFerrmeanavgnMin']
print('\nBEST FIT (Eq. 1.1):  n1 = %.5e   n2 = %.4f   n3 = %.4f   [Ferr]mean,min = %.5f' % (n1min, n2min, n3min, Fmin))
print('reference minimum :  n1 = %.5e   n2 = %.4f   n3 = %.4f   [Ferr]mean     = %.5f' % (ref3['n1'], ref3['n2'], ref3['n3'], ref3['F']))

# evolution of the population-average constants and error (skeleton plot, report style)
def evolution_plot(ax, h, title):
    g = numpy.arange(1, len(h['aFerrmeanavgn']) + 1)
    ax.plot(g, h['aFerrmeanavgn'], color=C[0], lw=0.8, label=r'$[F_{err}]_{mean}$ (aFerrmeanavgn)')
    ax.plot(g, h['n1avg'], color=C[1], lw=1.2, label=r'$n_1$ avg')
    ax.plot(g, h['n2avg'], color=C[2], lw=1.2, label=r'$n_2$ avg')
    ax.plot(g, h['n3avg'], color=C[6], lw=1.2, label=r'$n_3$ avg')
    ax.axhline(0.035, color=INK2, ls='--', lw=0.9)
    ax.set_xscale('log'); ax.set_yscale('log')
    ax.set_xlim(1, len(g)); ax.set_ylim(1e-4, 10)
    ax.set_xlabel('generation'); ax.set_title(title)
if bestrun['mode'] == 'single':
    fig, ax = plt.subplots(figsize=(6.4, 4.6)); axs = [ax]
    evolution_plot(ax, hist, '%s, MUT = %s, seed %d' % (bestrun['set'], bestrun['MUT'], bestrun['seed']))
else:
    fig, axs = plt.subplots(1, 2, figsize=(11, 4.4), sharey=True)
    evolution_plot(axs[0], r1, '(a) stage 1: %s, MUT = 0.40, NGEN = 6000' % bestrun['set'])
    evolution_plot(axs[1], r2, '(b) stage 2: restart, MUT = 0.03, NGEN = 6000')
axs[0].set_ylabel('population-average constants and error')
axs[0].text(1.2, 0.038, 'target 0.035', color=INK2, fontsize=9)
axs[-1].legend(loc='upper center', bbox_to_anchor=(-0.05 if len(axs) > 1 else 0.5, -0.16), ncol=4)
savefig('t22_evolution')

# predicted vs measured heat flux + deviation statistics
qpppred = [n1min * (ydata[i][1]**n2min) * ((ydata[i][2])**n3min) for i in range(ND)]
qppdata = [ydata[i][0] for i in range(ND)]
qp, qd = numpy.array(qpppred), numpy.array(qppdata)
dev = (qd - qp) / qp                                  # fractional deviation of data from prediction
stats22 = dict(rms_abs=float(numpy.sqrt(numpy.mean((qd - qp)**2))), rms_rel=float(numpy.sqrt(numpy.mean(dev**2))),
               mean_abs_rel=float(numpy.mean(numpy.abs(dev))), max_abs_rel=float(numpy.max(numpy.abs(dev))),
               within10=float(numpy.mean(numpy.abs(dev) <= 0.10)), mean_q=float(qd.mean()))
print('\nrms deviation of q"exp from q"pred       = %.2f W/cm2 (mean q" = %.1f W/cm2)' % (stats22['rms_abs'], stats22['mean_q']))
print('rms fractional deviation (q"exp-q"pred)/q"pred = %.4f' % stats22['rms_rel'])
print('mean absolute fractional deviation      = %.4f' % stats22['mean_abs_rel'])
print('max absolute fractional deviation       = %.4f' % stats22['max_abs_rel'])
print('fraction of points within +/-10 %%       = %.2f' % stats22['within10'])

```

```
groups22 = [(0.098, r'$g = 0.098$ m/s$^2$'), (9.8, r'$g = 9.8$ m/s$^2$'), (19.6, r'$g = 19.6$ m/s$^2$')]
bias22 = {}
fig, (a1, a2) = plt.subplots(1, 2, figsize=(11, 4.6))
xx = numpy.array([10, 1000])
a1.plot(xx, xx, '-', color=INK2, lw=1, label='perfect fit (slope 1)')
a1.plot(xx, 1.1 * xx, '--', color=INK2, lw=0.8, label=r'$\pm 10\,\%$')
a1.plot(xx, 0.9 * xx, '--', color=INK2, lw=0.8)
for c, (gv, lab) in enumerate(groups22):
    sel = numpy.array([ydata[i][2] == gv for i in range(ND)])
    a1.loglog(qp[sel], qd[sel], MK[c], color=C[c], ms=7, mec='white', mew=0.8, label=lab)
    a2.plot(Y[sel, 1], 100 * dev[sel], MK[c], color=C[c], ms=7, mec='white', mew=0.8, label=lab)
    bias22[str(gv)] = dict(mean_dev=float(dev[sel].mean()), rms_dev=float(numpy.sqrt(numpy.mean(dev[sel]**2))
    ), n=int(sel.sum()))
    print('group g = %5.3f : mean deviation %+.3f   rms deviation %.3f   (%d points)' % (gv, dev[sel].mean(),
      numpy.sqrt(numpy.mean(dev[sel]**2)), sel.sum()))
a1.set_xlim(30, 300); a1.set_ylim(30, 300)
plain_log_ticks(a1, [30, 50, 100, 200, 300], [30, 50, 100, 200, 300])
a1.set_xlabel(r'predicted heat flux $q^{\prime\prime}_{pred}$ (W/cm$^2$), Eq. (1.1)')
a1.set_ylabel(r'measured heat flux $q^{\prime\prime}_{exp}$ (W/cm$^2$)')
a1.set_title(r'(a) $n_1=%s$, $n_2=%.3f$, $n_3=%.4f$' % (sci(n1min, 3), n2min, n3min))
a1.legend(loc='upper left')
a2.axhspan(-10, 10, color='#f0efec', zorder=0)
a2.axhline(0, color=INK2, lw=1)
a2.set_xlabel(r'wall superheat $T_w - T_{sat}$ (°C)')
a2.set_ylabel(r'$(q^{\prime\prime}_{exp}-q^{\prime\prime}_{pred})/q^{\prime\prime}_{pred}$ (%)')
a2.set_title(r'(b) deviation from Eq. (1.1); shaded band $=\pm 10\,\%$')
a2.legend(loc='upper right')
savefig('t22_parity')

json.dump(dict(best=dict(n1=n1min, n2=n2min, n3=n3min, F=Fmin, set=bestrun['set'], init=bestrun['init'], MUT=
    bestrun['MUT'],
                         NGEN=bestrun['NGEN'], seed=bestrun['seed'], mode=bestrun['mode'], kmin=rb['kmin'],
                         n1end=rb['n1end'], n2end=rb['n2end'], n3end=rb['n3end'], Fend=rb['aFerrend'],
                         n1ta=rb['n1ta'], n2ta=rb['n2ta'], n3ta=rb['n3ta'], Fta=rb['aFerrta']),
               base=dict(n1=r0['n1min'], n2=r0['n2min'], n3=r0['n3min'], F=r0['aFerrmeanavgnMin'], kmin=r0['
    kmin'],
                         n1end=r0['n1end'], n2end=r0['n2end'], n3end=r0['n3end'], Fend=r0['aFerrend'],
                         n1ta=r0['n1ta'], n2ta=r0['n2ta'], n3ta=r0['n3ta'], Fta=r0['aFerrta'], k035=
    first_below(r0['aFerrmeanavgn'], 0.035)),
               ref=ref3, stats=stats22, bias=bias22, table=table22, runs=study22),
          open(RESDIR + 'task22.json', 'w'), indent=1)

# ------------------------------ Cell 8 ------------------------------
'''Task 2.2 - convergence histories of the 4 sets of initial guesses (seed 1) for the skeleton
   mutation amplitude (MUT = 0.085) and for MUT = 0.40, and trajectories of the population-average
   constants on the error landscape (min over n3 of [Ferr]mean in the (n2, n1) plane).'''
traj = {}
fig, axs = plt.subplots(1, 2, figsize=(11, 4.4), sharey=True)
for a, MUT in zip(axs, [0.085, 0.40]):
    for c, (name, init) in enumerate(SETS22.items()):
        r = run_GA(*init, NGEN=6000, MUT=MUT, rseed=1, verbose=False)
        traj[(name, MUT)] = (numpy.array(r['n1avg']), numpy.array(r['n2avg']))
        g = numpy.arange(1, 6001)
        a.plot(g, r['aFerrmeanavgn'], color=C[c], lw=0.5, alpha=0.25)
        a.plot(g, best_so_far(r['aFerrmeanavgn']), color=C[c], lw=1.8,
               label='%s $(%s,\\ %.3g,\\ %.3g)$' % (name, sci(init[0], 1), init[1], init[2]))
    a.axhline(0.035, color=INK2, ls='--', lw=0.9)
    a.axhline(ref3['F'], color=INK, ls=':', lw=0.9)
    a.set_xscale('log'); a.set_yscale('log'); a.set_xlim(1, 6000); a.set_ylim(0.025, 1)
    plain_log_ticks(a, None, [0.03, 0.05, 0.1, 0.2, 0.5, 1])
    a.set_xlabel('generation')
    a.set_title('maximum mutation amplitude MUT = $\\pm$%.1f %%' % (100 * MUT))
axs[0].set_ylabel(r'$[F_{err}]_{mean}$: best so far (thick), current (thin)')
axs[0].text(1.3, 0.0362, 'target 0.035', color=INK2, fontsize=9)
axs[0].text(1.3, 0.0262, 'true minimum %.4f' % ref3['F'], color=INK, fontsize=9)
axs[1].legend(loc='upper right', title=r'initial guesses $(n_1,n_2,n_3)$', title_fontsize=9)
savefig('t22_convergence')

# error landscape: for each (n1, n2) take the minimum over n3 of [Ferr]mean
N1g = numpy.logspace(-7, 0.3, 150)
N2g = numpy.linspace(0.8, 5.4, 150)
N3g = numpy.linspace(-0.1, 1.0, 111)
L1, L2, L3 = numpy.meshgrid(numpy.log(N1g), N2g, N3g, indexing='ij')
E = numpy.zeros(L1.shape)
for i in range(ND):
```

```
    E += numpy.abs(-lq[i] + L1 + L2 * ldT[i] + L3 * lg[i]) / abs(lq[i])
Fgrid = (E / ND).min(axis=2)
lev = [0.0, 0.03, 0.035, 0.05, 0.1, 0.2, 0.4, 0.8, 10]
GRAYS = LinearSegmentedColormap.from_list('grays', ['#8a8984', '#c3c2b7', '#f6f5f2'])   # dark = low error
norm = mpl.colors.BoundaryNorm(lev, GRAYS.N)
fig, axs = plt.subplots(1, 2, figsize=(11.5, 4.6), sharey=True)
for a, MUT in zip(axs, [0.085, 0.40]):
    cs = a.contourf(N2g, N1g, Fgrid, levels=lev, cmap=GRAYS, norm=norm)
    a.contour(N2g, N1g, Fgrid, levels=lev[1:-1], colors='white', linewidths=0.6)
    for c, name in enumerate(SETS22):
        n1h, n2h = traj[(name, MUT)]
        a.plot(n2h[::3], n1h[::3], '-', color=C[c], lw=0.7, alpha=0.9)
        a.plot(SETS22[name][1], SETS22[name][0], MK[c], color=C[c], ms=9, mec='white', mew=1, label=name + '
 start')
    a.plot(ref3['n2'], ref3['n1'], '*', color=C[7], ms=15, mec='white', label='true minimum')
    a.set_yscale('log'); a.set_xlim(0.8, 5.4); a.set_ylim(1e-7, 2)
    a.set_xlabel(r'$n_2$ (superheat exponent)')
    a.set_title('GA trajectories, MUT = $\\pm$%.1f %%, seed 1' % (100 * MUT))
    a.grid(False)
axs[0].set_ylabel(r'$n_1$')
axs[0].legend(loc='upper right', fontsize=8.5)
cb = fig.colorbar(cs, ax=axs, ticks=lev[1:-1], pad=0.02)
cb.set_label(r'$\min_{n_3}\,[F_{err}]_{mean}$')
savefig('t22_landscape')
```

A.3 Program C – Task 2.3 (P2_Task2.3.ipynb)

```
# ================================================================================
# Program C : P2_Task2.3.ipynb
# ================================================================================

# ------------------------------ Cell 1 ------------------------------
'''>>>>> start CodeP2.1F26
    V.P. Carey ME249, Fall 2026
    MODIFIED for Task 2.3 (five-constant model):
      - two data values corrected to match the Appendix listing of the project
        statement: row 0  q" 44.2 -> 44.1 W/cm2,  row 6  superheat 37.8 -> 37.5 C
      - modification (i): the red-text data lines (P = 9.5 and 7.0 kPa, gamma = 0
        and 1.71) are uncommented -> 77 data points
      - modification (ii): ND = NS = 77
      - added: the 15 validation data points of the Appendix are stored in the
        array vdata (used only to check the fitted relation, not for training)'''

# version 3 print function
from __future__ import print_function
# seed the pseudorandom number generator
from random import random
from random import seed
# seed random number generator
seed(1)

#import math and numpy packages
import math
import numpy

%matplotlib inline
# importing the required module
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [10, 8] # for square canvas


#import copy
from copy import copy, deepcopy


#create arrays
ydata = []
lydata = []

#Parameters for Evolution Loop
#set data parameters
ND = 77        #number of data vectors in array
```

```
DI = 5        #number of data items in vector
NS = 77       #total number of DNA strands

# j is column, i is row downward for ydata[i][j] - both start at zero
# so it is:  ydata[row][column]
# this is an array that is essentially a list of lists

#assembling data array
#store array where rows are data vectors
#[heat flux, superheat, gravity, surface tension parameter, pressure]

ydata =     [[44.1, 32.5, 0.098, 1.79, 5.5]]    # corrected (was 44.2)
ydata.append([47.4, 33.2, 0.098, 1.79, 5.5])
ydata.append([49.4, 34.2, 0.098, 1.79, 5.5])

ydata.append([59.2, 34.8, 0.098, 1.79, 5.5])
ydata.append([67.8, 36.3, 0.098, 1.79, 5.5])
ydata.append([73.6, 37.3, 0.098, 1.79, 5.5])
ydata.append([76.3, 37.5, 0.098, 1.79, 5.5])     # corrected (was 37.8)
ydata.append([85.3, 39.2, 0.098, 1.79, 5.5])
ydata.append([96.5, 39.3, 0.098, 1.79, 5.5])
ydata.append([111., 42.3, 0.098, 1.79, 5.5])
ydata.append([124., 43.5, 0.098, 1.79, 5.5])
ydata.append([136.2, 45.4, 0.098, 1.79, 5.5])

ydata.append([143.5, 46.7, 0.098, 1.79, 5.5])
ydata.append([154.6, 47.9, 0.098, 1.79, 5.5])
ydata.append([163.1, 48.6, 0.098, 1.79, 5.5])
ydata.append([172.8, 50.9, 0.098, 1.79, 5.5])
ydata.append([184.2, 51.7, 0.098, 1.79, 5.5])
ydata.append([203.7, 56.4, 0.098, 1.79, 5.5])

ydata.append([36.7, 30.2, 9.8, 1.79, 5.5])
ydata.append([55.1, 34.1, 9.8, 1.79, 5.5])
ydata.append([67.5, 35.3, 9.8, 1.79, 5.5])
ydata.append([78.0, 37.8, 9.8, 1.79, 5.5])
ydata.append([92.0, 38.1, 9.8, 1.79, 5.5])
ydata.append([120., 44.1, 9.8, 1.79, 5.5])
ydata.append([134.3, 46.9, 9.8, 1.79, 5.5])
ydata.append([150.3, 48.5, 9.8, 1.79, 5.5])
ydata.append([167., 49.2, 9.8, 1.79, 5.5])
ydata.append([184., 52.7, 9.8, 1.79, 5.5])
ydata.append([196.5, 53.1, 9.8, 1.79, 5.5])

ydata.append([42.4, 28.0, 19.6, 1.79, 9.5])
ydata.append([48.7, 29.3, 19.6, 1.79, 9.5])
ydata.append([54.5, 29.6, 19.6, 1.79, 9.5])

ydata.append([62.1, 28.5, 19.6, 1.79, 9.5])
ydata.append([70.8, 30.5, 19.6, 1.79, 9.5])
ydata.append([73.7, 30.3, 19.6, 1.79, 9.5])
ydata.append([81.8, 30.6, 19.6, 1.79, 9.5])
ydata.append([91.9, 34.5, 19.6, 1.79, 9.5])
ydata.append([103.9, 34.5, 19.6, 1.79, 9.5])
ydata.append([119.1, 35.4, 19.6, 1.79, 9.5])
ydata.append([133.7, 36.8, 19.6, 1.79, 9.5])
ydata.append([139.9, 38.1, 19.6, 1.79, 9.5])
ydata.append([148.3, 39.1, 19.6, 1.79, 9.5])
ydata.append([157.0, 40.0, 19.6, 1.79, 9.5])
ydata.append([169.1, 42.2, 19.6, 1.79, 9.5])
ydata.append([179.2, 43.2, 19.6, 1.79, 9.5])
ydata.append([205.0, 46.0, 19.6, 1.79, 9.5])

ydata.append([42.4, 29.7, 19.6, 1.79, 5.5])
ydata.append([48.7, 31.0, 19.6, 1.79, 5.5])
ydata.append([54.5, 31.2, 19.6, 1.79, 5.5])
ydata.append([70.8, 32.4, 19.6, 1.79, 5.5])
ydata.append([73.7, 31.4, 19.6, 1.79, 5.5])
ydata.append([81.8, 32.5, 19.6, 1.79, 5.5])
ydata.append([91.9, 36.3, 19.6, 1.79, 5.5])
ydata.append([103.9, 36.3, 19.6, 1.79, 5.5])
ydata.append([119.1, 37.2, 19.6, 1.79, 5.5])
ydata.append([133.7, 38.4, 19.6, 1.79, 5.5])
ydata.append([139.9, 39.7, 19.6, 1.79, 5.5])
ydata.append([148.3, 40.9, 19.6, 1.79, 5.5])
```

```
ydata.append([157.0, 41.6, 19.6, 1.79, 5.5])
ydata.append([169.1, 43.9, 19.6, 1.79, 5.5])
ydata.append([179.2, 45.0, 19.6, 1.79, 5.5])
ydata.append([205.0, 47.9, 19.6, 1.79, 5.5])

ydata.append([77.0, 41.5, 9.8, 0.00, 7.0])
ydata.append([71.0, 40.5, 9.8, 0.00, 7.0])
ydata.append([66.0, 39.5, 9.8, 0.00, 7.0])
ydata.append([62.0, 38.5, 9.8, 0.00, 7.0])
ydata.append([42.0, 34.0, 9.8, 0.00, 7.0])
ydata.append([60.0, 37.5, 9.8, 0.00, 7.0])
ydata.append([53.0, 37.0, 9.8, 0.00, 7.0])

ydata.append([71.7, 36.4, 0.098, 1.71, 5.5])
ydata.append([81.5, 38.5, 0.098, 1.71, 5.5])
ydata.append([90.7, 39.5, 0.098, 1.71, 5.5])
ydata.append([103.3, 41.6, 0.098, 1.71, 5.5])
ydata.append([117.0, 43.1, 0.098, 1.71, 5.5])
ydata.append([138.6, 45.4, 0.098, 1.71, 5.5])
ydata.append([161.7, 47.9, 0.098, 1.71, 5.5])
ydata.append([207.5, 50.9, 0.098, 1.71, 5.5])

# validation data (Appendix, blue text) - NOT used for training
vdata =     [[50.9, 37.2, 0.098, 1.79, 5.5]]
vdata.append([75.1, 37.7, 0.098, 1.79, 5.5])
vdata.append([99.2, 39.7, 0.098, 1.79, 5.5])
vdata.append([147.7, 47.2, 0.098, 1.79, 5.5])
vdata.append([172.1, 49.7, 9.8, 1.79, 5.5])
vdata.append([49.2, 29.6, 19.6, 1.79, 9.5])
vdata.append([71.6, 30.0, 19.6, 1.79, 9.5])
vdata.append([129.8, 36.4, 19.6, 1.79, 9.5])
vdata.append([173.8, 42.8, 19.6, 1.79, 9.5])
vdata.append([68.7, 32.1, 19.6, 1.79, 5.5])
vdata.append([128.5, 38.0, 19.6, 1.79, 5.5])
vdata.append([69.0, 40.1, 9.8, 0.0, 7.0])
vdata.append([58.2, 37.1, 9.8, 0.0, 7.0])
vdata.append([100.3, 41.2, 0.098, 1.71, 5.5])
vdata.append([156.9, 47.4, 0.098, 1.71, 5.5])
NV = len(vdata)
# print the data array
print ('ydata =', ydata)
print ('number of validation points NV =', NV)
# Task 2.1(a): check of the array size and of a single array element
print ('number of rows =', len(ydata), ' ;  ydata[1][0] =', ydata[1][0], ' ;  ydata[6][1] =', ydata[6][1])


''' need deepcopy to create an array of the same size as ydata,
#   since this array is a list(rows) of lists (column entries)  '''
lydata = deepcopy(ydata)  # create array to store ln of data values


# j is column, i is row downward for ydata[i][j] - both start at zero
# so it is:  ydata[row][column]
#now store log values for data
for j in range(DI):
    for i in range(ND):
        lydata[i][j]=math.log(ydata[i][j]+0.000000000010)

#OK now have stored array of log values for data

#end CodeP2.1F26

# ------------------------------ Cell 2 ------------------------------
'''>>>>> start CodeP2.2F26
    V.P. Carey ME249, Fall 2026
    MODIFIED for Task 2.3 (five-constant model, Eq. 1.5) - copy of the Task 2.2 program
    with modifications (iii)-(vii) of the project statement:
      (iii) error function Eq. (1.7) in the fitness and population-average loops
      (iv)  constants n4 and n5 included in mating/mutation
      (v)   final print-out of all five constants
      (vi)  plots of n4avg and n5avg added (see plotting cell)
      (vii) default initial guesses 0.000476, 3.028, 0.2249, 1.054, 0.217
    and, as in Task 2.2:
      - the evolution loop of the skeleton code is placed, unchanged, inside a
        function run_GA(...) so that it can be re-run for several initial guesses,
```

```
        mutation amplitudes, numbers of generations and random seeds
      - the maximum mutation amplitude (0.085 in the skeleton) is now the
        argument MUT (relative amplitude, i.e. +/- MUT*100 %)
      - both random generators (python random and numpy.random) are seeded with
        the argument rseed so that every run is reproducible
      - the history of the population-average constants and of aFerrmeanavgn is
        returned, together with the generation at which aFerrmeanavgn first drops
        below the target value (convergence-speed indicator) and the number of
        survivors nkeep of each generation (selection diagnostic)
      - BUG FIX: the skeleton line "import random" re-binds the name random to the
        module and hides the function random() imported in CodeP2.1F26, so the
        initialisation n1i+0.001*random() raises "TypeError: 'module' object is
        not callable".  The line is replaced by an explicit import of the function.'''
from random import random, seed   # (was: import random)

gen_en = 9.8   # earth-normal gravitational acceleration, m/s2

def run_GA(n1i=0.000476, n2i=3.028, n3i=0.2249, n4i=1.054, n5i=0.217, NGEN=6000, MUT=0.09,
           MFRAC=0.5, rseed=1, target=0.03, verbose=True):

    '''INITIALIZING PARAMETERS'''
    seed(rseed)                 # python random  (initial population)
    numpy.random.seed(rseed)    # numpy random   (mating / mutation)
    n = []
    ntemp = []
    gen=[0]
    n1avg = [0.0]
    n2avg = [0.0]
    n3avg = [0.0]
    n4avg = [0.0]
    n5avg = [0.0]
    meanAFerr=[0.0]
    aFerrmeanavgn=[0.0]

    # here the number of data vectors equals the number of DNA strands (or organisms)
    for k in range(NGEN-1):
        gen.append(k+1)   # generation array stores the
        meanAFerr.append(0.0)
        aFerrmeanavgn.append(0.0)
        n1avg.append(0.0)
        n2avg.append(0.0)
        n3avg.append(0.0)
        n4avg.append(0.0)
        n5avg.append(0.0)

    '''guesses for initial solution population: n0i = -1 (fixed), n1i ... n5i are arguments'''
    n0i =  -1.0

    #i initialize array where rows are dna vectors [n0i,n1i,...n5i] with random pertubations
    n =  [[-1., n1i+0.001*random(), n2i+0.1*random(), n3i+0.0001*random(), n4i+0.0001*random(), n5i+0.0001*
     random()]]
    for i in range(ND):
        n.append([-1., n1i+0.0001*random(), n2i+0.001*random(), n3i+0.0001*random(), n4i+0.0001*random(), n5i
     +0.0001*random()])

    # store also in wtemp
    ntemp = deepcopy(n)

    #define arrays of Ferr (error) functions
    Ferr =  [[0.0]]
    Ferravgn =  [[0.0]]
    aFerr =  [[0.0]]
    aFerravgn =  [[0.0]]
    for i in range(ND-1):
        Ferr.append([0.0])
        aFerr.append([0.0])
        Ferravgn.append([0.0])
        aFerravgn.append([0.0])

    aFerrmeanavgnMin=1000000000.0
    # these store the  n values for minimum population average error durng NGEN generations
    n1min = 0.0
    n2min = 0.0
    n3min = 0.0
    n4min = 0.0
```

```
    n5min = 0.0
    kmin = 0
    aFerrta = 0.0
    # these store the time averaged n values durng from generation 800 to NGEN generations
    n1ta = 0.0
    n2ta = 0.0
    n3ta = 0.0
    n4ta = 0.0
    n5ta = 0.0
    kconv = -1      # first generation with aFerrmeanavgn < target (-1 = never)
    nkeephist = []  # (added) number of survivors nkeep at each generation

    '''START OF EVOLUTION LOOP'''
    for k in range(NGEN):

        '''CALCULATING ERROR (FITNESS) - Eq. (1.3): organism i is paired with data point i'''
        for i in range(ND):

            # Eq. (1.7): -ln q" + ln n1 + n2 ln(Tw-Tsat) + n3 ln(g + n4 gen gamma) + n5 ln P
            Ferr[i] = n[i][0]*lydata[i][0] + math.log(n[i][1]) + n[i][2]*lydata[i][1]
            Ferr[i] = Ferr[i] + n[i][3]*math.log( ydata[i][2] + n[i][4]*gen_en*ydata[i][3] ) + n[i][5]*lydata
     [i][4]

            aFerr[i] = abs(Ferr[i])/abs(lydata[i][0])  #- absolute fractional error
        #-------------
        aFerrmean = numpy.mean(aFerr) #mean error for population for this generation
        meanAFerr[k]=aFerrmean  #store aFerrmean for this generation gen[k]=k
        aFerrmedian = numpy.median(aFerr) #median error for population for this generation

        '''SELECTION'''
        clim = MFRAC*aFerrmedian  #cut off limit is a fraction/multiplier MFRAC of median value for population
        nkeep = 0
        for j in range(NS):  # NS Ferr values, one for each solution in population
            if (aFerr[j] < clim):
                nkeep = nkeep + 1
                #ntemp[nkeep][0] = n[j][0] = -1 so it is unchanged;
                ntemp[nkeep-1][1] = n[j][1];
                ntemp[nkeep-1][2] = n[j][2];
                ntemp[nkeep-1][3] = n[j][3];
                ntemp[nkeep-1][4] = n[j][4];
                ntemp[nkeep-1][5] = n[j][5];
        #compute number to be added by mating
        nnew = NS - nkeep
        nkeephist.append(nkeep)   # (added) number of survivors, diagnostic of selection pressure

        '''MATING'''
        for j in range(nnew):
            # pick two survivors randomly
            nmate1 = numpy.random.randint(low=0, high=nkeep+1)
            nmate2 = numpy.random.randint(low=0, high=nkeep+1)

            '''here, do not change property ntemp[nkeep+j+1][0], it's always fixed at the value -1'''
            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][1] = n[nmate1][1]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 1,
     mutation added
            else:
                ntemp[nkeep+j+1][1] = n[nmate2][1]*(1.+MUT*2.*(0.5-numpy.random.rand()))

            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][2] = n[nmate1][2]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 2,
     mutation added
            else:
                ntemp[nkeep+j+1][2] = n[nmate2][2]*(1.+MUT*2.*(0.5-numpy.random.rand()))

            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][3] = n[nmate1][3]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 3,
     mutation added
            else:
                ntemp[nkeep+j+1][3] = n[nmate2][3]*(1.+MUT*2.*(0.5-numpy.random.rand()))

            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][4] = n[nmate1][4]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 4,
     mutation added
            else:
                ntemp[nkeep+j+1][4] = n[nmate2][4]*(1.+MUT*2.*(0.5-numpy.random.rand()))
```

```
339
340             if (numpy.random.rand() < 0.5):
341                 ntemp[nkeep+j+1][5] = n[nmate1][5]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 5,
    mutation added
342             else:
343                 ntemp[nkeep+j+1][5] = n[nmate2][5]*(1.+MUT*2.*(0.5-numpy.random.rand()))
344         #============================================
345         n = deepcopy(ntemp)   # save ntemp as n for use in next generation (next iteration)
346
347         '''AVERAGING OVER POPULATION AND OVER TIME, FINDING MINIMUM ERROR SET OF n VALUES'''
348         n1avg[k] = 0.0;
349         n2avg[k] = 0.0;
350         n3avg[k] = 0.0;
351         n4avg[k] = 0.0;
352         n5avg[k] = 0.0;
353         for j in range(NS):
354             n1avg[k] = n1avg[k] + n[j][1]/NS;
355             n2avg[k] = n2avg[k] + n[j][2]/NS;
356             n3avg[k] = n3avg[k] + n[j][3]/NS;
357             n4avg[k] = n4avg[k] + n[j][4]/NS;
358             n5avg[k] = n5avg[k] + n[j][5]/NS;
359
360         ''' CALCULATING MEAN ERROR FOR POPULATION (population-average constants vs all data)'''
361         for i in range(ND):
362             Ferravgn[i] = -1.*lydata[i][0] + math.log(n1avg[k]) + n2avg[k]*lydata[i][1]
363             Ferravgn[i] = Ferravgn[i] + n3avg[k]*math.log( ydata[i][2] + n4avg[k]*gen_en*ydata[i][3] ) +
    n5avg[k]*lydata[i][4]
364
365             aFerravgn[i] = abs(Ferravgn[i])/abs(lydata[i][0])
366         #-------------
367         aFerrmeanavgn[k] = numpy.mean(aFerravgn)
368         if (kconv < 0 and aFerrmeanavgn[k] < target):
369             kconv = k
370
371         # time average of n values in population for generations k > 800
372         aFerrta = aFerrta + aFerrmeanavgn[k]/NGEN
373         if (k > 800):
374             n1ta = n1ta + n1avg[k]/(NGEN-800)
375             n2ta = n2ta + n2avg[k]/(NGEN-800)
376             n3ta = n3ta + n3avg[k]/(NGEN-800)
377             n4ta = n4ta + n4avg[k]/(NGEN-800)
378             n5ta = n5ta + n5avg[k]/(NGEN-800)
379
380         # save minimum population-average error and corresponding n(i) values
381         if (aFerrmeanavgn[k] < aFerrmeanavgnMin):
382             aFerrmeanavgnMin = aFerrmeanavgn[k]
383             n1min = n1avg[k]
384             n2min = n2avg[k]
385             n3min = n3avg[k]
386             n4min = n4avg[k]
387             n5min = n5avg[k]
388             kmin = k
389         '''end of evolution loop'''
390
391     if verbose:
392         print('ENDING: pop.avg n1-n5,aFerrmean:', n1avg[k], n2avg[k], n3avg[k], n4avg[k], n5avg[k],
    aFerrmeanavgn[k])
393         print('MINUMUM:  avg n1-n5,aFerrmeanMin:', n1min, n2min, n3min, n4min, n5min, aFerrmeanavgnMin, ' at
    generation', kmin)
394         print('TIME AVG:    avg n1-n5,aFerrmean:', n1ta, n2ta, n3ta, n4ta, n5ta, aFerrta)
395
396     return dict(nkeephist=nkeephist, gen=gen, n1avg=n1avg, n2avg=n2avg, n3avg=n3avg, n4avg=n4avg, n5avg=n5avg
    ,
397                 aFerrmeanavgn=aFerrmeanavgn,
398                 n1min=n1min, n2min=n2min, n3min=n3min, n4min=n4min, n5min=n5min,
399                 aFerrmeanavgnMin=aFerrmeanavgnMin, kmin=kmin, kconv=kconv,
400                 n1end=n1avg[k], n2end=n2avg[k], n3end=n3avg[k], n4end=n4avg[k], n5end=n5avg[k],
401                 aFerrend=aFerrmeanavgn[k], n1ta=n1ta, n2ta=n2ta, n3ta=n3ta, n4ta=n4ta, n5ta=n5ta,
402                 aFerrta=aFerrta)
403 # end CodeP2.2F26 (function definition)
404
405 # ------------------------------ Cell 3 ------------------------------
406 # identical to Program A, Cell 2 (listed above)
407
408 # ------------------------------ Cell 4 ------------------------------
```

```
'''Task 2.3 - first run with the initial guesses (vii) of the project statement
   (0.000476, 3.028, 0.2249, 1.054, 0.217), mutation amplitude +/-9 % and NGEN = 6000,
   followed by the skeleton plot modified as in (vi)'''
r0 = run_GA(0.000476, 3.028, 0.2249, 1.054, 0.217, NGEN=6000, MUT=0.09, rseed=1)

plt.rcParams.update({'font.size': 18})
plt.figure(figsize=[10, 8])
plt.plot(r0['gen'], r0['aFerrmeanavgn'])
plt.plot(r0['gen'], r0['n1avg'])
plt.plot(r0['gen'], r0['n2avg'])
plt.plot(r0['gen'], r0['n3avg'])
#plt.legend(['aFerrmeanavgn', 'n1 avg', 'n2 avg', 'n3 avg'], loc='lower left')     # (vi) commented out
plt.plot(r0['gen'], r0['n4avg'])
plt.plot(r0['gen'], r0['n5avg'])
plt.legend([' aFerrmeanavgn', 'n1 avg', 'n2 avg', 'n3 avg', 'n4 avg', 'n5 avg'], loc='upper right')
plt.xlabel('generation')
plt.ylabel('constants and error')
plt.loglog()
plt.yticks([0.0001, 0.001, 0.01, 0.1, 1.0, 10])
plt.xticks([1, 10, 100, 1000, 10000])
plt.show()
plt.rcParams.update({'font.size': 11})

# ------------------------------ Cell 5 ------------------------------
'''(added) independent check: deterministic multi-start minimisation (scipy Nelder-Mead) of the SAME
   objective, Eq. (1.8) divided by ND. Used only to judge the genetic-algorithm result.'''
from scipy.optimize import minimize
Y = numpy.array(ydata)
lq, ldT, lP = numpy.log(Y[:, 0]), numpy.log(Y[:, 1]), numpy.log(Y[:, 4])
gg, gam = Y[:, 2], Y[:, 3]

def Fmean5(n1, n2, n3, n4, n5, lq=lq, ldT=ldT, gg=gg, gam=gam, lP=lP):
    s = gg + n4 * gen_en * gam
    if n1 <= 0 or numpy.any(s <= 0):
        return 1e9
    return numpy.mean(numpy.abs(-lq + math.log(n1) + n2 * ldT + n3 * numpy.log(s) + n5 * lP) / numpy.abs(lq))

def F5x(x):                          # x = [ln n1, n2, n3, n4, n5]
    return Fmean5(math.exp(x[0]), *x[1:])

rs = numpy.random.RandomState(0)
best = None
for trial in range(40):
    x0 = [math.log(0.000476) + rs.randn(), 3.0 + 0.3 * rs.randn(), 0.3 * math.exp(0.5 * rs.randn()),
          1.0 * math.exp(0.7 * rs.randn()), 0.3 + 0.3 * rs.randn()]
    rr = minimize(F5x, x0, method='Nelder-Mead', options=dict(xatol=1e-10, fatol=1e-13, maxiter=40000, maxfev
     =40000))
    if best is None or rr.fun < best.fun:
        best = rr
ref5 = dict(n1=math.exp(best.x[0]), n2=best.x[1], n3=best.x[2], n4=best.x[3], n5=best.x[4], F=best.fun)
print('reference minimum: n1 = %.5e  n2 = %.4f  n3 = %.4f  n4 = %.4f  n5 = %.4f   [Ferr]mean = %.5f'
      % (ref5['n1'], ref5['n2'], ref5['n3'], ref5['n4'], ref5['n5'], ref5['F']))
print('[Ferr]mean at the initial guesses (vii) of the statement: %.5f' % Fmean5(0.000476, 3.028, 0.2249,
     1.054, 0.217))

# ------------------------------ Cell 6 ------------------------------
'''Task 2.3 - range of initial guesses (5 sets) x 2 mutation amplitudes x 5 seeds (NGEN = 6000),
   then a three-stage refinement (MUT = 0.09 -> 0.03 -> 0.01, each stage restarted from the best
   constants of the previous stage) to obtain the best fit.'''
import time
SETS23 = {'A': (0.000476, 3.028, 0.2249, 1.054, 0.217),   # initial guesses (vii) of the statement
          'B': (0.001, 3.0, 0.25, 1.0, 0.01),             # no pressure effect, pure-fluid gravity exponent
          'C': (0.0001, 3.0, 0.5, 2.0, 0.5),              # stronger gravity/Marangoni and pressure effects
          'D': (0.01, 2.5, 0.1, 0.5, 1.0),                # far
          'E': (1.0, 1.0, 1.0, 1.0, 1.0)}                 # all constants of order 1
MUTS23 = [0.09, 0.20]
SEEDS = [1, 2, 3, 4, 5]
K5 = lambda r: (r['n1min'], r['n2min'], r['n3min'], r['n4min'], r['n5min'])

def summary5(r, **kw):
    d = dict(F=r['aFerrmeanavgnMin'], n1=r['n1min'], n2=r['n2min'], n3=r['n3min'], n4=r['n4min'], n5=r['n5min
     '],
             kmin=r['kmin'], k030=first_below(r['aFerrmeanavgn'], 0.03), Fend=r['aFerrend'],
             nkeep_mean=float(numpy.mean(r['nkeephist'])), nkeep0=float(numpy.mean(numpy.array(r['nkeephist'
     ]) == 0)))
```

```
    d.update(kw)
    return d

t0 = time.time()
study23 = []
for name, init in SETS23.items():
    for MUT in MUTS23:
        for s in SEEDS:
            r = run_GA(*init, NGEN=6000, MUT=MUT, rseed=s, verbose=False)
            study23.append(summary5(r, set=name, init=init, MUT=MUT, MFRAC=0.5, seed=s, mode='single'))
# set E: selection with MFRAC = 1.0 (survivors = organisms with error below the median)
for s in SEEDS[:3]:
    r = run_GA(*SETS23['E'], NGEN=6000, MUT=0.09, MFRAC=1.0, rseed=s, verbose=False)
    study23.append(summary5(r, set='E', init=SETS23['E'], MUT=0.09, MFRAC=1.0, seed=s, mode='single'))
print('single-stage study: %d runs in %.0f s' % (len(study23), time.time() - t0))

print('\n%-3s %-5s %-5s | %-8s %-8s %-8s | %-5s | %-6s | %-7s' % ('set', 'MUT', 'MFRAC', 'Fbest', 'Fmedian', 'Fworst', '<0.03', 'k030', 'nkeep=0'))
table23 = []
for name in SETS23:
    for (MUT, MF) in [(m, 0.5) for m in MUTS23] + ([(0.09, 1.0)] if name == 'E' else []):
        runs = [d for d in study23 if d['set'] == name and d['MUT'] == MUT and d['MFRAC'] == MF]
        Fs = numpy.array([d['F'] for d in runs])
        b = runs[int(numpy.argmin(Fs))]
        ks = [d['k030'] for d in runs if d['k030'] >= 0]
        row = dict(set=name, init=SETS23[name], MUT=MUT, MFRAC=MF, nruns=len(runs), Fbest=float(Fs.min()), Fmed=float(numpy.median(Fs)),
                   Fworst=float(Fs.max()), nsucc=int((Fs < 0.03).sum()), k030med=float(numpy.median(ks)) if ks else -1,
                   nkeep0=float(numpy.mean([d['nkeep0'] for d in runs])), **{k: b[k] for k in ['n1', 'n2', 'n3', 'n4', 'n5', 'seed']})
        table23.append(row)
        print('%-3s %-5s %-5s | %.5f  %.5f  %.5f | %d/%d   | %6s | %.2f   best: %.3e %.3f %.3f %.3f %.3f' % (name, MUT, MF, row['Fbest'], row['Fmed'], row['Fworst'],
              row['nsucc'], row['nruns'], '%d' % row['k030med'] if ks else 'never', row['nkeep0'], b['n1'], b['n2'], b['n3'], b['n4'], b['n5']))

# multi-stage refinement from the best single-stage run of each set (3 seeds)
t0 = time.time()
refine23 = []
for name in SETS23:
    for s in SEEDS[:3]:
        stage0 = min([d for d in study23 if d['set'] == name and d['seed'] == s], key=lambda d: d['F'])
        x = tuple(stage0[k] for k in ['n1', 'n2', 'n3', 'n4', 'n5'])
        hist_F = [stage0['F']]
        for st, MUT in enumerate([0.03, 0.01]):
            r = run_GA(*x, NGEN=6000, MUT=MUT, rseed=1000 * (st + 1) + s, verbose=False)
            if r['aFerrmeanavgnMin'] < hist_F[-1]:
                x = K5(r)
            hist_F.append(min(r['aFerrmeanavgnMin'], hist_F[-1]))
        refine23.append(dict(set=name, seed=s, start=dict(MUT=stage0['MUT'], MFRAC=stage0['MFRAC']), F_stages=hist_F,
                             F=hist_F[-1], n1=x[0], n2=x[1], n3=x[2], n4=x[3], n5=x[4]))
        print('set %s seed %d : [Ferr]mean after stages (MUT %s/0.03/0.01) = %s   n = %.4e %.4f %.4f %.4f %.4f' 
              % (name, s, stage0['MUT'], ' -> '.join('%.5f' % f for f in hist_F), *x))
print('refinement: %d sequences in %.0f s' % (len(refine23), time.time() - t0))

# ------------------------------ Cell 7 ------------------------------
'''Task 2.3 - best fit: the refinement sequence with the lowest error is replayed (same seeds) and its
   constants are used for the tables, the predicted-vs-measured plots and the validation check.'''
bestseq = min(refine23, key=lambda d: d['F'])
s = bestseq['seed']
st0 = min([d for d in study23 if d['set'] == bestseq['set'] and d['seed'] == s], key=lambda d: d['F'])
print('best sequence: set %s %s, seed %d, stage-0 MUT = %s, MFRAC = %s' % (bestseq['set'], SETS23[bestseq['set']], s, st0['MUT'], st0['MFRAC']))
stages = [run_GA(*SETS23[bestseq['set']], NGEN=6000, MUT=st0['MUT'], MFRAC=st0['MFRAC'], rseed=s, verbose=False)]
x = K5(stages[0]); Fx = stages[0]['aFerrmeanavgnMin']
for st, MUT in enumerate([0.03, 0.01]):
    print('--- refinement stage %d (MUT = %.2f) ---' % (st + 1, MUT))
    r = run_GA(*x, NGEN=6000, MUT=MUT, rseed=1000 * (st + 1) + s)
    stages.append(r)
    if r['aFerrmeanavgnMin'] < Fx:
        x, Fx = K5(r), r['aFerrmeanavgnMin']
```

```
n1min, n2min, n3min, n4min, n5min = x
best23 = dict(n1=n1min, n2=n2min, n3=n3min, n4=n4min, n5=n5min, F=Fx)
print('\nBEST FIT (Eq. 1.5): n1 = %.5e  n2 = %.4f  n3 = %.4f  n4 = %.4f  n5 = %.4f   [Ferr]mean,min = %.5f' %
      (*x, Fx))
print('reference minimum : n1 = %.5e  n2 = %.4f  n3 = %.4f  n4 = %.4f  n5 = %.4f   [Ferr]mean     = %.5f'
      % (ref5['n1'], ref5['n2'], ref5['n3'], ref5['n4'], ref5['n5'], ref5['F']))

# validation data (15 points of the Appendix, never used in the fit)
V = numpy.array(vdata)
Fval = Fmean5(*x, lq=numpy.log(V[:, 0]), ldT=numpy.log(V[:, 1]), gg=V[:, 2], gam=V[:, 3], lP=numpy.log(V[:,
    4]))
best23['Fval'] = Fval
print('validation data: [Ferr]mean = %.5f (15 points)' % Fval)

def q5(x, row):
    n1, n2, n3, n4, n5 = x
    return n1 * row[1]**n2 * (row[2] + n4 * gen_en * row[3])**n3 * row[4]**n5

stats23 = {}
for lab, D in [('train', ydata), ('val', vdata)]:
    qp = numpy.array([q5(x, row) for row in D]); qd = numpy.array([row[0] for row in D])
    dev = (qd - qp) / qp
    stats23[lab] = dict(rms_abs=float(numpy.sqrt(numpy.mean((qd - qp)**2))), rms_rel=float(numpy.sqrt(numpy.
    mean(dev**2))),
                        mean_abs_rel=float(numpy.mean(numpy.abs(dev))), within10=float(numpy.mean(numpy.abs(
    dev) <= 0.1)))
    print('%-5s: rms deviation %.2f W/cm2, rms fractional deviation %.4f, mean |dev| %.4f, within +/-10 %%:
    %.2f'
          % (lab, stats23[lab]['rms_abs'], stats23[lab]['rms_rel'], stats23[lab]['mean_abs_rel'], stats23[lab
    ]['within10']))

# deviation per test condition: is what remains random, or still systematic?
cond = [(0.098, 1.79, 5.5), (9.8, 1.79, 5.5), (19.6, 1.79, 5.5), (19.6, 1.79, 9.5), (9.8, 0.0, 7.0), (0.098,
    1.71, 5.5)]
bias23 = []
for cd in cond:
    rows = [row for row in ydata if (row[2], row[3], row[4]) == cd]
    dev = numpy.array([(row[0] - q5(x, row)) / q5(x, row) for row in rows])
    bias23.append(dict(g=cd[0], gamma=cd[1], P=cd[2], n=len(rows), mean_dev=float(dev.mean()), rms_dev=float(
    numpy.sqrt(numpy.mean(dev**2)))))
    print('g = %5.3f  gamma = %4.2f  P = %3.1f kPa (%2d points): mean deviation %+.3f   rms deviation %.3f'
          % (cd[0], cd[1], cd[2], len(rows), dev.mean(), numpy.sqrt(numpy.mean(dev**2))))

# evolution plots: (a) main GA run, (b) last refinement stage
fig, axs = plt.subplots(1, 2, figsize=(11, 4.4), sharey=True)
for a, r, title in [(axs[0], stages[0], '(a) set %s, MUT = %s, NGEN = 6000' % (bestseq['set'], st0['MUT'])),
                    (axs[1], stages[-1], '(b) final refinement stage, MUT = 0.01')]:
    g = numpy.arange(1, len(r['gen']) + 1)
    a.plot(g, r['aFerrmeanavgn'], color=C[0], lw=0.8, label=r'$[F_{err}]_{mean}$')
    for c, (k, lab) in enumerate([('n1avg', '$n_1$'), ('n2avg', '$n_2$'), ('n3avg', '$n_3$'), ('n4avg', '
    $n_4$'), ('n5avg', '$n_5$')]):
        a.plot(g, r[k], color=C[c + 1], lw=1.2, label=lab + ' avg')
    a.axhline(0.03, color=INK2, ls='--', lw=0.9)
    a.set_xscale('log'); a.set_yscale('log'); a.set_xlim(1, len(g)); a.set_ylim(1e-5, 10)
    a.set_xlabel('generation'); a.set_title(title)
axs[0].set_ylabel('population-average constants and error')
axs[0].text(1.2, 0.033, 'target 0.03', color=INK2, fontsize=9)
axs[1].legend(loc='upper center', bbox_to_anchor=(-0.05, -0.16), ncol=6)
savefig('t23_evolution')

# predicted vs measured: training (77) and validation (15) points, one symbol per test condition
fig, axs = plt.subplots(1, 2, figsize=(11, 4.8), sharey=True)
xx = numpy.array([10, 1000])
for a, D, title in [(axs[0], ydata, '(a) training data (77 points), $[F_{err}]_{mean}$ = %.4f' % Fx),
                    (axs[1], vdata, '(b) validation data (15 points), $[F_{err}]_{mean}$ = %.4f' % Fval)]:
    a.plot(xx, xx, '-', color=INK2, lw=1, label='perfect fit')
    a.plot(xx, 1.1 * xx, '--', color=INK2, lw=0.8, label=r'$\pm 10\,\%$')
    a.plot(xx, 0.9 * xx, '--', color=INK2, lw=0.8)
    for c, cd in enumerate(cond):
        rows = [row for row in D if (row[2], row[3], row[4]) == cd]
        if rows:
            a.loglog([q5(x, row) for row in rows], [row[0] for row in rows], MK[c], color=C[c], ms=7, mec='
    white', mew=0.8,
                     label=r'$g$=%g, $\gamma$=%g, $P$=%g kPa' % cd)
    a.set_xlim(30, 300); a.set_ylim(30, 300)
```

```
    plain_log_ticks(a, [30, 50, 100, 200, 300], [30, 50, 100, 200, 300])
    a.set_xlabel(r'predicted heat flux $q^{\prime\prime}_{pred}$ (W/cm$^2$), Eq. (1.5)')
    a.set_title(title)
axs[0].set_ylabel(r'measured heat flux $q^{\prime\prime}_{exp}$ (W/cm$^2$)')
axs[0].legend(loc='upper left', fontsize=8.5)
savefig('t23_parity')

json.dump(dict(best=best23, ref=ref5, base=dict(F=r0['aFerrmeanavgnMin'], n1=r0['n1min'], n2=r0['n2min'], n3=r0['n3min'],
               n4=r0['n4min'], n5=r0['n5min'], kmin=r0['kmin'], k030=first_below(r0['aFerrmeanavgn'], 0.03),
               n1end=r0['n1end'], n2end=r0['n2end'], n3end=r0['n3end'], n4end=r0['n4end'], n5end=r0['n5end'],
               Fend=r0['aFerrend'], n1ta=r0['n1ta'], n2ta=r0['n2ta'], n3ta=r0['n3ta'], n4ta=r0['n4ta'], n5ta=r0['n5ta'], Fta=r0['aFerrta']),
               bestseq=bestseq, stats=stats23, bias=bias23, table=table23, refine=refine23, runs=study23,
               F_init_vii=Fmean5(0.000476, 3.028, 0.2249, 1.054, 0.217)),
          open(RESDIR + 'task23.json', 'w'), indent=1)

# ------------------------------ Cell 8 ------------------------------
'''Task 2.3 - surface plot of q"/(Tw-Tsat)^n2 = n1 (g + n4 gen gamma)^n3 P^n5 versus g and gamma
   for 1 < g < 20 m/s2 and 0 < gamma < 2 at P = 10 kPa (best-fit constants)'''
from mpl_toolkits.mplot3d import Axes3D   # noqa: F401  (registers the 3d projection)
Pplot = 10.0
gs = numpy.linspace(1.0, 20.0, 60)
gms = numpy.linspace(0.0, 2.0, 60)
G, GM = numpy.meshgrid(gs, gms)
Z = n1min * (G + n4min * gen_en * GM)**n3min * Pplot**n5min       # W/cm2 per (deg C)^n2

fig = plt.figure(figsize=(12, 5.0))
ax = fig.add_subplot(1, 2, 1, projection='3d')
surf = ax.plot_surface(G, GM, Z * 1e4, cmap=BLUES, linewidth=0.2, edgecolor='white', antialiased=True, rstride=3, cstride=3)
ax.set_xlabel(r'$g$ (m/s$^2$)', labelpad=6); ax.set_ylabel(r'$\gamma$', labelpad=6)
ax.view_init(elev=24, azim=-128)
ax.set_box_aspect(None, zoom=0.88)
ax.set_title(r'(a) $10^4\times q^{\prime\prime}/(T_w-T_{sat})^{n_2}$ at $P = 10$ kPa ($n_2 = %.3f$)' % n2min)
a2 = fig.add_subplot(1, 2, 2)
cs = a2.contourf(G, GM, Z * 1e4, levels=14, cmap=BLUES)
cl = a2.contour(G, GM, Z * 1e4, levels=cs.levels[::2], colors='white', linewidths=0.7)
a2.clabel(cl, fmt='%.0f', fontsize=8)
cb = fig.colorbar(cs, ax=a2)
cb.set_label(r'$10^4\times q^{\prime\prime}/(T_w-T_{sat})^{n_2}$  (W cm$^{-2}$ °C$^{-n_2}$)')
a2.set_xlabel(r'gravitational acceleration $g$ (m/s$^2$)'); a2.set_ylabel(r'surface tension parameter $\gamma$')
a2.set_title('(b) contour map of the same surface')
a2.grid(False)
savefig('t23_surface')

# numbers quoted in the report
for gv, gmv in [(1, 0), (20, 0), (1, 2), (20, 2), (1, 1.79), (9.8, 1.79)]:
    print('g = %5.1f  gamma = %4.2f :  q"/(Tw-Tsat)^n2 = %.4e' % (gv, gmv, n1min * (gv + n4min * gen_en * gmv)**n3min * Pplot**n5min))
print('ratio max/min over the plotted domain: %.2f' % (Z.max() / Z.min()))

# ------------------------------ Cell 9 ------------------------------
'''(added) Task 2.3 - how well is each constant determined by the data?  Profile of the minimum
   achievable [Err]mean when one constant is fixed and the four others are re-optimised
   (deterministic minimisation of the same objective).  A flat profile = weakly determined constant.'''
prof = {}
specs = [(1, 'n2', numpy.linspace(2.6, 3.4, 9)), (2, 'n3', numpy.linspace(0.1, 1.0, 10)),
         (3, 'n4', numpy.array([0.4, 0.6, 0.8, 1.0, 1.25, 1.5, 2.0, 3.0, 4.0])), (4, 'n5', numpy.linspace(-0.2, 1.0, 9))]
xref = numpy.array([math.log(ref5['n1']), ref5['n2'], ref5['n3'], ref5['n4'], ref5['n5']])
for idx, name, vals in specs:
    out = []
    order = numpy.argsort(numpy.abs(vals - xref[idx]))      # start next to the minimum and move outwards
    sol = {}
    for j in order:
        v = vals[j]
        starts = [numpy.delete(xref, idx) * (1 + 0.05 * rs.randn(4)) for trial in range(4)]
        near = [sol[jj] for jj in sol if abs(jj - j) == 1]      # warm start from the neighbouring point(s)
        bestp = None
        for y0 in near + starts:
            rr = minimize(lambda y: F5x(numpy.insert(y, idx, v)), y0, method='Nelder-Mead',
                          options=dict(xatol=1e-9, fatol=1e-12, maxiter=20000, maxfev=20000))
            if bestp is None or rr.fun < bestp.fun:
```

```
                bestp = rr
        sol[j] = bestp.x
        out.append((j, bestp.fun))
    out = [fv for jj, fv in sorted(out)]
    prof[name] = (vals, numpy.array(out))
    print(name, ' '.join('%.3g:%.5f' % (v, f) for v, f in zip(vals, out)))

fig, axs = plt.subplots(1, 4, figsize=(12, 3.2), sharey=True)
for a, (idx, name, vals) in zip(axs, specs):
    v, f = prof[name]
    a.plot(v, f, '-o', color=C[0], ms=5, mec='white')
    a.axvline(best23[name], color=C[1], lw=1.2, label='GA best fit')
    a.axhline(0.03, color=INK2, ls='--', lw=0.9)
    a.set_xlabel('fixed $n_%s$' % name[1])
axs[0].set_ylabel(r'min $[F_{err}]_{mean}$')
axs[0].set_ylim(0.019, 0.04)
axs[0].legend(loc='upper center')
savefig('t23_profiles')
res = json.load(open(RESDIR + 'task23.json'))
res['profiles'] = {k: [list(map(float, v[0])), list(map(float, v[1]))] for k, v in prof.items()}
json.dump(res, open(RESDIR + 'task23.json', 'w'), indent=1)
```

A.4 Program D – Extra Task ET1 (`P2_ET1.ipynb`)

```
# ============================================================================
# Program D : P2_ET1.ipynb
# ============================================================================

# ------------------------------ Cell 1 ------------------------------
'''>>>>> Extra Task ET1 - fractional sensitivity of the heat flux predicted by Eq. (1.5)
   q" = n1 (Tw-Tsat)^n2 (g + n4 gen gamma)^n3 P^n5
   with the best-fit constants of Task 2.3 (read from results/task23.json written by P2_Task2.3).'''
from __future__ import print_function
import math, json, numpy
%matplotlib inline
import matplotlib.pyplot as plt

gen_en = 9.8
best = json.load(open('results/task23.json'))['best']
n1, n2, n3, n4, n5 = best['n1'], best['n2'], best['n3'], best['n4'], best['n5']
print('Task 2.3 best-fit constants: n1 = %.5e  n2 = %.4f  n3 = %.4f  n4 = %.4f  n5 = %.4f' % (n1, n2, n3, n4,
      n5))

def qpp(dT, g, gam, P):
    return n1 * dT**n2 * (g + n4 * gen_en * gam)**n3 * P**n5

# typical sets of inputs [(Tw-Tsat)1, g1, gamma1, P1]
STATES = {'S1 (earth gravity)': (40.0, 9.8, 1.79, 5.5),
          'S2 (microgravity)':  (40.0, 0.098, 1.79, 5.5),
          'S3 (2 gen, 9.5 kPa)': (35.0, 19.6, 1.79, 9.5)}
DX_SUPERHEAT = 2.0      # deg C  (step 2 of the statement)
FRAC = 0.05             # 5 % step for g, gamma and P (same relative size as 2 C / 40 C)

et1 = {}
for sname, (dT1, g1, gam1, P1) in STATES.items():
    q1 = qpp(dT1, g1, gam1, P1)                                    # step 1
    out = {'q1': q1}
    for var, x1 in [('dT', dT1), ('g', g1), ('gamma', gam1), ('P', P1)]:
        dx = DX_SUPERHEAT if var == 'dT' else FRAC * x1
        args = dict(dT=dT1, g=g1, gam=gam1, P=P1)
        args['gam' if var == 'gamma' else var] = x1 + dx           # step 2: change only this input
        q2 = qpp(**args)
        S = (q2 - q1) * x1 / (dx * q1)                             # step 3: fractional change ratio
        out[var] = dict(x1=x1, dx=dx, q2=q2, S=S)
    # analytical logarithmic derivatives (limit dx -> 0), for comparison
    s = g1 + n4 * gen_en * gam1
    out['analytic'] = dict(dT=n2, g=n3 * g1 / s, gamma=n3 * n4 * gen_en * gam1 / s, P=n5)
    et1[sname] = out
    print('\n%s: Tw-Tsat = %.1f C, g = %.3f m/s2, gamma = %.2f, P = %.1f kPa  ->  q"1 = %.2f W/cm2' % (sname,
      dT1, g1, gam1, P1, q1))
    for var in ['dT', 'g', 'gamma', 'P']:
        o = out[var]
```

```
        print('   %-6s x1 = %7.3f  dx = %6.3f  q"2 = %7.2f  dq"/q" / (dx/x) = %7.4f   (analytic %.4f)'
              % (var, o['x1'], o['dx'], o['q2'], o['S'], out['analytic'][var]))

# ------------------------------ Cell 2 ------------------------------
# identical to Program A, Cell 2 (listed above)

# ------------------------------ Cell 3 ------------------------------
'''ET1 - limiting fractional inaccuracy of the inputs for |dq"/q"| < 0.15
   To first order  dq"/q" = S_dT dT/dT + S_g dg/g + S_gamma dgamma/gamma + S_P dP/P, so:
   (i)   one input inaccurate at a time   : eps_x < 0.15 / |S_x|
   (ii)  all inputs with the same eps, errors adding in the worst direction : eps < 0.15 / sum|S_x|
   (iii) same eps, independent random errors (root-sum-square)            : eps < 0.15 / sqrt(sum S_x^2)'''
QTOL = 0.15
VARS = ['dT', 'g', 'gamma', 'P']
lim = {}
for sname, out in et1.items():
    S = numpy.array([abs(out[v]['S']) for v in VARS])
    single = QTOL / S
    worst = QTOL / S.sum()
    rss = QTOL / math.sqrt((S**2).sum())
    lim[sname] = dict(single=dict(zip(VARS, single.tolist())), worst=worst, rss=rss)
    # nonlinear check: all inputs off by the worst-case eps in the direction that increases q"
    dT1, g1, gam1, P1 = [out[v]['x1'] for v in VARS]
    sgn = numpy.sign([out[v]['S'] for v in VARS])
    qw = qpp(dT1 * (1 + sgn[0] * worst), g1 * (1 + sgn[1] * worst), gam1 * (1 + sgn[2] * worst), P1 * (1 +
     sgn[3] * worst))
    lim[sname]['check_worst'] = qw / out['q1'] - 1
    print('%s:' % sname)
    print('   (i)   single input : ' + ', '.join('%s < %.3f' % (v, e) for v, e in zip(VARS, single)) +
          '   -> superheat within +/- %.2f C' % (single[0] * dT1))
    print('   (ii)  worst case   : eps < %.4f  (superheat +/- %.2f C, g +/- %.3f m/s2, gamma +/- %.3f, P +/-
     %.3f kPa);'
          ' nonlinear check dq"/q" = %.4f' % (worst, worst * dT1, worst * g1, worst * gam1, worst * P1, lim[
     sname]['check_worst']))
    print('   (iii) RSS          : eps < %.4f  (superheat +/- %.2f C)' % (rss, rss * dT1))

fig, ax = plt.subplots(figsize=(6.6, 3.6))
w = 0.26
for c, (sname, out) in enumerate(et1.items()):
    ax.bar(numpy.arange(4) + (c - 1) * w, [out[v]['S'] for v in VARS], width=w - 0.03, color=C[c], label=
     sname)
ax.set_xticks(range(4))
ax.set_xticklabels([r'superheat $T_w-T_{sat}$', r'gravity $g$', r'surface tension $\gamma$', r'pressure $P$'
     ])
ax.set_ylabel(r'$(\Delta q^{\prime\prime}/q^{\prime\prime})/(\Delta x/x)$')
ax.axhline(0, color='#52514e', lw=0.8)
ax.legend(loc='upper right')
ax.grid(axis='x', visible=False)
savefig('et1_sensitivity')
json.dump(dict(constants=best, states=STATES, sens=et1, limits=lim), open(RESDIR + 'et1.json', 'w'), indent
     =1)
```

A.5 Program E – Extra Task ET2 (`P2_ET2.ipynb`)

```
# ============================================================================
# Program E : P2_ET2.ipynb
# ============================================================================

# ------------------------------ Cell 1 ------------------------------
# identical to Program C, Cell 1 (listed above)

# ------------------------------ Cell 2 ------------------------------
'''>>>>> Extra Task ET2 - reorganisation of the Task 2.3 data set (77 points + 15 validation points):
   q", g, gamma and P become the inputs and the superheat (Tw - Tsat) becomes the output.
   Each row [q", Tw-Tsat, g, gamma, P] becomes [Tw-Tsat, q", g, gamma, P] and lydata is recomputed,
   so that column 0 (the fitted output) is now ln(Tw-Tsat) and column 1 is ln q".'''
ydata = [[row[1], row[0], row[2], row[3], row[4]] for row in ydata]
vdata = [[row[1], row[0], row[2], row[3], row[4]] for row in vdata]
lydata = deepcopy(ydata)
for j in range(DI):
    for i in range(ND):
        lydata[i][j] = math.log(ydata[i][j] + 0.000000000010)
```

```
print('reorganised data: ND =', ND, ' first rows [Tw-Tsat, q", g, gamma, P]:', ydata[:2])

# ------------------------------ Cell 3 ------------------------------
'''>>>>> start CodeP2.2F26
    V.P. Carey ME249, Fall 2026
    MODIFIED for Extra Task ET2 - copy of the Task 2.3 program used to fit the inverse relation
        (Tw-Tsat) = nh1 (q")^nh2 (g + nh4 gen gamma)^nh3 P^nh5          (ET2.1)
    Because the data were reorganised as [Tw-Tsat, q", g, gamma, P], the error statements of the
    Task 2.3 program become exactly the error function of (ET2.1):
        ferr,i = -ln(Tw-Tsat)_i + ln nh1 + nh2 ln q"_i + nh3 ln(g_i + nh4 gen gamma_i) + nh5 ln P_i
        aFerr_i = |ferr,i| / |ln(Tw-Tsat)_i|
    so only the comments, the print-out labels and the initial guesses (no defaults, nh3 and nh5
    are negative - the multiplicative mutations keep their sign) were changed.
    In the code below n1 ... n5 stand for nh1 ... nh5.  As in Task 2.2 / 2.3:
      - the evolution loop of the skeleton code is placed, unchanged, inside a
        function run_GA(...) so that it can be re-run for several initial guesses,
        mutation amplitudes, numbers of generations and random seeds
      - the maximum mutation amplitude (0.085 in the skeleton) is now the
        argument MUT (relative amplitude, i.e. +/- MUT*100 %)
      - both random generators (python random and numpy.random) are seeded with
        the argument rseed so that every run is reproducible
      - the history of the population-average constants and of aFerrmeanavgn is
        returned, together with the generation at which aFerrmeanavgn first drops
        below the target value (convergence-speed indicator) and the number of
        survivors nkeep of each generation (selection diagnostic)
      - BUG FIX: the skeleton line "import random" re-binds the name random to the
        module and hides the function random() imported in CodeP2.1F26, so the
        initialisation n1i+0.001*random() raises "TypeError: 'module' object is
        not callable".  The line is replaced by an explicit import of the function.'''
from random import random, seed   # (was: import random)

gen_en = 9.8   # earth-normal gravitational acceleration, m/s2

def run_GA(n1i, n2i, n3i, n4i, n5i, NGEN=6000, MUT=0.09,
           MFRAC=0.5, rseed=1, target=0.03, verbose=True):

    '''INITIALIZING PARAMETERS'''
    seed(rseed)                 # python random  (initial population)
    numpy.random.seed(rseed)    # numpy random   (mating / mutation)
    n = []
    ntemp = []
    gen=[0]
    n1avg = [0.0]
    n2avg = [0.0]
    n3avg = [0.0]
    n4avg = [0.0]
    n5avg = [0.0]
    meanAFerr=[0.0]
    aFerrmeanavgn=[0.0]

    # here the number of data vectors equals the number of DNA strands (or organisms)
    for k in range(NGEN-1):
        gen.append(k+1)   # generation array stores the
        meanAFerr.append(0.0)
        aFerrmeanavgn.append(0.0)
        n1avg.append(0.0)
        n2avg.append(0.0)
        n3avg.append(0.0)
        n4avg.append(0.0)
        n5avg.append(0.0)

    '''guesses for initial solution population: n0i = -1 (fixed), n1i ... n5i are arguments'''
    n0i =  -1.0

    #i initialize array where rows are dna vectors [n0i,n1i,...n5i] with random pertubations
    n =  [[-1., n1i+0.001*random(), n2i+0.1*random(), n3i+0.0001*random(), n4i+0.0001*random(), n5i+0.0001*
     random()]]
    for i in range(ND):
        n.append([-1., n1i+0.0001*random(), n2i+0.001*random(), n3i+0.0001*random(), n4i+0.0001*random(), n5i
     +0.0001*random()])

    # store also in wtemp
    ntemp = deepcopy(n)

    #define arrays of Ferr (error) functions
```

```
    Ferr =  [[0.0]]
    Ferravgn =  [[0.0]]
    aFerr =  [[0.0]]
    aFerravgn =  [[0.0]]
    for i in range(ND-1):
        Ferr.append([0.0])
        aFerr.append([0.0])
        Ferravgn.append([0.0])
        aFerravgn.append([0.0])

    aFerrmeanavgnMin=1000000000.0
    # these store the  n values for minimum population average error durng NGEN generations
    n1min = 0.0
    n2min = 0.0
    n3min = 0.0
    n4min = 0.0
    n5min = 0.0
    kmin = 0
    aFerrta = 0.0
    # these store the time averaged n values durng from generation 800 to NGEN generations
    n1ta = 0.0
    n2ta = 0.0
    n3ta = 0.0
    n4ta = 0.0
    n5ta = 0.0
    kconv = -1      # first generation with aFerrmeanavgn < target (-1 = never)
    nkeephist = []  # (added) number of survivors nkeep at each generation

    '''START OF EVOLUTION LOOP'''
    for k in range(NGEN):

        '''CALCULATING ERROR (FITNESS) - Eq. (1.3): organism i is paired with data point i'''
        for i in range(ND):

            # (ET2.1): -ln(Tw-Tsat) + ln nh1 + nh2 ln q" + nh3 ln(g + nh4 gen gamma) + nh5 ln P
            Ferr[i] = n[i][0]*lydata[i][0] + math.log(n[i][1]) + n[i][2]*lydata[i][1]
            Ferr[i] = Ferr[i] + n[i][3]*math.log( ydata[i][2] + n[i][4]*gen_en*ydata[i][3] ) + n[i][5]*lydata
     [i][4]

            aFerr[i] = abs(Ferr[i])/abs(lydata[i][0])  #- absolute fractional error
        #-------------
        aFerrmean = numpy.mean(aFerr) #mean error for population for this generation
        meanAFerr[k]=aFerrmean  #store aFerrmean for this generation gen[k]=k
        aFerrmedian = numpy.median(aFerr) #median error for population for this generation

        '''SELECTION'''
        clim = MFRAC*aFerrmedian  #cut off limit is a fraction/multiplier MFRAC of median value for population
        nkeep = 0
        for j in range(NS):  # NS Ferr values, one for each solution in population
            if (aFerr[j] < clim):
                nkeep = nkeep + 1
                #ntemp[nkeep][0] = n[j][0] = -1 so it is unchanged;
                ntemp[nkeep-1][1] = n[j][1];
                ntemp[nkeep-1][2] = n[j][2];
                ntemp[nkeep-1][3] = n[j][3];
                ntemp[nkeep-1][4] = n[j][4];
                ntemp[nkeep-1][5] = n[j][5];
        #compute number to be added by mating
        nnew = NS - nkeep
        nkeephist.append(nkeep)   # (added) number of survivors, diagnostic of selection pressure

        '''MATING'''
        for j in range(nnew):
            # pick two survivors randomly
            nmate1 = numpy.random.randint(low=0, high=nkeep+1)
            nmate2 = numpy.random.randint(low=0, high=nkeep+1)

            '''here, do not change property ntemp[nkeep+j+1][0], it's always fixed at the value -1'''
            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][1] = n[nmate1][1]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 1,
     mutation added
            else:
                ntemp[nkeep+j+1][1] = n[nmate2][1]*(1.+MUT*2.*(0.5-numpy.random.rand()))

            if (numpy.random.rand() < 0.5):
```

```
                ntemp[nkeep+j+1][2] = n[nmate1][2]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 2, mutation added
            else:
                ntemp[nkeep+j+1][2] = n[nmate2][2]*(1.+MUT*2.*(0.5-numpy.random.rand()))

            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][3] = n[nmate1][3]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 3, mutation added
            else:
                ntemp[nkeep+j+1][3] = n[nmate2][3]*(1.+MUT*2.*(0.5-numpy.random.rand()))

            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][4] = n[nmate1][4]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 4, mutation added
            else:
                ntemp[nkeep+j+1][4] = n[nmate2][4]*(1.+MUT*2.*(0.5-numpy.random.rand()))

            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][5] = n[nmate1][5]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 5, mutation added
            else:
                ntemp[nkeep+j+1][5] = n[nmate2][5]*(1.+MUT*2.*(0.5-numpy.random.rand()))
        #============================================
        n = deepcopy(ntemp)    # save ntemp as n for use in next generation (next iteration)

        '''AVERAGING OVER POPULATION AND OVER TIME, FINDING MINIMUM ERROR SET OF n VALUES'''
        n1avg[k] = 0.0;
        n2avg[k] = 0.0;
        n3avg[k] = 0.0;
        n4avg[k] = 0.0;
        n5avg[k] = 0.0;
        for j in range(NS):
            n1avg[k] = n1avg[k] + n[j][1]/NS;
            n2avg[k] = n2avg[k] + n[j][2]/NS;
            n3avg[k] = n3avg[k] + n[j][3]/NS;
            n4avg[k] = n4avg[k] + n[j][4]/NS;
            n5avg[k] = n5avg[k] + n[j][5]/NS;

        ''' CALCULATING MEAN ERROR FOR POPULATION (population-average constants vs all data)'''
        for i in range(ND):
            Ferravgn[i] = -1.*lydata[i][0] + math.log(n1avg[k]) + n2avg[k]*lydata[i][1]
            Ferravgn[i] = Ferravgn[i] + n3avg[k]*math.log( ydata[i][2] + n4avg[k]*gen_en*ydata[i][3] ) + n5avg[k]*lydata[i][4]

            aFerravgn[i] = abs(Ferravgn[i])/abs(lydata[i][0])
        #-------------
        aFerrmeanavgn[k] = numpy.mean(aFerravgn)
        if (kconv < 0 and aFerrmeanavgn[k] < target):
            kconv = k

        # time average of n values in population for generations k > 800
        aFerrta = aFerrta + aFerrmeanavgn[k]/NGEN
        if (k > 800):
            n1ta = n1ta + n1avg[k]/(NGEN-800)
            n2ta = n2ta + n2avg[k]/(NGEN-800)
            n3ta = n3ta + n3avg[k]/(NGEN-800)
            n4ta = n4ta + n4avg[k]/(NGEN-800)
            n5ta = n5ta + n5avg[k]/(NGEN-800)

        # save minimum population-average error and corresponding n(i) values
        if (aFerrmeanavgn[k] < aFerrmeanavgnMin):
            aFerrmeanavgnMin = aFerrmeanavgn[k]
            n1min = n1avg[k]
            n2min = n2avg[k]
            n3min = n3avg[k]
            n4min = n4avg[k]
            n5min = n5avg[k]
            kmin = k
        '''end of evolution loop'''

    if verbose:
        print('ENDING: pop.avg nh1-nh5,aFerrmean:', n1avg[k], n2avg[k], n3avg[k], n4avg[k], n5avg[k], aFerrmeanavgn[k])
        print('MINUMUM:  avg nh1-nh5,aFerrmeanMin:', n1min, n2min, n3min, n4min, n5min, aFerrmeanavgnMin, ' at generation', kmin)
```

```
        print('TIME AVG:    avg nh1-nh5,aFerrmean:', n1ta, n2ta, n3ta, n4ta, n5ta, aFerrta)

    return dict(nkeephist=nkeephist, gen=gen, n1avg=n1avg, n2avg=n2avg, n3avg=n3avg, n4avg=n4avg, n5avg=n5avg
    ,
                aFerrmeanavgn=aFerrmeanavgn,
                n1min=n1min, n2min=n2min, n3min=n3min, n4min=n4min, n5min=n5min,
                aFerrmeanavgnMin=aFerrmeanavgnMin, kmin=kmin, kconv=kconv,
                n1end=n1avg[k], n2end=n2avg[k], n3end=n3avg[k], n4end=n4avg[k], n5end=n5avg[k],
                aFerrend=aFerrmeanavgn[k], n1ta=n1ta, n2ta=n2ta, n3ta=n3ta, n4ta=n4ta, n5ta=n5ta,
                aFerrta=aFerrta)
# end CodeP2.2F26 (function definition)

# ------------------------------ Cell 4 ------------------------------
# identical to Program A, Cell 2 (listed above)

# ------------------------------ Cell 5 ------------------------------
'''ET2 - values implied by the Task 2.3 fit, genetic-algorithm fit of (ET2.1), comparison table.'''
import time
from scipy.optimize import minimize
K5 = lambda r: (r['n1min'], r['n2min'], r['n3min'], r['n4min'], r['n5min'])
b23 = json.load(open(RESDIR + 'task23.json'))
fwd = b23['best']                    # Task 2.3 GA best fit
fref = b23['ref']                    # Task 2.3 deterministic reference minimum

def implied(c):
    '''nh values obtained by solving Eq. (1.5) for (Tw - Tsat)'''
    return dict(n1=c['n1']**(-1. / c['n2']), n2=1. / c['n2'], n3=-c['n3'] / c['n2'], n4=c['n4'], n5=-c['n5']
    / c['n2'])
imp = implied(fwd)
print('implied by Task 2.3 GA fit : nh1 = %.4f  nh2 = %.5f  nh3 = %.5f  nh4 = %.4f  nh5 = %.5f' % tuple(imp[k
    ] for k in ['n1', 'n2', 'n3', 'n4', 'n5']))

# generic objective (same as the GA): mean | -ln y + ln c1 + c2 ln x + c3 ln(g + c4 gen gamma) + c5 ln P | / |
    ln y|
Y = numpy.array(ydata)
def Fgen(c, ly, lx, gg, gam, lP):
    s = gg + c[3] * gen_en * gam
    if c[0] <= 0 or numpy.any(s <= 0):
        return 1e9
    return numpy.mean(numpy.abs(-ly + math.log(c[0]) + c[1] * lx + c[2] * numpy.log(s) + c[4] * lP) / numpy.
    abs(ly))
lT, lQ, GG, GM, LP = numpy.log(Y[:, 0]), numpy.log(Y[:, 1]), Y[:, 2], Y[:, 3], numpy.log(Y[:, 4])
Finv = lambda c: Fgen(c, lT, lQ, GG, GM, LP)          # inverse relation (ET2.1)
Ffwd = lambda c: Fgen(c, lQ, lT, GG, GM, LP)          # forward relation (1.5)

def fitNM(F, c0):
    '''deterministic multi-start check (Nelder-Mead on [ln c1, c2..c5])'''
    best = None
    for k in range(6):
        x0 = numpy.array([math.log(c0[0]), *c0[1:]]) * (1 + (0.05 * numpy.random.RandomState(k).randn(5) if k
     else 0))
        rr = minimize(lambda x: F([math.exp(x[0]), *x[1:]]), x0, method='Nelder-Mead',
                      options=dict(xatol=1e-10, fatol=1e-13, maxiter=40000, maxfev=40000))
        if best is None or rr.fun < best.fun:
            best = rr
    return [math.exp(best.x[0]), *best.x[1:]], best.fun

print('[Ferr]mean of (ET2.1) with the implied values: %.5f' % Finv([imp[k] for k in ['n1', 'n2', 'n3', 'n4',
    'n5']]))

# genetic algorithm: 3 sets of initial guesses x 3 seeds, then refinement MUT 0.03 -> 0.01
SETSE2 = {'I (implied by Task 2.3)': tuple(imp[k] for k in ['n1', 'n2', 'n3', 'n4', 'n5']),
          'II': (10.0, 0.5, -0.1, 1.0, -0.1),
          'III': (40.0, 0.25, -0.25, 2.0, -0.3)}
t0 = time.time()
seqE2 = []
for name, init in SETSE2.items():
    for s in [1, 2, 3]:
        r = run_GA(*init, NGEN=6000, MUT=0.09, rseed=s, verbose=False)
        x, Fx, Fst = K5(r), r['aFerrmeanavgnMin'], [r['aFerrmeanavgnMin']]
        for st, MUT in enumerate([0.03, 0.01]):
            r2 = run_GA(*x, NGEN=6000, MUT=MUT, rseed=1000 * (st + 1) + s, verbose=False)
            if r2['aFerrmeanavgnMin'] < Fx:
                x, Fx = K5(r2), r2['aFerrmeanavgnMin']
            Fst.append(Fx)
```

```
        seqE2.append(dict(set=name, init=init, seed=s, F=Fx, F_stages=Fst, n1=x[0], n2=x[1], n3=x[2], n4=x[3], n5=x[4],
                          F0=r['aFerrmeanavgnMin'], k0=r['kmin']))
        print('%-24s seed %d : [Ferr]mean %s   nh = %.4f %.5f %.5f %.4f %.5f' % (name, s, ' -> '.join('%.5f' % f for f in Fst), *x))
print('GA runs: %.0f s' % (time.time() - t0))
bE2 = min(seqE2, key=lambda d: d['F'])
ga2 = {k: bE2[k] for k in ['n1', 'n2', 'n3', 'n4', 'n5']}

# deterministic references: exact minimum of (ET2.1) and values implied by the exact minimum of (1.5)
cinv, Finv_ref = fitNM(Finv, [ga2[k] for k in ['n1', 'n2', 'n3', 'n4', 'n5']])
inv_ref = dict(zip(['n1', 'n2', 'n3', 'n4', 'n5'], cinv))
imp_ref = implied(fref)

print('\n%-5s | %-11s %-11s %-9s | %-11s %-11s %-9s' % ('', 'GA (ET2.1)', 'implied GA', 'diff %', 'exact ET2.1', 'impl. exact', 'diff %'))
tabE2 = []
for k in ['n1', 'n2', 'n3', 'n4', 'n5']:
    d1 = 100 * (ga2[k] - imp[k]) / abs(imp[k]); d2 = 100 * (inv_ref[k] - imp_ref[k]) / abs(imp_ref[k])
    tabE2.append(dict(k=k, ga=ga2[k], imp=imp[k], diff=d1, inv_ref=inv_ref[k], imp_ref=imp_ref[k], diff_ref=d2))
    print('nh%s   | %-11.5g %-11.5g %+8.1f  | %-11.5g %-11.5g %+8.1f' % (k[1], ga2[k], imp[k], d1, inv_ref[k], imp_ref[k], d2))
print('[Ferr]mean of (ET2.1): GA best %.5f, exact minimum %.5f, with implied values %.5f'
      % (bE2['F'], Finv_ref, Finv([imp[k] for k in ['n1', 'n2', 'n3', 'n4', 'n5']])))

# superheat predicted by the two routes
def dT_inv(c, row):            # row = [Tw-Tsat, q", g, gamma, P]
    return c['n1'] * row[1]**c['n2'] * (row[2] + c['n4'] * gen_en * row[3])**c['n3'] * row[4]**c['n5']
for lab, c in [('GA fit of (ET2.1)', ga2), ('Eq. (1.5) solved for Tw-Tsat', imp)]:
    for D, nm in [(ydata, 'train'), (vdata, 'valid')]:
        p = numpy.array([dT_inv(c, row) for row in D]); m = numpy.array([row[0] for row in D])
        print('%-30s %s: rms deviation of superheat = %.3f C, mean |dev|/dT = %.4f' % (lab, nm, numpy.sqrt(numpy.mean((m - p)**2)), numpy.mean(numpy.abs(m - p) / m)))

fig, ax = plt.subplots(figsize=(6.0, 4.8))
xx = numpy.array([20, 70])
ax.plot(xx, xx, '-', color=INK2, lw=1, label='perfect fit')
ax.plot([dT_inv(imp, row) for row in ydata], [row[0] for row in ydata], 's', color=C[1], ms=6.5, mec='white', mew=0.7, label='Eq. (1.5) solved for $T_w-T_{sat}$')
ax.plot([dT_inv(ga2, row) for row in ydata], [row[0] for row in ydata], 'o', color=C[0], ms=6.5, mec='white', mew=0.7, label='GA fit of (ET2.1)')
ax.set_xlim(25, 60); ax.set_ylim(25, 60)
ax.set_xlabel(r'predicted superheat (°C)'); ax.set_ylabel(r'measured superheat $T_w-T_{sat}$ (°C)')
ax.legend(loc='upper left')
ax.set_title('ET2: superheat predicted by the two routes (77 points)')
savefig('et2_parity')

# ------------------------------ Cell 6 ------------------------------
'''ET2 - is the noise in the data responsible for the differences?  Test with synthetic data generated
   from Eq. (1.5) (Task 2.3 constants) at the 77 measured (q" or Tw-Tsat, g, gamma, P):
   (a) noise-free data  -> the GA fit of (ET2.1) must recover the implied values exactly;
   (b) scatter added to q" (log-normal, same rms as the residuals of the Task 2.3 fit), 30 realisations;
   (c) the same scatter expressed as superheat scatter (sigma/n2 on ln(Tw-Tsat)), 30 realisations.
   For (b) and (c) the forward fit (1.5) and the inverse fit (ET2.1) are both made with the deterministic
   minimiser of the same objectives (much faster than the GA, identical minimum).'''
KEYS = ['n1', 'n2', 'n3', 'n4', 'n5']
cf = [fwd[k] for k in KEYS]
def q_model(c, dT, g, gam, P):
    return c[0] * dT**c[1] * (g + c[3] * gen_en * gam)**c[2] * P**c[4]
qd, dTd = Y[:, 1], Y[:, 0]
res_ln = numpy.log(qd) - numpy.log(q_model(cf, dTd, GG, GM, Y[:, 4]))
sig = float(numpy.sqrt(numpy.mean(res_ln**2)))
print('rms of ln(q"_data/q"_pred) for the Task 2.3 fit: sigma = %.4f' % sig)

# (a) noise-free: superheat computed exactly from Eq. (1.5) at the measured q", g, gamma, P
imp_v = [imp[k] for k in KEYS]
dT_exact = imp_v[0] * qd**imp_v[1] * (GG + imp_v[3] * gen_en * GM)**imp_v[2] * Y[:, 4]**imp_v[4]
ydata_saved, lydata_saved = deepcopy(ydata), deepcopy(lydata)
for i in range(ND):
    ydata[i][0] = float(dT_exact[i]); lydata[i][0] = math.log(dT_exact[i])
start = (15.0, 0.4, -0.2, 1.0, -0.1)                    # deliberately away from the implied values
r = run_GA(*start, NGEN=6000, MUT=0.09, rseed=7, verbose=False)
x, Fx = K5(r), r['aFerrmeanavgnMin']
for st, MUT in enumerate([0.03, 0.01, 0.003]):
```

```
    r2 = run_GA(*x, NGEN=6000, MUT=MUT, rseed=70 + st, verbose=False)
    if r2['aFerrmeanavgnMin'] < Fx:
        x, Fx = K5(r2), r2['aFerrmeanavgnMin']
ydata, lydata = ydata_saved, lydata_saved
noisefree = dict(zip(KEYS, x)); noisefree['F'] = Fx
print('(a) noise-free synthetic data, GA fit of (ET2.1): [Ferr]mean = %.2e' % Fx)
for k in KEYS:
    print('    nh%s  GA %.5g   implied %.5g   diff %+.2f %%' % (k[1], noisefree[k], imp[k], 100 * (noisefree[
    k] - imp[k]) / abs(imp[k])))

# (b) and (c): noisy synthetic data
rsn = numpy.random.RandomState(249)
NREAL = 30
synth = {'b': [], 'c': []}
t0 = time.time()
for case in ['b', 'c']:
    for k in range(NREAL):
        eps = rsn.randn(ND)
        if case == 'b':
            q_s, dT_s = q_model(cf, dTd, GG, GM, Y[:, 4]) * numpy.exp(sig * eps), dTd
        else:
            dT_s = dT_exact * numpy.exp(sig / fwd['n2'] * eps); q_s = qd
        cfw, _ = fitNM(lambda c: Fgen(c, numpy.log(q_s), numpy.log(dT_s), GG, GM, LP), cf)
        civ, _ = fitNM(lambda c: Fgen(c, numpy.log(dT_s), numpy.log(q_s), GG, GM, LP), imp_v)
        im = implied(dict(zip(KEYS, cfw)))
        synth[case].append([100 * (civ[j] - im[KEYS[j]]) / abs(im[KEYS[j]]) for j in range(5)])
    synth[case] = numpy.array(synth[case])
print('synthetic noisy fits: %.0f s' % (time.time() - t0))
print('\npercentage difference  (inverse fit - implied by forward fit) / |implied| :')
print('%-5s | %-16s | %-22s | %-22s' % ('', 'measured data', '(b) noise on q" (30x)', '(c) noise on dT (30x)'
    ))
noise_tab = []
for j, k in enumerate(KEYS):
    row = dict(k=k, data=tabE2[j]['diff_ref'], b_mean=float(synth['b'][:, j].mean()), b_std=float(synth['b'
    ][:, j].std()),
               c_mean=float(synth['c'][:, j].mean()), c_std=float(synth['c'][:, j].std()))
    noise_tab.append(row)
    print('nh%s   | %+8.1f         | %+8.1f +/- %6.1f      | %+8.1f +/- %6.1f' % (k[1], row['data'], row['
    b_mean'], row['b_std'], row['c_mean'], row['c_std']))

json.dump(dict(implied=imp, implied_ref=imp_ref, ga=ga2, ga_F=bE2['F'], inv_ref=inv_ref, inv_ref_F=Finv_ref,
    table=tabE2, runs=seqE2,
               noisefree=noisefree, sigma=sig, noise_table=noise_tab, F_implied=Finv(imp_v)),
          open(RESDIR + 'et2.json', 'w'), indent=1)
```

A.6 Program F – Tasks 2.4 and 2.5 (`P2_Task2.4-2.5.ipynb`)

```
# ============================================================================
# Program F : P2_Task2.4-2.5.ipynb
# ============================================================================

# ------------------------------ Cell 1 ------------------------------
'''>>>>> start CodeP2.3F26
    V.P. Carey ME249, Fall 2026
    MODIFIED for Tasks 2.4 / 2.5:
      - matplotlib imported (plots are made in this program)
      - the 122 data points are kept in yalldata and randomly split (fixed seed,
        reproducible) into a training set (ydata, ND = 98 points, about 80 %) and a
        validation set (vdata, NV = 24 points, about 20 %); the genetic algorithm
        only uses ydata, as the Task 2.2 program does

    Heat pipe performance data for Part 2 of Project 2
   122 data points'''

# version 3 print function
from __future__ import print_function
# seed the pseudorandom number generator
from random import random
from random import seed
# seed random number generator
seed(1)

```

```
#import math and numpy packages
import math
import numpy as np
import numpy

%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = [10, 8] # for square canvas

#import copy
from copy import copy, deepcopy


#create array
ydata = []


# j is column, i is row downward for ydata[i][j] - both start at zero
# so it is:  ydata[row][column]
# this is an array that is essentially a list of lists

#assembling heat pipe performance data array
#=========================================
#store array where rows are data vectors
#[Ta,in(deg C), Ta,out(deg C), qdot(W)]

ydata =     [[ 27.2,     15.9,      31.2]]
ydata.append([ 29.6,     17.1,      37.0])
ydata.append([ 29.5,     18.1,      34.2])
ydata.append([ 30.4,     19.0,      31.])
ydata.append([ 30.4,     20.0,      28.5])
ydata.append([ 30.2,     21.0,      25.7])
ydata.append([ 29.8,     22.1,      22.8])
ydata.append([ 29.8,     22.9,      20.0])
ydata.append([ 29.9,     24.1,      17.1])
ydata.append([ 29.6,     24.9,      14.3])
ydata.append([ 29.5,     26.1,      11.4])
ydata.append([ 30.3,     26.9,       8.6])
ydata.append([ 32.5,     16.9,      45.7])
ydata.append([ 32.6,     18.1,      42.8])
ydata.append([ 32.6,     19.1,      40.0])
ydata.append([ 32.7,     20.2,      37.2])
ydata.append([ 32.9,     21.1,      34.3])
ydata.append([ 33.0,     22.0,      31.5])
ydata.append([ 32.5,     23.1,      28.7])
ydata.append([ 32.8,     24.1,      25.8])
ydata.append([ 33.0,     24.9,      23.0])
ydata.append([ 32.9,     26.0,      20.1])
ydata.append([ 33.1,     27.0,      17.2])
ydata.append([ 35.7,     17.0,      54.4])
ydata.append([ 35.6,     18.1,      51.6])
ydata.append([ 36.2,     19.1,      48.7])
ydata.append([ 36.3,     20.2,      45.9])
ydata.append([ 36.4,     21.1,      43.1])
ydata.append([ 35.5,     22.2,      40.2])
ydata.append([ 36.2,     22.9,      37.4])
ydata.append([ 36.2,     24.1,      34.5])
ydata.append([ 35.7,     25.0,      31.7])
ydata.append([ 36.0,     26.1,      28.8])
ydata.append([ 35.8,     27.1,      25.9])
ydata.append([ 38.8,     16.9,      63.1])
ydata.append([ 39.4,     18.1,      60.3])
ydata.append([ 38.9,     19.2,      57.5])
ydata.append([ 39.2,     20.1,      54.7])
ydata.append([ 38.5,     21.2,      51.8])
ydata.append([ 39.0,     22.0,      49.0])
ydata.append([ 39.0,     23.1,      46.1])
ydata.append([ 39.3,     24.0,      43.3])
ydata.append([ 39.3,     24.9,      40.4])
ydata.append([ 39.1,     26.2,      37.6])
ydata.append([ 39.1,     27.2,      34.7])
ydata.append([ 42.2,     17.0,      71.9])
ydata.append([ 42.3,     18.1,      69.1])
ydata.append([ 42.4,     18.9,      66.3])
ydata.append([ 41.9,     20.0,      63.5])
```

```
ydata.append([ 41.8,     21.0,      60.6])
ydata.append([ 41.8,     22.1,      57.8])
ydata.append([ 42.2,     22.8,      54.9])
ydata.append([ 41.5,     24.2,      52.1])
ydata.append([ 42.3,     25.3,      49.2])
ydata.append([ 41.8,     25.9,      46.3])
ydata.append([ 42.3,     26.8,      43.5])
ydata.append([ 45.2,     17.1,      80.8])
ydata.append([ 44.5,     18.2,      78.0])
ydata.append([ 44.8,     18.8,      75.1])
ydata.append([ 45.2,     20.0,      72.3])
ydata.append([ 44.9,     21.2,      69.5])
ydata.append([ 45.3,     22.1,      66.6])
ydata.append([ 44.5,     22.8,      63.8])
ydata.append([ 44.9,     24.1,      60.9])
ydata.append([ 44.8,     25.2,      58.1])
ydata.append([ 45.3,     26.2,      55.2])
ydata.append([ 44.9,     26.9,      52.3])
ydata.append([ 48.0,     17.2,      89.7])
ydata.append([ 47.7,     18.1,      86.9])
ydata.append([ 48.2,     18.9,      84.0])
ydata.append([ 48.3,     19.9,      81.2])
ydata.append([ 48.0,     20.0,      78.3])
ydata.append([ 48.4,     22.8,      75.5])
ydata.append([ 48.2,     23.0,      72.7])
ydata.append([ 47.6,     24.0,      69.8])
ydata.append([ 48.2,     25.1,      66.9])
ydata.append([ 48.2,     25.8,      64.1])
ydata.append([ 47.8,     26.9,      61.2])
ydata.append([ 50.5,     17.0,      98.6])
ydata.append([ 50.6,     18.0,      95.8])
ydata.append([ 50.5,     19.1,      93.0])
ydata.append([ 50.9,     20.1,      90.1])
ydata.append([ 51.0,     20.8,      87.3])
ydata.append([ 51.0,     21.9,      84.4])
ydata.append([ 50.5,     23.0,      81.6])
ydata.append([ 51.2,     24.1,      78.7])
ydata.append([ 50.8,     25.2,      75.9])
ydata.append([ 51.2,     25.9,      73.0])
ydata.append([ 51.0,     26.9,      70.1])
ydata.append([ 54.2,     17.1,     107.6])
ydata.append([ 53.5,     18.1,     104.8])
ydata.append([ 54.1,     19.1,     101.9])
ydata.append([ 53.5,     20.1,      99.1])
ydata.append([ 53.5,     21.2,      96.3])
ydata.append([ 54.2,     21.9,      93.4])
ydata.append([ 54.1,     23.2,      90.6])
ydata.append([ 53.8,     24.1,      87.7])
ydata.append([ 53.6,     25.0,      84.8])
ydata.append([ 53.9,     26.0,      82.0])
ydata.append([ 53.7,     26.8,      79.1])
ydata.append([ 57.2,     17.0,     116.7])
ydata.append([ 56.9,     18.1,     113.8])
ydata.append([ 57.0,     19.1,     111.0])
ydata.append([ 56.7,     19.8,     108.1])
ydata.append([ 57.3,     20.9,     105.3])
ydata.append([ 56.8,     21.9,     102.4])
ydata.append([ 57.4,     23.2,      99.6])
ydata.append([ 57.3,     23.8,      96.7])
ydata.append([ 56.6,     25.2,      93.8])
ydata.append([ 57.0,     25.8,      91.0])
ydata.append([ 57.2,     27.0,      88.1])
ydata.append([ 60.2,     16.9,     125.7])
ydata.append([ 59.9,     17.9,     122.9])
ydata.append([ 59.6,     19.1,     120.0])
ydata.append([ 60.2,     20.1,     117.2])
ydata.append([ 60.2,     20.9,     114.3])
ydata.append([ 60.4,     21.8,     111.5])
ydata.append([ 59.1,     22.9,     108.6])
ydata.append([ 59.1,     24.1,     105.8])
ydata.append([ 60.1,     25.2,     102.9])
ydata.append([ 59.1,     25.8,     100.0])
ydata.append([ 60.3,     26.9,      97.1])

```

```
#=========================================
# random 80 % / 20 % split into training and validation sets
#=========================================
yalldata = deepcopy(ydata)
NALL = len(yalldata)                  # 122 data points
NV = int(round(0.2*NALL))             # 24 validation points
ND = NALL - NV                        # 98 training points
NS = ND                               # number of DNA strands = number of training points
DI = 3                                # number of data items in vector

rng_split = np.random.RandomState(2026)          # fixed seed -> reproducible split
perm = rng_split.permutation(NALL)
ival = sorted(perm[:NV])                         # indices of validation points
itrain = sorted(perm[NV:])                       # indices of training points
ydata = [yalldata[i] for i in itrain]            # training data used by the GA
vdata = [yalldata[i] for i in ival]              # validation data (never used in the GA)

print('total data points NALL =', NALL, ' training ND =', ND, ' validation NV =', NV)
print('validation set indices:', ival)

#end CodeP2.3F26

# ------------------------------ Cell 2 ------------------------------
'''>>>>> start CodeP2.2F26
    V.P. Carey ME249, Fall 2026
    MODIFIED for Tasks 2.4 / 2.5 (heat pipe model, Eqs. 2.3-2.6) - copy of the Task 2.2
    program in which:
      - the data are the heat pipe data of CodeP2.3F26 (training set ydata, ND points)
      - n1 = (UA)e, n2 = (UA)c, n3 = alpha ; n0 = -1 and n4, n5 are not used
      - the error function (2 locations: fitness loop and population-average loop)
        is the fractional error of the heat transfer rate, Eqs. (2.4)-(2.5):
          aFerr_i = | qdot_data,i - qdot_model(n1,n2,n3,Ta_in,i,Ta_out,i) | / | qdot_data,i |
      - the mean absolute error of the validation set (vdata) is also computed at
        each generation with the population-average constants (not used for selection)
    and, as in Task 2.2:
      - the evolution loop of the skeleton code is placed, unchanged, inside a
        function run_GA(...) so that it can be re-run for several initial guesses,
        mutation amplitudes, numbers of generations and random seeds
      - the maximum mutation amplitude (0.085 in the skeleton) is now the
        argument MUT (relative amplitude, i.e. +/- MUT*100 %)
      - both random generators (python random and numpy.random) are seeded with
        the argument rseed so that every run is reproducible
      - the history of the population-average constants and of aFerrmeanavgn is
        returned, together with the generation at which aFerrmeanavgn first drops
        below the target value (convergence-speed indicator) and the number of
        survivors nkeep of each generation (selection diagnostic)
      - BUG FIX: the skeleton line "import random" re-binds the name random to the
        module and hides the function random() imported in CodeP2.1F26, so the
        initialisation n1i+0.001*random() raises "TypeError: 'module' object is
        not callable".  The line is replaced by an explicit import of the function.'''
from random import random, seed   # (was: import random)

sigma = 5.67e-8     # Stefan-Boltzmann constant, W m-2 K-4
hcref = 80.0        # reference convective coefficient of condenser fin section, W m-2 K-1

def hrad(Tain, Taout):
    '''radiation coefficient h_r = sigma (Tin + Tout)(Tin^2 + Tout^2), absolute temperatures'''
    Ti = Tain + 273.15
    To = Taout + 273.15
    return sigma*(Ti + To)*(Ti**2 + To**2)

def qdot_model(n1, n2, n3, Tain, Taout):
    '''Eq. (2.6): qdot = n2 (1 + n3 hr/hcref) [ n1 (Tain - Taout) / ( n1 + n2 (1 + n3 hr/hcref) ) ]'''
    UAcr = n2*(1. + n3*hrad(Tain, Taout)/hcref)
    return UAcr*( n1*(Tain - Taout)/(n1 + UAcr) )

def run_GA(n1i, n2i, n3i, n4i=1.0, n5i=1.0, NGEN=6000, MUT=0.09, MFRAC=0.5,
           rseed=1, target=0.045, verbose=True):

    '''INITIALIZING PARAMETERS'''
    seed(rseed)                 # python random  (initial population)
    numpy.random.seed(rseed)    # numpy random   (mating / mutation)
    n = []
    ntemp = []
    gen=[0]
```

```
    n1avg = [0.0]
    n2avg = [0.0]
    n3avg = [0.0]
    n4avg = [0.0]
    n5avg = [0.0]
    meanAFerr=[0.0]
    aFerrmeanavgn=[0.0]
    aFerrval=[0.0]

    # here the number of data vectors equals the number of DNA strands (or organisms)
    for k in range(NGEN-1):
        gen.append(k+1)   # generation array stores the
        meanAFerr.append(0.0)
        aFerrmeanavgn.append(0.0)
        aFerrval.append(0.0)
        n1avg.append(0.0)
        n2avg.append(0.0)
        n3avg.append(0.0)
        n4avg.append(0.0)
        n5avg.append(0.0)

    '''guesses for initial solution population: n0i = -1 (fixed), n1i ... n5i are arguments'''
    n0i =  -1.0

    #i initialize array where rows are dna vectors [n0i,n1i,...n5i] with random pertubations
    n =  [[-1., n1i+0.001*random(), n2i+0.1*random(), n3i+0.0001*random(), n4i+0.0001*random(), n5i+0.0001*
     random()]]
    for i in range(ND):
        n.append([-1., n1i+0.0001*random(), n2i+0.001*random(), n3i+0.0001*random(), n4i+0.0001*random(), n5i
     +0.0001*random()])

    # store also in wtemp
    ntemp = deepcopy(n)

    #define arrays of Ferr (error) functions
    Ferr =  [[0.0]]
    Ferravgn =  [[0.0]]
    aFerr =  [[0.0]]
    aFerravgn =  [[0.0]]
    for i in range(ND-1):
        Ferr.append([0.0])
        aFerr.append([0.0])
        Ferravgn.append([0.0])
        aFerravgn.append([0.0])

    aFerrmeanavgnMin=1000000000.0
    # these store the  n values for minimum population average error durng NGEN generations
    n1min = 0.0
    n2min = 0.0
    n3min = 0.0
    n4min = 0.0
    n5min = 0.0
    kmin = 0
    aFerrvalmin = 0.0
    aFerrta = 0.0
    # these store the time averaged n values durng from generation 800 to NGEN generations
    n1ta = 0.0
    n2ta = 0.0
    n3ta = 0.0
    n4ta = 0.0
    n5ta = 0.0
    kconv = -1      # first generation with aFerrmeanavgn < target (-1 = never)
    nkeephist = []  # (added) number of survivors nkeep at each generation

    '''START OF EVOLUTION LOOP'''
    for k in range(NGEN):

        '''CALCULATING ERROR (FITNESS) - Eq. (1.3): organism i is paired with data point i'''
        for i in range(ND):

            # Eq. (2.4): data heat transfer rate minus model heat transfer rate
            Ferr[i] = ydata[i][2] - qdot_model(n[i][1], n[i][2], n[i][3], ydata[i][0], ydata[i][1])

            aFerr[i] = abs(Ferr[i])/abs(ydata[i][2])  #- absolute fractional error
        #-------------
```

```
        aFerrmean = numpy.mean(aFerr) #mean error for population for this generation
        meanAFerr[k]=aFerrmean  #store aFerrmean for this generation gen[k]=k
        aFerrmedian = numpy.median(aFerr) #median error for population for this generation

        '''SELECTION'''
        clim = MFRAC*aFerrmedian  #cut off limit is a fraction/multiplier MFRAC of median value for population
        nkeep = 0
        for j in range(NS):  # NS Ferr values, one for each solution in population
            if (aFerr[j] < clim):
                nkeep = nkeep + 1
                #ntemp[nkeep][0] = n[j][0] = -1 so it is unchanged;
                ntemp[nkeep-1][1] = n[j][1];
                ntemp[nkeep-1][2] = n[j][2];
                ntemp[nkeep-1][3] = n[j][3];
                ntemp[nkeep-1][4] = n[j][4];
                ntemp[nkeep-1][5] = n[j][5];
        #compute number to be added by mating
        nnew = NS - nkeep
        nkeephist.append(nkeep)   # (added) number of survivors, diagnostic of selection pressure

        '''MATING'''
        for j in range(nnew):
            # pick two survivors randomly
            nmate1 = numpy.random.randint(low=0, high=nkeep+1)
            nmate2 = numpy.random.randint(low=0, high=nkeep+1)

            '''here, do not change property ntemp[nkeep+j+1][0], it's always fixed at the value -1'''
            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][1] = n[nmate1][1]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 1,
    mutation added
            else:
                ntemp[nkeep+j+1][1] = n[nmate2][1]*(1.+MUT*2.*(0.5-numpy.random.rand()))

            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][2] = n[nmate1][2]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 2,
    mutation added
            else:
                ntemp[nkeep+j+1][2] = n[nmate2][2]*(1.+MUT*2.*(0.5-numpy.random.rand()))

            if (numpy.random.rand() < 0.5):
                ntemp[nkeep+j+1][3] = n[nmate1][3]*(1.+MUT*2.*(0.5-numpy.random.rand()))  # property 3,
    mutation added
            else:
                ntemp[nkeep+j+1][3] = n[nmate2][3]*(1.+MUT*2.*(0.5-numpy.random.rand()))
            # properties 4 and 5 are not used by the heat pipe model
        #============================================
        n = deepcopy(ntemp)   # save ntemp as n for use in next generation (next iteration)

        '''AVERAGING OVER POPULATION AND OVER TIME, FINDING MINIMUM ERROR SET OF n VALUES'''
        n1avg[k] = 0.0;
        n2avg[k] = 0.0;
        n3avg[k] = 0.0;
        n4avg[k] = 0.0;
        n5avg[k] = 0.0;
        for j in range(NS):
            n1avg[k] = n1avg[k] + n[j][1]/NS;
            n2avg[k] = n2avg[k] + n[j][2]/NS;
            n3avg[k] = n3avg[k] + n[j][3]/NS;
            n4avg[k] = n4avg[k] + n[j][4]/NS;
            n5avg[k] = n5avg[k] + n[j][5]/NS;

        ''' CALCULATING MEAN ERROR FOR POPULATION (population-average constants vs all data)'''
        for i in range(ND):
            Ferravgn[i] = ydata[i][2] - qdot_model(n1avg[k], n2avg[k], n3avg[k], ydata[i][0], ydata[i][1])

            aFerravgn[i] = abs(Ferravgn[i])/abs(ydata[i][2])
        #-------------
        aFerrmeanavgn[k] = numpy.mean(aFerravgn)
        # validation-set mean absolute fractional error with the same constants (monitoring only)
        aFerrval[k] = numpy.mean([abs(vdata[i][2] - qdot_model(n1avg[k], n2avg[k], n3avg[k], vdata[i][0],
    vdata[i][1]))/abs(vdata[i][2]) for i in range(NV)])
        if (kconv < 0 and aFerrmeanavgn[k] < target):
            kconv = k

        # time average of n values in population for generations k > 800
```

```
        aFerrta = aFerrta + aFerrmeanavgn[k]/NGEN
        if (k > 800):
            n1ta = n1ta + n1avg[k]/(NGEN-800)
            n2ta = n2ta + n2avg[k]/(NGEN-800)
            n3ta = n3ta + n3avg[k]/(NGEN-800)
            n4ta = n4ta + n4avg[k]/(NGEN-800)
            n5ta = n5ta + n5avg[k]/(NGEN-800)

        # save minimum population-average error and corresponding n(i) values
        if (aFerrmeanavgn[k] < aFerrmeanavgnMin):
            aFerrmeanavgnMin = aFerrmeanavgn[k]
            n1min = n1avg[k]
            n2min = n2avg[k]
            n3min = n3avg[k]
            n4min = n4avg[k]
            n5min = n5avg[k]
            aFerrvalmin = aFerrval[k]
            kmin = k
        '''end of evolution loop'''

    if verbose:
        print('ENDING: pop. avg n1-n3,aFerrmean:', n1avg[k], n2avg[k], n3avg[k], aFerrmeanavgn[k])
        print('MINUMUM:  avg n1-n3,aFerrmeanMin:', n1min, n2min, n3min, aFerrmeanavgnMin, ' at generation', kmin)
        print('          validation mean abs. error for these constants:', aFerrvalmin)
        print('TIME AVG:    avg n1-n3,aFerrmean:', n1ta, n2ta, n3ta, aFerrta)

    return dict(nkeephist=nkeephist, gen=gen, n1avg=n1avg, n2avg=n2avg, n3avg=n3avg, aFerrmeanavgn=aFerrmeanavgn,
                aFerrval=aFerrval, aFerrvalmin=aFerrvalmin,
                n1min=n1min, n2min=n2min, n3min=n3min, aFerrmeanavgnMin=aFerrmeanavgnMin,
                kmin=kmin, kconv=kconv, n1end=n1avg[k], n2end=n2avg[k], n3end=n3avg[k],
                aFerrend=aFerrmeanavgn[k], n1ta=n1ta, n2ta=n2ta, n3ta=n3ta, aFerrta=aFerrta)
# end CodeP2.2F26 (function definition)

# ------------------------------ Cell 3 ------------------------------
# identical to Program A, Cell 2 (listed above)

# ------------------------------ Cell 4 ------------------------------
'''Task 2.5 - 4 sets of initial guesses of order 1 for (n1, n2, n3) = ((UA)e, (UA)c, alpha), run with
   the skeleton selection (MFRAC = 0.5) and two mutation amplitudes, and with MFRAC = 1.0;
   then effect of the number of generations NGEN.'''
import time
SETS25 = {'Set 1': (1.0, 1.0, 1.0),
          'Set 2': (2.0, 2.0, 0.5),
          'Set 3': (5.0, 5.0, 1.0),
          'Set 4': (10.0, 3.0, 2.0)}
CONFIGS25 = [(0.09, 0.5), (0.30, 0.5), (0.09, 1.0)]       # (MUT, MFRAC)
SEEDS = [1, 2, 3]

def summary_hp(r, **kw):
    d = dict(F=r['aFerrmeanavgnMin'], Fval=r['aFerrvalmin'], n1=r['n1min'], n2=r['n2min'], n3=r['n3min'],
             kmin=r['kmin'],
             k045=first_below(r['aFerrmeanavgn'], 0.045), Fend=r['aFerrend'], nkeep_mean=float(numpy.mean(r['nkeephist'])),
             nkeep0=float(numpy.mean(numpy.array(r['nkeephist']) == 0)))
    d.update(kw)
    return d

t0 = time.time()
study25 = []
for name, init in SETS25.items():
    for (MUT, MF) in CONFIGS25:
        for s in SEEDS:
            r = run_GA(*init, NGEN=6000, MUT=MUT, MFRAC=MF, rseed=s, verbose=False)
            study25.append(summary_hp(r, set=name, init=init, MUT=MUT, MFRAC=MF, NGEN=6000, seed=s))
print('initial-guess study: %d runs in %.0f s' % (len(study25), time.time() - t0))

print('\n%-6s %-15s %-5s %-5s | %-8s %-8s %-8s | %-8s | %-5s | %-6s | %-7s' % ('set', 'init', 'MUT', 'MFRAC', 'Fbest', 'Fmedian', 'Fworst', 'Fval', '<.045', 'k045', 'nkeep=0'))
table25 = []
for name in SETS25:
    for (MUT, MF) in CONFIGS25:
        runs = [d for d in study25 if d['set'] == name and d['MUT'] == MUT and d['MFRAC'] == MF]
        Fs = numpy.array([d['F'] for d in runs]); b = runs[int(numpy.argmin(Fs))]
```

```
        ks = [d['k045'] for d in runs if d['k045'] >= 0]
        row = dict(set=name, init=SETS25[name], MUT=MUT, MFRAC=MF, nruns=len(runs), Fbest=float(Fs.min()), Fmed=float(numpy.median(Fs)),
                   Fworst=float(Fs.max()), Fval=b['Fval'], nsucc=int((Fs < 0.045).sum()), k045med=float(numpy.median(ks)) if ks else -1,
                   nkeep0=float(numpy.mean([d['nkeep0'] for d in runs])), n1=b['n1'], n2=b['n2'], n3=b['n3'], seed=b['seed'])
        table25.append(row)
        print('%-6s %-15s %-5s %-5s | %.5f  %.5f  %.5f | %.5f  | %d/%d   | %6s | %.2f   best: %.4f %.4f %.4f' % (name, str(SETS25[name]), MUT, MF,
              row['Fbest'], row['Fmed'], row['Fworst'], row['Fval'], row['nsucc'], row['nruns'], '%d' % row['k045med'] if ks else 'never',
              row['nkeep0'], b['n1'], b['n2'], b['n3']))

# effect of the number of generations
t0 = time.time()
ngen25 = []
for name, MUT, MF in [('Set 3', 0.09, 0.5), ('Set 1', 0.09, 1.0)]:
    for NG in [100, 500, 2000, 6000, 20000]:
        for s in SEEDS:
            r = run_GA(*SETS25[name], NGEN=NG, MUT=MUT, MFRAC=MF, rseed=s, verbose=False)
            ngen25.append(summary_hp(r, set=name, init=SETS25[name], MUT=MUT, MFRAC=MF, NGEN=NG, seed=s))
print('\nNGEN study: %d runs in %.0f s' % (len(ngen25), time.time() - t0))
for name, MUT, MF in [('Set 3', 0.09, 0.5), ('Set 1', 0.09, 1.0)]:
    for NG in [100, 500, 2000, 6000, 20000]:
        runs = [d for d in ngen25 if d['set'] == name and d['NGEN'] == NG]
        print('%s MUT %.2f MFRAC %.1f NGEN %5d : Fmin train = %s   Fval = %s' % (name, MUT, MF, NG,
              ' '.join('%.5f' % d['F'] for d in runs), ' '.join('%.5f' % d['Fval'] for d in runs)))

# ------------------------------ Cell 5 ------------------------------
'''Task 2.5 - best fit (lowest TRAINING error of all runs; the validation set is not used to choose it),
   replayed with the same seed; training and validation mean absolute errors; plots.'''
from mpl_toolkits.mplot3d import Axes3D   # noqa: F401  (registers the 3d projection)
bestrun = min(study25 + ngen25, key=lambda d: d['F'])
print('best run:', bestrun['set'], bestrun['init'], 'MUT =', bestrun['MUT'], 'MFRAC =', bestrun['MFRAC'], 'NGEN =', bestrun['NGEN'], 'seed =', bestrun['seed'])
rb = run_GA(*bestrun['init'], NGEN=bestrun['NGEN'], MUT=bestrun['MUT'], MFRAC=bestrun['MFRAC'], rseed=bestrun['seed'])
n1min, n2min, n3min = rb['n1min'], rb['n2min'], rb['n3min']

def mae(D, n1, n2, n3):
    return numpy.mean([abs(row[2] - qdot_model(n1, n2, n3, row[0], row[1])) / abs(row[2]) for row in D])

Ftr, Fva, Fall = mae(ydata, n1min, n2min, n3min), mae(vdata, n1min, n2min, n3min), mae(yalldata, n1min, n2min, n3min)
hrm = numpy.mean([hrad(row[0], row[1]) for row in yalldata])
UAeff = 1. / (1. / n1min + 1. / (n2min * (1. + n3min * hrm / hcref)))
print('\nBEST FIT: (UA)e = n1 = %.4f W/C   (UA)c = n2 = %.4f W/C   alpha = n3 = %.4f' % (n1min, n2min, n3min))
print('mean absolute fractional error: training %.5f   validation %.5f   all data %.5f' % (Ftr, Fva, Fall))
print('overall conductance at the mean radiation coefficient hr = %.3f W/m2K : UA_eff = %.4f W/C' % (hrm, UAeff))

# evolution of the error (training and validation) and of the constants
fig, ax = plt.subplots(figsize=(6.6, 4.6))
g = numpy.arange(1, len(rb['gen']) + 1)
ax.plot(g, rb['aFerrmeanavgn'], color=C[0], lw=0.8, label='training error')
ax.plot(g, rb['aFerrval'], color=C[7], lw=0.8, alpha=0.8, label='validation error')
ax.plot(g, rb['n1avg'], color=C[1], lw=1.3, label=r'$n_1=(UA)_e$ avg')
ax.plot(g, rb['n2avg'], color=C[2], lw=1.3, label=r'$n_2=(UA)_c$ avg')
ax.plot(g, rb['n3avg'], color=C[6], lw=1.3, label=r'$n_3=\alpha$ avg')
ax.axhline(0.045, color=INK2, ls='--', lw=0.9); ax.text(1.2, 0.049, 'target 0.045', color=INK2, fontsize=9)
ax.set_xscale('log'); ax.set_yscale('log'); ax.set_xlim(1, len(g)); ax.set_ylim(0.005, 30)
ax.set_xlabel('generation'); ax.set_ylabel('population-average constants and error')
ax.set_title('Task 2.5: best-fit run (%s, MUT = %.2f, MFRAC = %.1f)' % (bestrun['set'], bestrun['MUT'], bestrun['MFRAC']))
ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.16), ncol=3)
savefig('t25_evolution')

# log-log plot of predicted vs data heat transfer rate: training (circles) and validation (squares)
fig, axs = plt.subplots(1, 2, figsize=(11, 4.8), sharey=True)
xx = numpy.array([5, 200])
for a, D, mk, col, title in [(axs[0], ydata, 'o', C[0], '(a) training data (%d points), MAE = %.4f' % (ND, Ftr)),
```

```
                            (axs[1], vdata, 's', C[1], '(b) validation data (%d points), MAE = %.4f' % (NV, Fva))]:
    qd = [row[2] for row in D]; qp = [qdot_model(n1min, n2min, n3min, row[0], row[1]) for row in D]
    a.plot(xx, xx, '-', color=INK2, lw=1, label='perfect fit (slope 1)')
    a.plot(xx, 1.1 * xx, '--', color=INK2, lw=0.8, label=r'$\pm 10\,\%$')
    a.plot(xx, 0.9 * xx, '--', color=INK2, lw=0.8)
    a.loglog(qd, qp, mk, color=col, ms=6.5, mec='white', mew=0.7, label='data points')
    a.set_xlim(7, 150); a.set_ylim(7, 150)
    plain_log_ticks(a, [10, 20, 50, 100, 150], [10, 20, 50, 100, 150])
    a.set_xlabel(r'data heat transfer rate $\dot q_{data}$ (W)'); a.set_title(title)
    a.legend(loc='upper left')
axs[0].set_ylabel(r'predicted heat transfer rate $\dot q$ (W), Eq. (2.6)')
savefig('t25_parity')

# surface plot of qdot versus Ta,in and Ta,out over the range of the data
Yall = numpy.array(yalldata)
Ti_g = numpy.linspace(Yall[:, 0].min(), Yall[:, 0].max(), 50)
To_g = numpy.linspace(Yall[:, 1].min(), Yall[:, 1].max(), 50)
TI, TO = numpy.meshgrid(Ti_g, To_g)
QS = qdot_model(n1min, n2min, n3min, TI, TO)
fig = plt.figure(figsize=(12, 5.0))
ax = fig.add_subplot(1, 2, 1, projection='3d')
ax.plot_surface(TI, TO, QS, cmap=BLUES, linewidth=0.2, edgecolor='white', rstride=3, cstride=3, alpha=0.9)
Yt, Yv = numpy.array(ydata), numpy.array(vdata)
ax.scatter(Yt[:, 0], Yt[:, 1], Yt[:, 2], c=C[1], s=10, depthshade=False, label='training data')
ax.scatter(Yv[:, 0], Yv[:, 1], Yv[:, 2], c=INK, marker='s', s=14, depthshade=False, label='validation data')
ax.set_xlabel(r'$T_{a,in}$ (°C)', labelpad=6); ax.set_ylabel(r'$T_{a,out}$ (°C)', labelpad=6)
ax.view_init(elev=22, azim=-125)
ax.legend(loc='upper left', fontsize=8.5)
ax.set_title(r'(a) $\dot q$ (W) from Eq. (2.6) with best-fit constants')
a2 = fig.add_subplot(1, 2, 2)
cs = a2.contourf(TI, TO, QS, levels=numpy.arange(0, 135, 10), cmap=BLUES)
cl = a2.contour(TI, TO, QS, levels=numpy.arange(0, 135, 20), colors='white', linewidths=0.7)
a2.clabel(cl, fmt='%d W', fontsize=8)
a2.plot(Yt[:, 0], Yt[:, 1], 'o', color=C[1], ms=4, mec='white', mew=0.4, label='training data')
a2.plot(Yv[:, 0], Yv[:, 1], 's', color=INK, ms=4.5, mec='white', mew=0.4, label='validation data')
cb = fig.colorbar(cs, ax=a2); cb.set_label(r'$\dot q$ (W)')
a2.set_xlabel(r'cabinet air temperature $T_{a,in}$ (°C)'); a2.set_ylabel(r'outside air temperature $T_{a,out}$ (°C)')
a2.set_title('(b) contour map and data locations'); a2.grid(False)
a2.legend(loc='upper center', bbox_to_anchor=(0.5, -0.13), ncol=2, fontsize=8.5)
savefig('t25_surface')

# survivors per generation: frozen population when all errors are similar
fig, ax = plt.subplots(figsize=(6.6, 3.6))
for c, (name, MF) in enumerate([('Set 1', 0.5), ('Set 1', 1.0), ('Set 3', 0.5)]):
    r = run_GA(*SETS25[name], NGEN=300, MUT=0.09, MFRAC=MF, rseed=1, verbose=False)
    ax.plot(numpy.arange(1, 301), r['nkeephist'], color=C[[7, 0, 2][c]], lw=1.3,
            label='%s %s, MFRAC = %.1f  (min error %.3f)' % (name, SETS25[name], MF, r['aFerrmeanavgnMin']))
ax.set_xlabel('generation'); ax.set_ylabel('number of survivors nkeep')
ax.set_xlim(1, 300); ax.set_ylim(-2, ND)
ax.legend(loc='upper right')
savefig('t25_nkeep')

json.dump(dict(best=dict(n1=n1min, n2=n2min, n3=n3min, Ftrain=Ftr, Fval=Fva, Fall=Fall, UAeff=UAeff, hr_mean=hrm, kmin=rb['kmin'],
                         set=bestrun['set'], init=bestrun['init'], MUT=bestrun['MUT'], MFRAC=bestrun['MFRAC'], NGEN=bestrun['NGEN'], seed=bestrun['seed'],
                         n1end=rb['n1end'], n2end=rb['n2end'], n3end=rb['n3end'], Fend=rb['aFerrend'], n1ta=rb['n1ta'], n2ta=rb['n2ta'], n3ta=rb['n3ta']),
               split=dict(NALL=NALL, ND=ND, NV=NV, ival=[int(i) for i in ival]), table=table25, runs=study25, ngen=ngen25),
          open(RESDIR + 'task25.json', 'w'), indent=1)

# ------------------------------ Cell 6 ------------------------------
'''(added) Task 2.5 - are (UA)e, (UA)c and alpha individually determined by the data?
   (a) minimum training error when n1 = (UA)e is fixed and n2, n3 are re-optimised (deterministic
       minimisation of the same objective);  (b) constants found by all converged GA runs.'''
from scipy.optimize import minimize
Ytr = numpy.array(ydata)
Ti_t, To_t, q_t = Ytr[:, 0], Ytr[:, 1], Ytr[:, 2]

def Ftrain(n1, n2, n3):
    if n1 <= 0 or n2 <= 0:
        return 1e9
```

```
    return numpy.mean(numpy.abs(q_t - qdot_model(n1, n2, n3, Ti_t, To_t)) / q_t)

best = None
for x0 in [(5, 5, 1), (10, 3, 2), (3, 20, 1), (50, 3, 1)]:
    rr = minimize(lambda x: Ftrain(*x), x0, method='Nelder-Mead', options=dict(xatol=1e-10, fatol=1e-13,
     maxiter=20000))
    if best is None or rr.fun < best.fun:
        best = rr
print('reference minimum (training set): n1 = %.4f  n2 = %.4f  n3 = %.4f   MAE = %.6f' % (*best.x, best.fun))

n1fix = numpy.logspace(numpy.log10(3.2), 3, 25)
pf = []
for v in n1fix:
    bb = None
    for x0 in [(3, 1), (5, 2), (10, 5), (3, 0.1)]:
        rr = minimize(lambda y: Ftrain(v, y[0], y[1]), x0, method='Nelder-Mead', options=dict(xatol=1e-10,
     fatol=1e-13, maxiter=20000))
        if bb is None or rr.fun < bb.fun:
            bb = rr
    pf.append((v, bb.x[0], bb.x[1], bb.fun))
pf = numpy.array(pf)
for row in pf[::4]:
    print('n1 fixed = %8.3f -> n2 = %7.4f  n3 = %7.4f  training MAE = %.6f' % tuple(row))

conv = [d for d in study25 + ngen25 if d['F'] < 0.02]
fig, (a1, a2) = plt.subplots(1, 2, figsize=(11, 4.3))
a1.semilogx(pf[:, 0], pf[:, 3], '-', color=C[0], lw=1.6, label=r'min over $(n_2,n_3)$ at fixed $n_1$')
a1.semilogx([d['n1'] for d in conv], [d['F'] for d in conv], 'o', color=C[1], ms=6, mec='white', mew=0.7,
     label='GA runs (min error)')
a1.set_ylim(0.0164, 0.0172)
a1.set_xlabel(r'$n_1=(UA)_e$ (W/°C)'); a1.set_ylabel('training mean absolute error')
a1.set_title('(a) error valley: the minimum is flat in $n_1$')
a1.legend(loc='upper right')
a2.loglog(pf[:, 0], pf[:, 1], '-', color=C[0], lw=1.6, label=r'$n_2$ on the valley floor')
a2.loglog(pf[:, 0], pf[:, 2], '-', color=C[2], lw=1.6, label=r'$n_3=\alpha$ on the valley floor')
a2.loglog([d['n1'] for d in conv], [d['n2'] for d in conv], 'o', color=C[1], ms=6, mec='white', mew=0.7,
     label=r'GA $n_2$')
a2.loglog([d['n1'] for d in conv], [d['n3'] for d in conv], 's', color=C[6], ms=6, mec='white', mew=0.7,
     label=r'GA $n_3$')
a2.set_xlabel(r'$n_1=(UA)_e$ (W/°C)'); a2.set_ylabel(r'$n_2=(UA)_c$ (W/°C) and $n_3=\alpha$')
a2.set_title('(b) constants along the valley floor')
a2.legend(loc='upper right')
savefig('t25_degeneracy')
UAs = [1. / (1. / d['n1'] + 1. / (d['n2'] * (1. + d['n3'] * hrm / hcref))) for d in conv]
print('%d converged GA runs: n1 from %.2f to %.2f, n2 from %.2f to %.2f, n3 from %.2f to %.2f, but UA_eff
     from %.4f to %.4f W/C'
      % (len(conv), min(d['n1'] for d in conv), max(d['n1'] for d in conv), min(d['n2'] for d in conv), max(d[
     'n2'] for d in conv),
         min(d['n3'] for d in conv), max(d['n3'] for d in conv), min(UAs), max(UAs)))
res = json.load(open(RESDIR + 'task25.json'))
res['ref'] = dict(n1=best.x[0], n2=best.x[1], n3=best.x[2], F=best.fun, Fval=mae(vdata, *best.x))
res['profile'] = pf.tolist()
res['conv_range'] = dict(n=len(conv), n1=[min(d['n1'] for d in conv), max(d['n1'] for d in conv)], n2=[min(d['
     n2'] for d in conv), max(d['n2'] for d in conv)],
                         n3=[min(d['n3'] for d in conv), max(d['n3'] for d in conv)], UAeff=[min(UAs), max(UAs
     )])
json.dump(res, open(RESDIR + 'task25.json', 'w'), indent=1)
```